

UNIT 3:

Naive Bayes is a **supervised learning algorithm** (when we give the algorithm **examples with answers** (called **labels**), and it learns to **predict answers for new examples**.) used for **classification** tasks.

It is:

- Based on **Bayes' Theorem** (probability theory)
- We Called "Naïve" because it assumes that all features (like words in a sentence) **are independent from each other**.
- Especially useful in **text classification**, where you need to assign labels like *spam/ham*, *positive/negative*, or *sports/politics* to text data.

Points to note on the concept:

- **Naive Bayes** is a **probability-based classifier** --- **this means It** calculates probabilities **for each possible class** — like *Spam* or *Not Spam* — and then **chooses the one with the highest probability..**
- It assumes all features (words) are **independent**.
- Uses **Bayes' Theorem** to calculate the probability of each class.
- **Simple, fast**, and great for **text classification**.
- Works well for tasks like **spam detection**, **sentiment analysis**, etc.

Let us say we have RED, ROUND, SWEET in the basket and now if we look into this we can guess tomato, apple, cheery fruit, the probability is APPLE ONLY. Some times we have the mails showing SPAM though It is the INBOX but to check how it is we look into the $P(\text{CLASS} | \text{TEXT})$.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

- $P(A|B)$ → Probability of event A occurring given B is true.
- $P(B|A)$ → Probability of event B occurring given A is true.
- $P(A)$ → Probability of event A occurring (prior)
- $P(B)$ → Probability of event B occurring

In Naïve Bayes (Adapted Formula for Text Classification):

$$P(\text{Category}|\text{Text}) = \frac{P(\text{Text}|\text{Category}) \cdot P(\text{Category})}{P(\text{Text})}$$

$P(\text{Category}|\text{Text})$: Probability that a text belongs to a category

$P(\text{Text}|\text{Category})$: Probability of the text appearing in that category

$P(\text{Category})$: Probability of the category itself (prior probability)

$P(\text{Text})$: Probability of the text (often ignored because it's the same across all categories)

Email	Text	Category
E1	"win prize now"	Spam
E2	"free win cash"	Spam
E3	"project update"	Not Spam
E4	"meeting today"	Not Spam

What is Text Classification?

Text classification is a task in **Natural Language Processing (NLP)** where we want to assign a **category/label** to a given piece of text.

- Classify emails as **Spam** or **Not Spam**
- Detect if a review is **Positive** or **Negative**
- Categorize news articles as **Politics**, **Sports**, or **Technology**

For the email below let us predict how it works:

Steps:

1. Count frequency of each word in Spam and Not Spam emails.
2. Calculate:
 - **$P(\text{word}|\text{Spam})$** and **$P(\text{word}|\text{NotSpam})$**
3. Multiply the probabilities for the new email in each category.
4. Pick the category with **higher probability**.

Type	Description	Example Use
Multinomial NB	Works with word counts (how many times a word appears)	Text classification (e.g. spam detection)
Bernoulli NB	Works with binary features (word present or not)	Email filters
Gaussian NB	Works with continuous data	Medical diagnosis, sensor data

USE CASES INCLUSIVE OF NLP:

Application	Task
-------------	------

Application	Task
Email Clients	Spam detection
Product Reviews	Sentiment analysis
News Aggregators	Topic classification
Social Media	Detecting offensive or fake content
Support Bots	Categorizing customer queries

Naïve Bayes helps by using **word probabilities** from already-labeled data (called training data) to **predict the label** of a new, unseen text like the below one:

- **Frequency of words** in each category
- **Bayes' Theorem** to calculate **how likely a message belongs to a category**

Building the sentiment analysis:

Review	Label
"I love this product"	Positive
"This is a great item"	Positive
"Worst thing ever"	Negative
"I hate it"	Negative

For the above we can give the new review:

"love this item"

Working concept text classification:

Step-by-Step Process

1. **Extract Words (Tokenization)**
→ "love", "this", "item"
2. **Calculate Word Probabilities for Each Class**

Word	P(Word Positive)	P(Word Negative)
-----	-----	-----
love	0.4	0.05
this	0.2	0.1
item	0.3	0.1

3. Use Naïve Bayes Formula to calculate:

$$P(\text{Positive}|\text{Text}) \propto P(\text{love}|\text{Positive}) \cdot P(\text{this}|\text{Positive}) \cdot P(\text{item}|\text{Positive}) \cdot P(\text{Positive})$$

Similarly for Negative.

Assume:

- $P(\text{Positive}) = 0.5$
- $P(\text{Negative}) = 0.5$

$$P(\text{Positive}|\text{Text}) = 0.4 \cdot 0.2 \cdot 0.3 \cdot 0.5 = 0.012$$

$$P(\text{Negative}|\text{Text}) = 0.05 \cdot 0.1 \cdot 0.1 \cdot 0.5 = 0.00025$$

✓ Since $0.012 > 0.00025$, the model predicts the text **Positive**.

Benefits Of the Text Classifications with the Naïve Bayes concept:

Feature	Reason
Fast	Works with large text datasets quickly
Simple	Easy to understand and implement
Effective	Often gives good accuracy for basic NLP problems
Requires less training data	Great when labeled data is limited

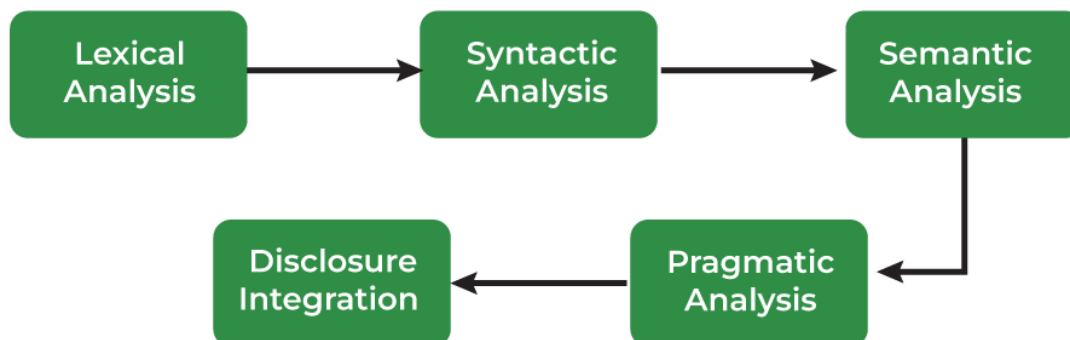
Feature	Bayes' Theorem	Naïve Bayes Classifier
Type	A mathematical formula	A machine learning algorithm
Purpose	Calculates conditional probability	Uses Bayes' Theorem to classify data
Formula	$P(A)$ (checks the occurrence)	$B) = P(B)$ checks both
Assumption	No assumptions about data	Assumes features are independent (called "naïve" assumption)
Used In	Statistics, probability theory	NLP, spam filtering, sentiment analysis, text classification
Output	Gives a probability value (e.g., 70% chance of rain)	Predicts a class (e.g., "Spam" or "Not Spam")
Example	What's the probability that it will rain given the sky is cloudy?	Given the words "free", "win", "prize" → is this email spam or not?
Training	Doesn't require training data	Needs training data with labels to learn from
Core Idea	Single event probability update	Applies Bayes' Theorem to many features and chooses the highest probability class
Example	I see and believe, but also can update after new happening	Here I will use the all independent features for finding out the result.

1. Every one or the group will pick a review (without label).
2. They refer to a pre-prepared list of known positive/negative words with frequencies.

3. Score the review:
 - +1 for each positive word
 - -1 for each negative word
4. If score > 0 : classify as **Positive**
If score < 0 : classify as **Negative**



Training and Optimization for sentiment analysis:



Lexical Analysis (Breaking Down Sentences)

- **What it does:** Splits a sentence into words and identifies their types (nouns, verbs, etc.).
- **Example:**
Sentence: "I love pizza."
→ Words: ["I", "love", "pizza"]
→ Tags: I (pronoun), love (verb), pizza (noun)

Syntactic Analysis (Grammar Check)

- **What it does:** Checks if the sentence follows correct grammar rules and builds a structure (like a tree).
- **Example:**
"Pizza love I" → Invalid order
"I love pizza" → Valid order

Semantic Analysis (Understanding Meaning)

- **What it does:** Understands the actual meaning of the words and sentence.
- **Example:**
"I love pizza" → Meaning: A positive feeling about food.
"I hate pizza" → Meaning: A negative feeling about food.

Pragmatic Analysis (Contextual Meaning)

- **What it does:** Understands what the speaker *really* means based on situation or tone.
- **Example:**
• "That pizza was sick!"
→ In a formal setting: It sounds bad.
→ Among friends: It might mean the pizza was awesome!

Discourse Integration (Linking Sentences Together)

- **What it does:** Connects current sentence to previous ones for full understanding.
- **Example:**
Text: "I was hungry. I ordered a pizza. It was delicious."
→ "It" refers to the pizza in the previous sentence.
→ Helps maintain flow and meaning across multiple lines.

PRE and POST STRUCTURE OF SENTIMENT ANALYSIS:

Step 1: Data Collection (Before Sentiment Analysis)



Step 2: Text Preprocessing



Step 3: Feature Extraction



Step 4: Sentiment Analysis (Classification)



Step 5: Post-Processing / Applications

Data Collection

Connected with: Web scraping, APIs, Data Warehousing

- Collect text data from:
 - Product reviews
 - Tweets, Facebook posts
 - Surveys or feedback forms
 - News articles

Pre-Processing the Text

Connected with: Text Normalization Techniques

Task	Example	Method
Lowercasing	"GREAT" → "great"	Normalization
Removing stop words	"I am happy" → "happy"	Token Filtering
Lemmatization	"running" → "run"	Morphological analysis
Tokenization	"I love it" → [I, love, it]	Tokenizer

Feature Extraction

Connected with: NLP Vectorization Methods, CONVERT TEXT to NUMBERS

Method	What it Does
Bag of Words	Count how many times each word appears
TF-IDF	Give importance to unique words
Word Embeddings (Word2Vec, GloVe)	Capture meaning and context in vector form

Sentiment Analysis (The Main Part)

Connected with: Supervised Machine Learning

Common classifiers used:

- **Naive Bayes**
- **Logistic Regression**
- **SVM**
- **Deep Learning (LSTM, CNN)**

The model predicts if the sentence is:

- **Positive**
- **Negative**
- **Neutral**

Sentiment Analysis is the process of identifying emotions or opinions in a text (positive, negative, or neutral).

Example:

- “I love this phone!” → Positive
- “This food is terrible.” → Negative
- “It is a phone.” → Neutral

Training a Sentiment Analysis Model

Training means teaching a computer to **understand emotions** in text using labeled data.

Step 1: Collect the Data

- Use a dataset of **text + labeled emotions**
- Example:

Review Text	Sentiment
“I love this laptop.”	Positive
“Worst movie ever watched.”	Negative
“It was okay, not great.”	Neutral

Step 2: Preprocessing

- Clean the text:
 - Remove punctuation: “I love this!” → “I love this”
 - Convert to lowercase: “Love” → “love”
 - Remove stopwords: “I, the, is, and...”

Step 3: Feature Extraction

- Convert words into numbers (because ML models work with numbers).

Example techniques:

- **Bag of Words (BoW)** – Count how often each word appears.
- **TF-IDF** – Give importance to unique words.
- **Word Embeddings** – Convert words into vector forms (like Word2Vec, GloVe).

Step 4: Choose a Model

- Use a **supervised learning algorithm** like:
 - Naïve Bayes → >>>>>>>explain in detail about the NAIVES BAYES
 - Logistic Regression
 - SVM
 - Neural Networks

Step 5: Train the Model

- Feed the text and labels into the algorithm.
- The model learns patterns like:

“love” → positive
“worst” → negative

OPTIMIZATION:

Optimization means **improving the model** so that it makes better predictions.

Optimization means improving the model so that it makes better predictions. Here are some common optimization techniques used in machine learning, especially in text classification and sentiment analysis tasks, with simple explanations and real-time examples.

Optimization Technique	Simple Explanation	Real-Time Example (Sentiment Analysis)	Why It Helps
Hyperparameter Tuning	Adjust settings like learning rate , number of iterations , etc., to improve accuracy.	While training a Naïve Bayes model on Amazon product reviews , try different smoothing values to avoid zero probabilities for rare words like 'phenomenal'.	Fine-tunes the model to increase accuracy using the best settings .
Cross Validation	Split data into multiple parts and train/test on different combinations.	Split 1000 restaurant reviews into 5 parts, training on 4 and testing on 1 in each round.	Prevents overfitting by checking performance on different data chunks.

Feature Selection	Remove unhelpful or noisy words, keeping only useful ones.	Remove words like 'the', 'movie' and keep strong indicators like 'boring', 'fantastic' in movie reviews.	Speeds up the model and improves accuracy by focusing on valuable features.
Grid Search	Try all combinations of hyperparameters and pick the best one.	Test different logistic regression settings like penalty types and regularization values for hotel reviews.	Finds the most effective hyperparameter settings automatically.
Early Stopping	Stop training if model starts to perform worse on test data.	In an LSTM network, stop training after 10 epochs if accuracy starts dropping.	Prevents overfitting and saves training time.

Evaluation Metrics for Sentiment Analysis

When we train a model for tasks like sentiment analysis, we need to check how well the model performs. Here are some common evaluation metrics used, along with their simple meanings and real-time discussion for better understanding:

Metric	What it Tells	Formula	Real-Time Example	Why It's Important
Accuracy	Correct predictions out of total predictions made by the model.	$Accuracy = (TP + TN) / (TP + TN + FP + FN)$	Out of 100 reviews, 85 were correctly labeled → Accuracy = 85%.	Good for balanced data, gives an overall picture.
Precision	Out of all positive predictions, how many were correct.	$Precision = TP / (TP + FP)$	Predicted 50 positives, 40 were actually positive → Precision = 80%.	Important when false positives are risky.
Recall	Out of actual positives, how many were caught.	$Recall = TP / (TP + FN)$	There were 60 real positives, caught 48 → Recall = 80%.	Helps avoid missing important positive cases.
F1 Score	Balanced average of Precision and Recall.	$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$	Precision = 80%, Recall = 70% → F1 ≈ 74.8%	Best when data is imbalanced.

Post Preprocessing and Applications:

Use Case	How It's Connected
Business Reports	Create pie charts of customer moods
Recommendation Systems	Suggest products based on positive reviews
Chatbots	React based on user emotion
Social Media Monitoring	Alert if negative trends rise

Introduction information retrieval:

What is Information Retrieval (IR)?

IR is the process of **obtaining relevant information** from a large repository of data (usually unstructured), based on a user's query.

But we should understand that **Information Retrieval $\neq$ Data Retrieval**.

Data retrieval fetches **exact matches** (like in databases), while Information Retrieval deals with **approximate matches** using relevance.

What is an Information Retrieval System (IRS)?

An Information Retrieval System is a **software system** that stores, indexes, and retrieves information from data sources, based on user needs.

Main Goal: Return **relevant documents** or results that best match the user's query.

Basic Architecture of IRS:

1. **Document Collection** – Source data (web pages, books, PDFs, etc.)
2. **Preprocessing** – Cleaning, removing stop words, stemming, tokenizing.
3. **Indexing** – Creating an efficient structure to retrieve data faster.
4. **Query Processing** – Understand user query.
5. **Matching** – Comparing query with indexed data.
6. **Ranking** – Sorting results based on relevance.
7. **Presentation** – Displaying results to the user.

1. Document Collection

- This is the **input source** for the IR system.
- It consists of a large set of unstructured documents such as:
 - Web pages
 - Articles
 - Research papers
 - PDFs
 - Emails or blogs
- These documents are stored in a central **repository or database** to be later searched.

2. Preprocessing

This step prepares the documents for indexing. It includes:

- **Tokenization:**
Breaking text into small units like words or terms (called tokens).
E.g., “Information Retrieval System” → [“Information”, “Retrieval”, “System”]
- **Stop Word Removal:**
Removes common but meaningless words like is, the, and, on that add noise to results.

- **Stemming:**
Reduces words to their root forms.
E.g., “running”, “runner”, “runs” → “run”

This step improves both storage efficiency and search quality.

3. Indexing

- Creates a **search-friendly data structure**, often an **inverted index**, where:
 - Each word maps to a list of documents in which it appears.
- **Example:**

"retrieval" → [doc1, doc3, doc7]

Without this step, the system would have to search every word in every document in real time (which is slow).

4. Query Input

- This is the **user's search text** (like what we type in Google).
 - Example: “Top AI colleges in India”
- It's the **starting point of retrieval** from the user side.

5. Query Processing

- The query undergoes **preprocessing similar to the documents**:
 - Tokenization
 - Stop-word removal
 - Possibly **query expansion** (e.g., adding synonyms)
 - Spelling correction, etc.
- The goal is to match the query terms to the indexed documents more accurately.

6. Matching Engine

- This component **compares** the processed query with the indexed documents.
- It finds **all documents containing terms that match** the query terms.
- This is a **filtering** step—only relevant documents are passed on for ranking.

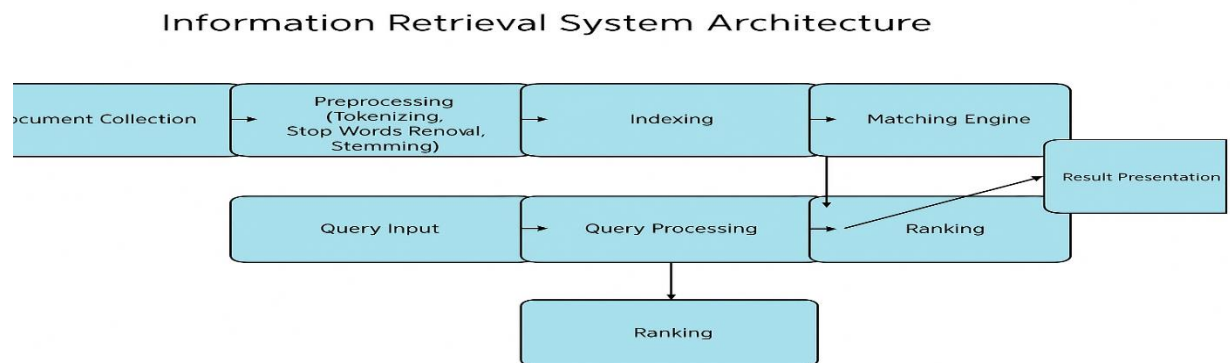
7. Ranking

- All matched documents are **scored** based on how relevant they are to the query.
- Some ranking models include:
 - **TF-IDF** (Term Frequency – Inverse Document Frequency)
 - **BM25** (probabilistic model)
 - **Vector Space Model**
- Documents are sorted in **descending order of relevance score**.

8. Result Presentation

- The final output shown to the user:
 - Document titles

- Snippets (sample content)
- URLs or links
- Date, author, file type, etc.
- May include options like **filters, categories, or pagination**.



Characteristics of a Good IRS:

- High **precision** (returns only relevant documents).
- High **recall** (returns all relevant documents).
- Fast and scalable.
- Understands user needs.
- Adapts to new data.

Real-Time Example 1: Google Search

- Google is the most powerful INFORMATION RETRIEVAL SYSTEM.
- When you type a query like “top tourist places in India,” it:
 - Understands the intent **using AI and NLP**.
 - Searches billions of indexed documents.
 - Uses ranking algorithms (like PageRank) to show the most relevant results.

Real-Time Example 2: Netflix Recommendation Engine

- Netflix suggests **movies based on your watch history**.
- It uses **collaborative filtering** (user behavior similarity) and **content-based filtering** (movie metadata).
- This is a form of **personalized IR**, using intelligent methods.

Applications of Information Retrieval Systems:

- Search Engines (Google, Bing, Yahoo)
- Digital Libraries
- Recommender Systems (Amazon, Netflix, Spotify)
- E-learning Platforms (Coursera, edX)

- Chatbots and Virtual Assistants (ChatGPT, Siri, Alexa)

Understanding the goals of IR in NLP:

Goal	Meaning
Understand the query	What is the user asking for?
Match documents	Find documents that answer the query
Rank results	Show the most helpful ones first
Support natural language	Accept full sentences, not just keywords

Application of both IR and NLP together:

Application	IR + NLP in Action
Google Search	Understands full questions, not just keywords
Chatbots	Finds correct answers from help documents
Voice Assistants	Finds and speaks answers from large databases
E-commerce Search	Shows most relevant products by meaning
Academic Search Tools	Finds research papers related to a given topic

Information Retrieval models

What is an Information Retrieval (IR) Model?

An **Information Retrieval model** is a **mathematical or logical system** used to **find the most relevant documents** from a large collection based on a **user's query**.

Think of it like a **search engine brain** – it **decides how to match** what the user types (like keywords) with documents (webpages, books, articles, etc.).

Why Do We Need IR Models?

- There are **millions of documents**. Here we have lot of documents
- People **search using short queries**, not full sentences. People give simple keywords
- We must **rank documents** from most relevant to least. What ever meets will be shown
- Models help to **score and rank** each document for the given query. Based on rank and score
- IR models help us find:
 - Accurate answers
 - Fast results
 - Efficient use of storage

Real Time Examples:

Platform	What IR Model Does
----------	--------------------

Platform	What IR Model Does
Google Search	Finds most relevant webpages for your keywords
Gmail	Finds emails matching your search (by sender, subject, etc.)
YouTube	Matches your video search with the best titles/descriptions
Library Software	Finds books/articles based on title, author, subject
E-commerce sites	Shows products based on keywords or categories

Types of Information Retrieval Model:

Type	Based On	Example Models
1. Set-Theoretic	Binary (Yes/No) Matching	Boolean Model
2. Algebraic (Vector)	Ranking using similarity	Vector Space Model (VSM), LSI
3. Probabilistic	Ranking using probabilities	BM25, Binary Independence Model
4. Neural (ML-based)	Learning from past data	BERT, DSSM, DeepRank

1. Boolean Model (Set-Theoretic)

- Uses **AND, OR, NOT** operations.
- Result: **Exact match** (document either relevant or not).

Example:

Query: "AI AND Law"

Only documents having **both** "AI" and "Law" are shown.

- **Pros:** Simple and fast.
- **Cons:** No ranking, either yes or no.

2. Vector Space Model (VSM)

- Treats documents and queries as **vectors** in a space.

Formula:

$$\cos(\theta) = \frac{D \cdot Q}{||D|| \times ||Q||}$$

Where:

- **D** = document vector
- **Q** = query vector
- **.** = dot product
- **||D||** = magnitude of D
- Score lies between **0 (no match)** and **1 (perfect match)**

Example:

If Query = "apple iphone camera", documents having similar words in title/desc get **higher score**.

- **Pros:** Ranking possible, handles partial matches.
- **Cons:** Ignores word order and meaning.

3. **Probabilistic Model:** each document is probability relevant to the query.

Binary Independence Model (BIM):

Uses the idea:

$$P(R|Q) \propto \frac{P(Q|R)}{P(Q|\neg R)}$$

BM25 Model (Most popular):

BM25 is a scoring function:

$$BM25(D, Q) = \sum_{i=1}^n IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k + 1)}{f(q_i, D) + k(1 - b + b \cdot \frac{|D|}{avgDL})}$$

Where:

- $f(q_i, D)$ = frequency of term q_i in document D
- $|D|$ = length of document
- $avgDL$ = average document length
- k and b = tuning parameters
- **Pros:** Very effective for real-world IR.
- **Cons:** Complex formula.



Vector Space Model (Algebraic Model)..

This model treats each document and query as a **vector of numbers**. These numbers represent how often words appear. It uses **cosine similarity** to measure **how close** the query is to the document.

◆ Formula:

$$\cos(\theta) = \frac{D \cdot Q}{||D|| \cdot ||Q||}$$

Where:

- D = Document vector
- Q = Query vector
- Closer to 1 = more similar

Real-Life Example:

Imagine you're searching for "smartphone camera features".

- **Doc A** talks about camera quality and battery.
- **Doc B** talks about smart homes.

Doc A gets a higher **cosine similarity** score than Doc B.

Advantages:

- Gives a **ranked list** of results
- Works well for partial matches

Disadvantages:

- Doesn't understand word meanings
- Can't handle synonyms well

Term Weighting Schemes (TF, IDF, TF-IDF)

Words that occur often in a document are **important**, but if they occur in **every** document, they are **less useful**.

In Document 1: “AI is changing the world of technology”.

Document 2: “AI and Machine Learning are good concepts”.

Document 3: Cooking at home is better than eating outside.

In **Doc 1**, “AI” appears **1 time** out of 7 words → $TF = 1/7$

In **Doc 2**, “AI” appears **1 time** out of 7 words → $TF = 1/7$

In **Doc 3**, “AI” appears **0 times** → $TF = 0$

“AI” appears in **2 out of 3 documents**

$$IDF = \log \left(\frac{3}{2} \right) \approx 0.176$$

Appears **only in Doc 3**

$TF = 1/9$ (Doc 3 has 9 words)

$IDF = \log(3/1) \approx 0.477$

$TF-IDF = 1/9 \times 0.477 \approx 0.053$

“cooking” has a **higher TF-IDF** than “AI” in Doc 3 → It is **more unique and important** to that document.

This model assigns **weights** to each word to measure how important it is.

Term	Meaning
TF	Term Frequency – how many times a word appears in the document
IDF	Inverse Document Frequency – how rare the word is across all documents
TF-IDF	Combined measure of term importance in document and rarity in the collection

TF-IDF Formula:

$$\text{TF-IDF} = \text{TF} \times \log \left(\frac{N}{\text{DF}} \right)$$

ere:

N = Total number of documents

DF = Number of documents containing the word

Real-Life Example:

- In a job portal, words like "**Python**", "**AI**" might have **high TF-IDF** in a Data Science job ad.
- Words like "**the**", "**is**" have **low TF-IDF** (appear everywhere, not useful).

Probabilistic Retrieval Models (Binary Independence Model)

These models assume that **each document has a probability** of being relevant to a query. The idea is to **rank documents** by this probability.

The **Binary Independence Model** is a **probabilistic model** used in Information Retrieval (IR). It tries to calculate the **probability that a document is relevant** to a user's query, based on the presence or absence of each word (binary choice: yes or no)

The most basic one is **Binary Independence Model (BIM)**:

- Treats each word as **present or absent**.
- Assumes each word is **independent** from others.
- **Binary**: It only checks whether a word is **present or not** (not how many times).
- **Independence**: It assumes that each word in the query acts **independently** of the others.

Suppose you're searching for "cyber attack 2023".

- The BIM checks: Does the document contain "cyber"? Yes/No
- Then: Does it contain "attack"? Yes/No
- Then: "2023"? Yes/No

How it works?

The model calculates:

$$P(R|D) = \text{Probability that document D is relevant to query Q}$$

It uses training data or assumptions to calculate this.

It uses training data or assumptions to calculate this.

Real-Life Example:

- When you search for "**cyber attack India 2023**", the model gives higher probability to news articles from 2023 that mention those keywords.
- Documents that just mention "**India**" or "**attack**" may be given lower relevance scores.

Advantages:

- More intelligent than simple matching
- Supports ranking
- Can improve with user feedback

Disadvantages:

- Requires estimation of probabilities
- Needs some **training data**

Classical Information retrieval Model

Classical Information Retrieval Models are the earliest and most widely used models in the field of information retrieval.

Their main goal is to **find the most relevant documents** for a user's query from a collection.

They work by:

1. Representing documents and queries in a structured way.
2. Defining a **matching function** (yes / no) to compare them.
3. Ranking results according to relevance.

These models are “classical” because they form the **foundation** of modern IR systems like Google, Bing, and library databases.

Key Characteristics:

- **Mathematical foundation** – based on formal definitions and equations.
- **Clear representation** – documents and queries are represented in a consistent way.
- **Ranking capability** – allows sorting documents by relevance.
- **Efficiency** – designed for large-scale text retrieval.

Boolean Model

Definition

- A **set theory-based** model.
- Documents are retrieved **only if** they exactly match the query conditions using logical operators like **AND, OR, NOT**.
- Each document is either **relevant** or **not relevant** – no ranking in between.

How it works

1. Represent documents as sets of words (terms).
2. The query is expressed as a Boolean expression.
3. Retrieve only documents that satisfy the exact query condition.

Real-Time Example

- **Library Search:** A student wants books about "Machine Learning" but **not** about "Neural Networks".
Query: "Machine Learning" AND NOT "Neural Networks"
The system retrieves only books containing *Machine Learning* but excluding *Neural Networks*.

Vector Space Model (VSM)

Definition

- Uses **geometry and linear algebra** concepts.
- Documents and queries are represented as **vectors** in a multi-dimensional space.
- Each dimension represents a **term**.

- The **similarity** between document and query is calculated (often using **cosine similarity**).

How it works

1. This will create a **term-document matrix** where each value is a weight (like TF-IDF score).
2. This will represent query as a vector.
3. Then Calculate cosine similarity between query vector and each document vector.
4. Finally Rank documents based on similarity score.

Real-Time Example

- **E-commerce Search:** A user searches for "red running shoes".
 - Document 1: "Comfortable red sports shoes for running" → high similarity score.
 - Document 2: "Black formal shoes" → low similarity score.
 - The system ranks Document 1 higher.

Probabilistic Model

Definition

- Based on **probability theory**.
- The system predicts the **probability that a document is relevant** to the query.
- Documents are ranked from most to least probable relevance.

How it works

1. Assume an **ideal answer set** (most relevant documents).
2. Use statistical methods to estimate the probability that a document belongs to this set.
3. Rank documents based on calculated probability.

Real-Time Example

- **News App Search:** You search for "Olympics 2028".
 - The system calculates that Document A has an **80% chance** of being relevant and Document B has **60%**.
 - You see Document A first.

Real-Life Applications

1. **Boolean Model** → Legal document search, Library catalogs.
2. **Vector Space Model** → Google search ranking, E-commerce filtering.
3. **Probabilistic Model** → News recommendation, Email spam filtering.

COMPARING ALL THE THREE MODELS:

Feature	Boolean Model	Vector Space Model	Probabilistic Model
Ranking	No	Yes	Yes
Ease of Use	Low	Medium	Medium
Mathematical Basis	Set theory	Linear algebra	Probability theory

Feature	Boolean Model	Vector Space Model	Probabilistic Model
Real-World Use	Library DB	Search engines	News & recommendation systems

Non-Classical Information Retrieval Models:

Non-classical IR models are **information retrieval models** that **do not strictly follow classical probability, Boolean logic, or vector space assumptions**.

They take a more *flexible, modern, and often human-like approach* to retrieving relevant information.

In simple way we can say that:

Instead of treating documents and queries in rigid mathematical terms, non-classical models try to **mimic how humans think, judge relevance, or learn from feedback**.

“restaurant dataset” (5–6 restaurants, each with short descriptions, prices, and features like vegetarian, live music, location).

Restaurant	Description	Price Level	Features
Green Leaf	Pure vegetarian, budget-friendly, near bus stand	₹	Veg, Affordable
Spice Hub	Mixed menu, live music, rooftop	₹₹₹	Live Music
Veggie Vibes	Affordable vegetarian, live music	₹₹	Veg, Affordable, Live Music
Royal Dine	Non-veg special, buffet, expensive	₹₹₹	Buffet
Melody Café	Vegetarian snacks, affordable, sometimes live guitar	₹	Veg, Affordable, Occasional Music

"best affordable vegetarian restaurants with live music" is our target.

Round 1 – Classical Search (Boolean)

- Now here we check if **all query words** are present in the restaurant details.
- If yes → “Match” ✓
- If no → “No Match” ✗
- They’ll quickly see only 1–2 restaurants match exactly.

This is the result which is shown finally.

Round 2 – Non-Classical Search (Fuzzy Scoring)

- Now let us give scoring rules:
 - ✓ Vegetarian = +0.4
 - ✓ Affordable = +0.3
 - ✓ Live Music = +0.3

- Here we score each restaurant (add up points for matching features).
- Sort by score and see new rankings — even partial matches appear.

After the both rounds we can say one thing that the Round 2 is easy to understand where exactly the requirement does meets instead of YES / NO, this gives the result with atleast matching concepts.

Same let us try for the below table :

Restaurant	Description	Price Level	Features
Rayalaseema Ruchulu	Pure vegetarian, budget-friendly, near APSRTC Bus Stand	₹	Veg, Affordable
Spicy Tandoor	Mixed menu, live music nights, rooftop dining	₹₹₹	Live Music
Veggie Vibes Kurnool	Affordable vegetarian meals, weekend live music	₹₹	Veg, Affordable, Live Music
Royal Biryani House	Non-veg special biryani, grand buffet, slightly expensive	₹₹₹	Buffet
Melody Café Kurnool	Vegetarian snacks, affordable, occasional live guitar performances	₹	Veg, Affordable, Occasional Music

Target: "affordable vegetarian restaurants in Kurnool with rooftop seating"

Restaurant	Veg (+0.4)	Affordable (+0.3)	Rooftop (+0.3)	Total Score
Rayalaseema Ruchulu	0.4	0.3	0	0.7
Spicy Tandoor	0	0	0.3	0.3
Veggie Vibes Kurnool	0.4	0.3	0	0.7
Royal Biryani House	0	0	0	0.0
Melody Café Kurnool	0.4	0.3	0	0.7

Ranked Results:

Rayalaseema Ruchulu – 0.7
 Veggie Vibes Kurnool – 0.7
 Melody Café Kurnool – 0.7
 Spicy Tandoor – 0.3
 Royal Biryani House – 0.0

Why do we use these models:

Classical models (Boolean, Vector Space, Probabilistic) have limitations:

- They **ignore relationships** between terms.
- They **do not adapt well** to user feedback.
- They can be **too strict** — either a document matches or it doesn't.
- They treat terms independently, ignoring meaning/context.

Non-classical models solve this by:

- ✓ Using **fuzzy logic** for partial matches.
- ✓ Using **neural networks** to learn patterns.
- ✓ Using **latent semantics** to detect hidden meanings.
- ✓ Using **Bayesian reasoning** for uncertainty.

To discuss the different type of Non classical models we have to compare below table:

Model	Main Idea	Real-Time Example
Fuzzy Set Model	Documents can partially match a query with a degree (0 to 1).	Searching “cheap hotels near beach” → A hotel 500m from the beach with medium price gets 0.7 score instead of YES/NO.
Extended Boolean Model	Improves Boolean model by allowing partial matches using p-norms.	Searching “AI AND education” → Even if a doc mostly talks about AI but briefly about education, it still ranks.
Latent Semantic Indexing (LSI)	Uses matrix decomposition (SVD) to find hidden relationships between words.	Searching “car” also finds docs with “automobile,” “vehicle,” “sedan” even if the word “car” is missing.
Neural Network Model	Learns term-document relevance through training.	A trained search engine learns from click data: if many users click a doc for “best biryani,” it ranks it higher next time.
Bayesian Network Model	Models dependencies between terms using probability graphs.	If “virus” appears with “mask” often, and query has “mask,” it predicts “virus” related docs might be relevant.

Let us take the below example:

Scenario: You search for

"affordable vegetarian restaurants in Kurnool with live music"

- **Boolean Model:** Returns only restaurants that have all those exact words.
- **Vector Space Model:** Ranks restaurants by similarity of words.
- **Non-Classical Fuzzy Set Model:**
 - Restaurant A: Veg + live music but expensive → 0.6 relevance
 - Restaurant B: Veg + affordable but no live music → 0.7 relevance
 - Restaurant C: Veg + affordable + live music → 0.95 relevance

Here, the **partial match scoring** from the fuzzy model feels more “human-like.”

Advantages:

- Handles **partial relevance** well.
- Can **capture hidden meanings** (LSI).
- Adapts to **user feedback and trends** (Neural Networks).
- Handles **uncertainty** better (Bayesian).

Comparing both the models:

We have a Query: "affordable wireless headphones with noise cancellation under ₹1500"

Product Name	Price	Key Features	Classical IR (Boolean)	Non-Classical IR (Fuzzy/Semantic)	Reason for Difference
Zebronics Wireless	₹1,299	Wireless, Mic,	✗	✓ (Partial match:	No ANC but other

Product Name	Price	Key Features	Classical IR (Boolean)	Non-Classical IR (Fuzzy/Semantic)	Reason for Difference
Headphones with Mic		No ANC		Wireless + Affordable)	features match
Boat Rockerz 450 Bluetooth Headphones ANC	₹1,499	Bluetooth, ANC	✗	✓ (Synonym: Bluetooth = Wireless)	Price slightly above ₹1500, includes ANC
pTron Tangentbeat Wireless Earphones	₹899	Wireless, Affordable	✗	✓ (Partial match: Affordable + Wireless)	Missing ANC, but good price
Sony WH-CH710N Wireless ANC Headphones	₹7,990	Wireless, ANC	✗	✗	Price way above budget
Boult Audio AirBass Bluetooth ANC Earbuds	₹1,299	Bluetooth, ANC, Affordable	✗	✓ (Synonym: Bluetooth = Wireless)	Meets most criteria despite different wording
Local Brand Wired Earphones	₹299	Wired, Affordable	✗	✗	Not wireless, no ANC

Alternative Models in Information Retrieval (IR)

- Alternative models are **non-traditional approaches** to finding and ranking information.
- These are **different ways of finding and ranking information** that do **not use the strict Boolean search method**.
- **Alternative models = Smarter search methods that understand meaning, importance, and closeness of words, not just exact matches.**
- Instead of only giving results that exactly match all search words, they use **mathematics, statistics, probability, or meaning-based methods** to show results that are **more relevant and flexible**.
- They **don't strictly follow** the rules of:
 - Boolean model (True/False logic)
 - Vector space model (mathematical similarity)
 - Probabilistic model (fixed probability formulas)

Instead, they try to **mimic human thinking** and handle situations where information is **uncertain, fuzzy, or subjective**.

What is the need of it?

- **Humans don't always search with exact keywords.** We use incomplete phrases, spelling mistakes, or vague ideas.
- **Data can be unstructured** (like videos, images, or social media posts).
- **Meaning matters more than exact words** (e.g., "cheap" ≈ "budget-friendly").
- Works better for **modern, complex searches**.

Types of Alternative Models:

Fuzzy Set Model

- **Idea:** Matches results that are *close enough*, even if not exact.
- **Example:** Searching “affordable restaurants” also shows “budget-friendly caf  s” and “low-price dining” in Kurnool.
- **Real-Life Analogy:** Asking a friend for “nearby coffee shops” and they also tell you about tea stalls you might like.

Extended Boolean Model

- **Idea:** Mixes Boolean logic with partial matching.
- **Example:** Searching “Laptop AND 16GB RAM” in Flipkart — it shows exact matches first but also lists 8GB RAM laptops with upgrade options.
- **Real-Life Analogy:** You want *both* pizza and burger, but the restaurant suggests pizza+garlic bread combos too.

Probabilistic Alternative Models

- **Idea:** Uses **learning and statistics** to guess what you’ll like.
- **Example:** Searching “best smartphones under ₹20,000” → System learns you prefer good cameras and shows those first.
- **Real-Life Analogy:** Netflix recommending a movie because “you watched similar ones.”

Neural / AI-based Models

- **Idea:** Uses deep learning to understand meaning and context.
- **Example:** Searching “place to eat biryani near me” → Google shows biryani restaurants even if their page says “Hyderabadi rice & curry.”
- **Real-Life Analogy:** A friend understands what you *mean*, not just what you say.

Feature	Classical IR (Boolean)	Alternative Models
Keyword Match	Exact only	Exact + Near Match
Flexibility	Low	High
Handles Synonyms	✗	✓
Learns from User	✗	✓
Works for Complex Queries	✗	✓

Alternative Search:

let us search “Laptop 16GB SSD” without Boolean operators (AND, OR) → notice the site still finds close matches with 8GB RAM but upgradable, or HDD + SSD combo.

Fuzzy Logic: Definition

A **Fuzzy Model** in Natural Language Processing (NLP) is a way of understanding and matching text where **things don’t have to be exactly the same to be considered a match**. It uses **degrees of similarity** (0 to 1, or 0% to 100%) instead of strict **yes/no** rules.

Let us Imagine you have an **online food delivery app**.

A customer searches for “**chiken biryani**” (with a spelling mistake).

- **Strict matching (Boolean model):** ✗ No results, because "chicken" is not exactly "chicken."
- **Fuzzy model:** ✓ Shows "**chicken biryani**" results because it understands that "chicken" is 90% similar to "chicken."

Why to use it?

- Natural language is **imperfect** — people make spelling mistakes, use synonyms, or write things in different ways.
- A normal exact search may miss relevant results.
- Fuzzy models **catch close matches** and give more **human-like search results**.

How it Works?

- Words or sentences are compared using **similarity scores** (e.g., Levenshtein distance, cosine similarity).
- The search engine or NLP system sets a **threshold** (e.g., match if similarity > 0.7).
- Matches above the threshold are shown as results.

Real time implementation:

Chatbot Understanding

- You type to a customer care bot: "My acct is bloked."
- Fuzzy NLP → Understands you meant "My account is blocked" and responds accordingly.

Feature	Meaning	Example
Handles Typos	Finds results even if words are misspelled	"Resturant" → "Restaurant"
Understands Similar Words	Matches synonyms or related words	"Car" → "Automobile"
Works on Partial Matches	Matches even if only part of the phrase is correct	"Galaxy S24" → "Samsung Galaxy S24 Ultra"
Ranking by Similarity	Results shown in order of closeness to query	90% match first, 70% match later

Advantages

- Improves search results when users make spelling mistakes
- More human-like understanding of queries
- Works well for large datasets and varied language styles.

Cluster Model in NLP

A **Cluster Model** in NLP is a method where similar texts, words, or documents are **grouped together** into "clusters" based on how alike they are.

The goal is to put **related things in the same group** so we can easily find patterns or related information.

RAW EXAMPLE in clothes shop:

In any clothes shop we do sorting clothes:

- All T-shirts in one pile
 - All jeans in another
 - All jackets together
- Even if they have different colors, you group them by type (similarity).

Working concept:

1. **Features Extraction** – Words or documents are converted into numerical form (word embeddings, TF-IDF, etc.).

What is the main concept : turn words or documents into numbers (so math can be applied).

Why do we need? algorithms work on vectors — not raw text.

Common methods (with short notes):

- **Bag-of-Words (BoW):** count how many times each word appears → a vector of counts. Simple but ignores word order and meaning.

- **TF-IDF:** scales counts by how *rare* a word is across documents. Good for highlighting important words.

$$TF_{t,d} = \frac{\text{count of } t \text{ in } d}{\text{total words in } d}, \quad IDF_t = \log \frac{N}{1 + \text{docs containing } t}$$

then $TF-IDF = TF \times IDF$.

- **Word embeddings (word2vec / GloVe / FastText):** words → dense vectors that capture meaning (e.g., “chicken” near “poultry”).
- **Sentence / document embeddings (Sentence-BERT, doc2vec):** produce 1 vector per sentence/doc that captures semantics.
- **Feature engineering (for mixed data):** numeric fields (price, rating) must be scaled; categorical features one-hot or embedding.

Preprocessing steps (always do these first):

- Tokenize, lowercase, remove punctuation.
- Remove stopwords (optional).
- Lemmatize/stem (turn “running” → “run”).
- Optionally remove very rare or very common terms.

Real-time example: for a recipe site, convert each recipe description to a TF-IDF vector or a sentence embedding so “grilled chicken bowl” and “low-fat poultry salad” become numerically close.

2. **Similarity Calculation** – Measure how close the items are (e.g., cosine similarity, Euclidean distance).

What it means: measure how “close” two vectors (documents) are.

Common similarity / distance measures:

Cosine similarity (most used for text): measures angle between vectors (0 = orthogonal, 1 = identical).

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

- Use when vectors are high-dimensional and direction matters (TF-IDF, embeddings).
- **Euclidean distance:** straight-line distance. Works well when vectors are scaled and numeric distances matter.
- **Manhattan (L1) distance:** sum of absolute differences; more robust to outliers in some cases.
- **Jaccard similarity:** for sets (or binary vectors) — intersection over union. Useful for short tag lists or keywords.

Which to use when:

- TF-IDF / embeddings → **cosine** is a safe default.
- Raw numeric features → **Euclidean** (after scaling).
- Sparse binary features (presence/absence) → **Jaccard**.

Tip: in very high dimensions (TF-IDF) cosine similarity often outperforms Euclidean.

3. **Clustering Algorithm** – Common ones are:
 - **K-Means**

K-Means

Idea: place K centroids, assign points to nearest centroid, update centroids, repeat.

Algorithm (short):

1. Choose K (number of clusters).
2. Initialize K centroids (kmeans++ is a good smart init).
3. Repeat until convergence:
 - Assign each point to nearest centroid.
 - Recompute centroids as mean of assigned points.

Pros: fast, simple, works well for spherical clusters.

Cons: must choose K; sensitive to outliers; assumes clusters of similar size/shape.

Choosing K: elbow method (plot inertia vs K), silhouette score, or domain knowledge.

- **Hierarchical Clustering**

Hierarchical Clustering

Two types:

- **Agglomerative (bottom-up):** start with each point as a cluster, merge closest clusters step by step.
- **Divisive (top-down):** start with one cluster, split recursively (less common).

Linkage methods: single, complete, average, Ward (affects merging behavior).

Output: a **dendrogram** (tree). You “cut” the tree at some height to get clusters.

Pros: no need to predefine K; provides full hierarchy; interpretable.

- DBSCAN

DBSCAN:

Idea: clusters are high-density regions separated by low-density regions.

Key params:

- **eps:** radius to search neighbors.
- **minPts:** minimum points to form a dense region.

Behavior: labels points as core (dense area), border, or noise (outlier). Finds arbitrarily shaped clusters and handles noise naturally.

Pros: finds clusters of any shape; handles outliers; no need to pick K.

Cons: sensitive to eps; struggles with varying density.

4. **Group Formation** – Similar items get grouped into the same cluster.

Group Formation (interpretation & output)

After clustering we get the below result:

- **Cluster labels** (which doc belongs to which cluster).
- **Cluster centers** (e.g., centroids for K-Means).
- **Noise points** (DBSCAN).

Understand the below:

- **Cluster labels** tell you which group an item belongs to.
- **Cluster centers** summarize the “average” or “heart” of each group (used in K-Means).
- **Noise points** are outsiders that don’t belong to any group (DBSCAN concept).

How to summarize a cluster:

- Show **top terms** (highest mean TF-IDF) in that cluster → gives human-readable label.
- Show **example documents** from the cluster.
- Compute cluster statistics: size, average similarity, common features.

Evaluation (how good are clusters?):

- **Internal metrics** (no ground truth):
 - **Silhouette score** (−1 to 1): how close point is to its own cluster vs others.
 - **Davies-Bouldin index** (lower is better).
- **External metrics** (if you have labels): adjusted rand index, purity, F1.

Real-Time Example

Customer Reviews on Amazon

- Reviews like “Tasty and fresh,” “Loved the flavor,” “Delicious food” go into one cluster (Positive Feedback).
- Reviews like “Too salty,” “Stale taste,” “Not fresh” go into another cluster (Negative Feedback).

This helps Amazon **quickly see customer sentiment trends** without reading every single review.

Advantages

- ✓ Groups large amounts of data without manual labeling
- ✓ Helps discover hidden patterns.
- ✓ Improves recommendation systems

Scenario	RAW (No NLP)	NLP-enhanced
Context	You run an online recipe search system that clusters recipes based only on keywords.	You run the same system but use NLP to understand meaning, synonyms, and relationships.
User Query	“Healthy chicken salad”	“Healthy chicken salad”
How Clustering Works	System looks at recipes and clusters them purely based on exact word matches like “chicken” and “salad.”	System uses NLP (semantic similarity, lemmatization, word embeddings) to understand that “grilled chicken bowl” and “low-fat poultry salad” are also related to the query.
Example Cluster	Recipes containing “chicken” + “salad” in title: • <i>Chicken Caesar Salad</i> • <i>Chicken Pasta Salad</i>	Semantic cluster containing meaning-related recipes: • <i>Grilled Chicken Salad</i> • <i>Low-fat Poultry Salad</i> • <i>Protein-packed Chicken Bowl</i> • <i>Healthy Cobb Salad</i>
Drawback / Limitation	Misses relevant items that don’t contain exact keywords.	Groups relevant items even when wording is different, improving recall & user satisfaction.
Real-life Analogy	Like grouping books on a shelf only by exact title words.	Like grouping books by their themes, even if titles are different.

Content difference concept understanding in easy terms for the above table:

CONTEXT:

➤ RAW (No NLP):

- You have an **online recipe search system**.
- It groups (clusters) recipes only by looking at the **exact keywords** in them.
- If two recipes have the exact same words, they get grouped together. If not, they won’t be in the same group — even if they mean the same thing.

➤ NLP-enhanced:

- Same recipe search system, but now it uses **Natural Language Processing** to understand the **meaning** of the text.
- It can detect synonyms (“poultry” ≈ “chicken”), related words, and context, so grouping is **based on meaning**, not just exact spelling.

USER QUERY:

- **Both cases:** The user searches for “**Healthy chicken salad**”.
- This is the starting point. The difference will be in **how the system interprets and clusters recipes**.

HOW CLUSTERING WORKS:

- **RAW (No NLP):**
 - The system literally checks for the **exact words** “chicken” and “salad” in the recipe title or description.
 - If one recipe says “chicken bowl” but doesn’t have “salad” in it, it gets ignored — even if it’s clearly similar.
- **NLP-enhanced:**
 - Uses techniques like **semantic similarity** (how close in meaning words are), **lemmatization** (turning words into their base form), and **word embeddings** (vector representations of words that capture meaning).
 - Recognizes that “low-fat poultry salad” is still related to “healthy chicken salad” because “poultry” means “chicken,” and “low-fat” is related to “healthy.”

Example Cluster

- **RAW (No NLP):**
 - Matches **only exact keywords**:
 - *Chicken Caesar Salad*
 - *Chicken Pasta Salad*
 - These match because both have “chicken” and “salad” exactly.
- **NLP-enhanced:**
 - Includes recipes that are **conceptually related**:
 - *Grilled Chicken Salad* (meaning match)
 - *Low-fat Poultry Salad* (synonym match)
 - *Protein-packed Chicken Bowl* (healthy + chicken dish)
 - *Healthy Cobb Salad* (healthy + salad, even if “chicken” isn’t in title)

Drawback / Limitation

- **RAW (No NLP):**
 - Misses **many relevant items** because it doesn’t understand meaning — only looks for exact words.
- **NLP-enhanced:**
 - Finds **more relevant items** because it understands meaning and context.
 - This improves **recall** (how many correct results you get) and **user satisfaction**.

Real-life Analogy

- **RAW (No NLP):**
 - Like arranging books **only by matching exact title words**.

- If a book has a slightly different title, it won't be in the same section — even if it's about the same topic.
- **NLP-enhanced:**
 - Like arranging books **by their themes/subjects**.
 - Even if the titles differ, all books about “space exploration” go in the same section.

LSTM-based Information Retrieval in NLP

LSTM (Long Short-Term Memory) is a special type of neural network that is very good at **understanding sequences**, like sentences, paragraphs, or conversations. In Information Retrieval, **LSTM** helps the system **understand context and meaning over time** — not just individual words.

Example – Customer Support Chatbot Search where we implement in the day life:

- Suppose a user types in a chat:
Day 1: "I bought a phone yesterday."
Day 2: "It's not charging properly."
- A normal keyword-based search might only focus on the second sentence ("charging problem") and show random charging articles.
- **LSTM remembers the earlier context ("bought a phone yesterday") from Day 1.**
- When retrieving help articles, it searches specifically for "**phone charging issues**", not "laptop charging" or "power bank charging."
- This way, the system finds **more accurate results** because it understands the conversation over time.

How LSTM Works in Information Retrieval

1. Input Text

- The process starts when a **sentence, paragraph, or query** is given to the LSTM network.
- The **input is broken into smaller parts (usually words or tokens) so that the model can process them in order.**
- **Example:**
 Query: "Best budget phones with good camera for night photography"
 → LSTM **will process word by word, in sequence.**

2. Memory Cells – The Core of LSTM

LSTM has **special units called memory cells** that act like a “brain” for remembering and forgetting information.

Each memory cell is controlled by three **gates**:

- **a) Input Gate**
 - Decides which new information from the current word should be stored in the memory.
 - Example: In the query above, the words "budget", "good camera", and "night photography" are important, so they are stored.
- **b) Forget Gate**

➤ Decides what old information **is no longer useful and should be removed from memory.**

- Example: If earlier parts of a conversation talked about *laptops*, the forget gate removes that, focusing on *phones* now.

- **c) Output Gate**

- Decides what information from the memory should be passed forward to the next step in the network.
- Example: After reading the whole query, it outputs **the context that this is about affordable smartphones with strong night camera performance.**

3. Context Understanding

- Unlike normal models that just count keywords, LSTM **understands word order and relationships.**

- It learns that:

- "*budget phones with good camera*" means something different from "*good camera with budget phones*" in a different context.

- "*night photography*" **modifies** "*camera*", **not** "*phones*" **directly.**

- This context helps the retrieval system **know the real meaning** behind the query.

4. Ranking & Retrieval

- Once the LSTM has a clear understanding of the context, it searches the database or document collection.
- It doesn't just match keywords — it looks for documents where the **meaning** matches the query.
- The system then ranks the results so that the most relevant ones (matching the intended meaning) appear at the top.
- **Example:**
 - Instead of showing all **phones or all cameras**, it shows **smartphones that are affordable and have excellent low-light photography features.**

Real-Time Example

Google Search with Conversational Queries

- Query: "*Who is the president of the USA?*" → LSTM helps understand context.
- Follow-up Query: "*How old is he?*" → LSTM remembers "he" refers to the president from the previous question.

Without LSTM, the search might not understand that "he" refers to the previous answer.

Advantages

Understands long sentences and context

Good for chatbots, voice assistants, and question-answer systems

Handles time-based or sequence-based meaning

Remarks we can say:

- Requires more computing power than simple models
- Needs large amounts of training data
- Can be slower for very large datasets

1. Supervised Approaches in NLP

□ Supervised learning means:

You have **data** (text) and also the **correct answers (labels)**.

A **machine learning model** learns from this labeled data to make predictions on new text.

Key Points:

Needs **labeled training data**.

Uses algorithms like Logistic Regression, SVM, Decision Trees, or Deep Learning.

Works well if you have a **large amount of quality data**.

Real-time Examples:

Spam Email Detection

Input: Emails.

Labels: "Spam" or "Not Spam".

Model learns patterns (e.g., "free money", "lottery") → predicts if a new email is spam.

1. Sentiment Analysis for Reviews

Input: Movie/Restaurant reviews.

Labels: "Positive", "Negative".

Model learns from labeled reviews → predicts sentiment of a new review.

Chatbot Intent Recognition

Input: User queries like "Book a ticket", "Cancel order".

Labels: Intent categories ("Booking", "Cancellation").

Model learns intents → chatbot replies correctly.

2. Dictionary-Based Approaches in NLP

Dictionary-based means:

You create a **list of predefined words** (dictionary/lexicon) with meanings, scores, or categories.

The system **looks up words** in the dictionary to analyze text.

Key Points:

Does **not need training data**.

Simple but less flexible (depends on dictionary quality).

Works well for quick rule-based tasks.

Real-time Examples:

Sentiment Analysis using Word Lists

Dictionary:

Positive words = ["happy", "good", "excellent"]

Negative words = ["bad", "sad", "terrible"]

Text: "The movie was good but a bit long."

System counts positive vs negative words → predicts sentiment.

Profanity/Abuse Detection

Dictionary of bad/offensive words.

If user message contains any → flag as abusive.

Medical Text Mining

Dictionary: Diseases = ["diabetes", "cancer", "asthma"].

Text: "The patient has diabetes."

System recognizes "diabetes" from dictionary → tags it as disease.

Feature	Supervised Approach (with real-time examples)	Dictionary-Based Approach (with real-time examples)
Data Requirement	Needs large labeled datasets (e.g., millions of spam & non-spam emails)	Needs only word lists (e.g., a list of offensive/profane words)
Accuracy	High, if trained well (e.g., Gmail's spam filter correctly catching 99% spam)	Moderate, depends on dictionary (e.g., a profanity filter missing creative spellings like "f@ke")
Flexibility	Learns new patterns automatically (e.g., Netflix learning new slang for "good" or "bad" reviews)	Limited to words in dictionary (e.g., won't recognize new slang like "lit" = positive)
Example	Spam email detection – Gmail/Outlook automatically moving spam emails	Profanity detection – Social media platforms flagging abusive words from a dictionary
Another Example	Chatbot Intent Recognition – Swiggy/Zomato bots detecting "Order Pizza" vs "Cancel Order"	Medical Reports Mining – System highlighting disease names like "Diabetes" or "Cancer" from reports
One More	Sentiment Analysis in E-commerce –	Sentiment by Word Lists – Counting

Feature	Supervised Approach (with real-time examples)	Dictionary-Based Approach (with real-time examples)
Example	Amazon predicting if a review is "positive" or "negative"	positive/negative words in a tweet

UNIT 4:

Semantic interpretation and logical analysis (Form).

- **Semantic Interpretation** is the process of understanding the meaning of a sentence based on words, grammar, and context.
- **Logical Form** is a structured representation of meaning using symbols and logic to show relationships between concepts.
- **Example:** "All dogs are animals" → **Semantic Meaning:** Dogs belong to the animal category; **Logical Form:** $\forall x (\text{Dog}(x) \rightarrow \text{Animal}(x))$.

$\forall x (\text{Dog}(x) \rightarrow \text{Animal}(x))$ represents a **logical statement** in **First-Order Logic (FOL)**

- ❖ $\forall x \rightarrow$ "For all x" (Universal quantifier, meaning the rule applies to everything).
- ❖ $\text{Dog}(x) \rightarrow$ "x is a dog" (A function or predicate that checks if x is a dog).
- ❖ $\text{Animal}(x) \rightarrow$ "x is an animal" (A function or predicate that checks if x is an animal).
- ❖ $\rightarrow$ "Implies" (If the first part is true, then the second part must also be true).

Logical analysis :

"For all x, if x is a dog, then x is an animal."

This means every dog is an animal in logical terms.

Understanding the Sentence Structure (Syntactic Analysis)

First, we analyze the structure of the sentence. This is called syntactic analysis. It involves breaking down the sentence into its grammatical components.

Example Sentence: "The ball is red."

- **S (Sentence):** The entire sentence.
- **NP (Noun Phrase):** "The ball" – This is the subject of the sentence.
- **VP (Verb Phrase):** "is red" – This is the predicate that describes the subject.
- **ART (Article):** "The" – This specifies the noun.
- **N (Noun):** "ball" – This is the main subject.
- **V (Verb):** "is" – This links the subject to the predicate.(about relationship)
- **ADJP (Adjective Phrase):** "red" – This describes the subject.

Step 2: Semantic Interpretation

Here, we **interpret(understand and explain) the meaning** of the sentence. This is called semantic interpretation. It involves understanding what the sentence is conveying.

Here are the key steps involved:

1. **Lexical Semantics:** Understanding the meaning of individual words.
2. **Compositional Semantics:** Combining the meanings of words to form the meaning of phrases and sentences.
3. **Contextual Understanding:** Considering the context in which the language is used to fully grasp the intended meaning

Example: "The ball is red."

- The sentence is stating that a specific ball has the property of being red.

Step 3: Logical Form

The logical form is an intermediate representation that captures the meaning of the sentence in a structured way. It often uses symbols and logical expressions to represent the relationships between the elements of the sentence.

Example Logical Form: (RED1 <THE b1 BALL>)

- **RED1:** This represents the property of being red.
- **THE b1 BALL:** This represents a specific ball (b1).

This logical form can be read as: "The ball b1 has the property of being red."

Step 4: Contextual Interpretation

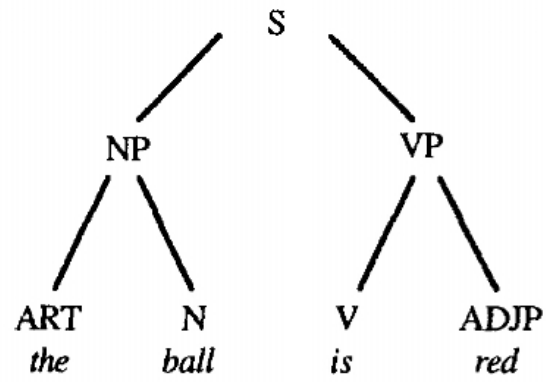
Finally, we consider the context in which the sentence is used. This helps in understanding the final representation of the sentence.

Example Final Representation: Red(BO73)

- **Red:** This is the property of being red.
- **BO73:** This is a specific identifier for the ball in the given context.

This final representation can be read as: "The ball identified as BO73 is red."

SYNTACTIC
ANALYSIS



semantic interpretation

LOGICAL
FORM

(RED1 <THE b1 BALL>)

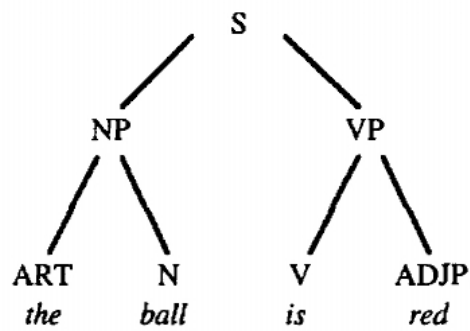
contextual interpretation

FINAL REPRESENTATION

Red(BO73)

Figure 8.1 Logical form as an intermediate representation

SYNTACTIC ANALYSIS



semantic interpretation

LOGICAL FORM

(ASSERT (RED1 <THE b1 BALL>))

{(BALL B0005),
(PERSON P86),
(OWNS P86 B0005)}

CONTEXTUAL
INTERPRETATION

{(BALL B0005),
(PERSON P86),
(OWNS P86 B0005),
(RED B0005)}

INITIAL DISCOURSE
SITUATION

UPDATED DISCOURSE
SITUATION

Figure 8.2 Logical form as a function

- B0005 is a way to refer to the ball.
- P86 is a way to refer to a person.
- ASSERT is used to state that something is true, like "the ball is red."

Logical Interpretation

Logical Interpretation involves representing the meaning of sentences in a formal, structured way, often using logical expressions. This helps in analyzing and reasoning about the meaning systematically. Here are the key steps involved:

1. **Logical Form:** Converting the sentence into a logical expression that captures its meaning.
2. **Predicate-Argument Structure:** Identifying the predicates (actions or states) and their arguments (entities involved).
3. **Quantifiers and Modifiers:** Incorporating elements like "all," "some," "not," etc., to represent the scope and nuances of the meaning.

For example, the sentence "The ball is red" can be represented in logical form as:

- **ASSERT (RED1 <THE b1 BALL>)**
 - **ASSERT**: Declares that the following statement is true.
 - **RED1**: The predicate indicating the color red.
 - **<THE b1 BALL>**: The specific ball being referred to.

Initial Discourse

- **Initial Discourse** represents the **starting point of the conversation** or the existing knowledge before any new information is introduced.
- It includes the facts, entities, and relationships that are already known or established in the context.

For example, in the image, the initial discourse might include information like:

- There is a ball identified as **B0005**.
- There is a person identified as **P86**.
- The person **P86** owns the ball **B0005**.

Updated Discourse

- **Updated Discourse** represents the state of **the conversation or knowledge after new information has been added or after an update has been made**.
- It includes the new facts or changes to the existing information.

For example, in the image, the updated discourse might include the new information that:

- The ball **B0005** is red.

So, the updated discourse would now include:

- There is a ball **identified as B0005**.
- There is a person **identified as P86**.
- The person **P86** owns the ball **B0005**.
- The ball **B0005** **is red**.

WORD SENSES AND AMBIGUITY:

1. Understanding Word Senses

- **Word Senses**: Words can have **different meanings depending on the context**. For example, the word "go" can mean "move," "depart," "pass," etc. Each of these meanings is called a "sense."
- **Synonyms**: Different words can have similar senses. For example, **"go" and "depart"** can mean the same thing in some contexts.

2. Dealing with Ambiguity

- **Ambiguity:** A word is ambiguous if it has more than one sense. For example, "kid" can mean a baby goat or a human child.
- **Vagueness:** Some words are vague and cover a broad range of meanings. For example, "horse" is a general term that includes mares and colts, but it doesn't specify which one.

3. Organizing Word Senses

- **Ontology:** This is a way to classify different senses into categories. a set of concepts and categories in a subject area or domain that shows their properties and the relations between them.
- "what's new about our on
- For example, Aristotle categorized things into classes like physical objects, actions, events, etc.
- **Events and Actions:** These are important categories in semantics. An event is something that happens, and an action is something someone does. For example, "We lifted the box" describes an action.

4. Testing for Ambiguity

- **Linguistic Tests:** These help determine if a word is ambiguous. For example, the sentence "I have two kids and George has three" can only mean both have children or both have goats, not a mix of both.
- **Verb Ambiguity:** Verbs can also be ambiguous. For example, "run" can mean running in a race or running for office.

5. Structural Ambiguity

- **Syntactic Ambiguity:** This happens when the structure of a sentence is unclear. For example, "Happy cats and dogs live on the farm" can mean both cats and dogs are happy or just the cats.
- **Semantic Ambiguity:** This happens when the meaning is unclear even if the sentence structure is clear. For example, "Every boy loves a dog" can mean all boys love the same dog or each boy loves a different dog.

6. Quantifier Scoping

- **Quantifiers:** Words like "every" and "a" can change the meaning of a sentence. For example, "Every boy loves a dog" can have two meanings:
 - There is one dog that every boy loves. Or (2 men bought a laptop)
 - Each boy loves a different dog.

Technical Concept to remember:

- **Word Senses:** Words can have multiple meanings.
- **Ambiguity:** Words can be ambiguous (multiple meanings) or vague (broad meanings).
- **Ontology:** Categories help organize different senses.
- **Tests:** Linguistic tests help identify ambiguity.
- **Structural Ambiguity:** Sentence structure can affect meaning.
- **Quantifiers:** Words like "every" and "a" can change the meaning of sentences.

Basic logical form language:

1. Basic Concepts in this:

- **Terms:** These are the basic building blocks that represent objects, events, or situations. For example, "Fido" is a term.
- **Predicates:** These describe relationships or properties. For example, "is a dog" is a predicate.

2. Simple Sentences: understand (predicate & term)

- A simple sentence like "Fido is a dog" can be represented as (DOG1 FIDO1), where DOG1 is the predicate and FIDO1 is the term.
- For the sentence "Sue loves Jack", it would be represented as (LOVES1 SUE1 JACK1), where LOVES1 is the predicate, and SUE1 and JACK1 are terms.

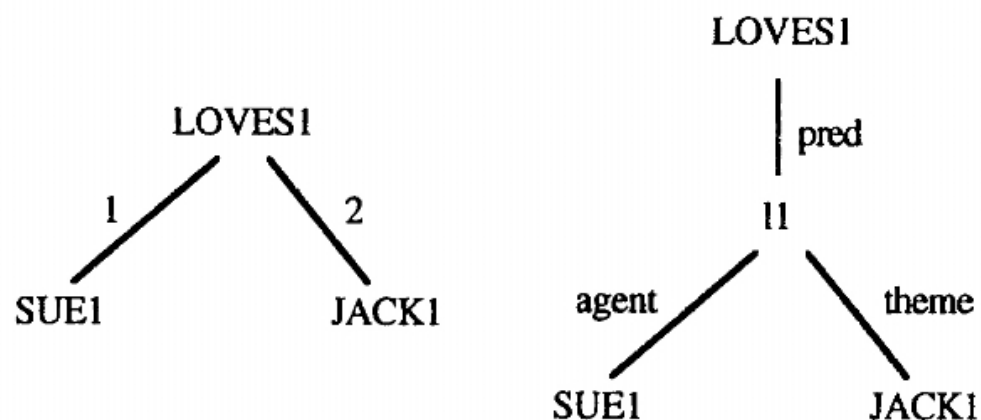


Figure 8.3 Two possible network representations of *Sue loves Jack*

3. Types of Predicates: let us take 2 sentences:

- **Unary Predicates:** Take one argument (e.g., "is a dog").

1. "Rahul loves Priya."
2. "Rahul is a student."

1. "Rahul is a student."

- **Logical Form:** (STUDENT1 RAHUL1)
- **Type:** Unary Predicate
- **Explanation:**
 - **STUDENT1** is the **unary predicate** because it describes a property of **one term** (Rahul).
 - **RAHUL1** is the **term** (the object being described).
- **Binary Predicates:** Take two arguments (e.g., "loves").

2. "Rahul loves Priya."

- **Logical Form:** (LOVES1 RAHUL1 PRIYA1)
- **Type:** Binary Predicate
- **Explanation:**
 - **LOVES1** is the **binary predicate** because it describes a relationship between **two terms** (Rahul and Priya).
 - **RAHUL1** is the **first term** (the subject).
 - **PRIYA1** is the **second term** (the object).

N ary predicate:

- **Sentence:** "Sonia bought Raj a gift."
- **Logical Form:** (BOUGHT1 SONIA1 RAJ1 GIFT1)

Explanation:

- **BOUGHT1** is the **n-ary predicate** (it describes the action of buying and involves **three terms**).
- **SONIA1** is the **first term** (the buyer).
- **RAJ1** is the **second term** (the recipient).
- **GIFT1** is the **third term** (the object being bought).

Key Point:

In above example, the **n-ary predicate** describes an action or relationship involving **multiple terms**

4. Logical Operators:

- **NOT:** Used to **negate a proposition**. For example, "Sue does not love Jack" would be (NOT (LOVES1 SUE1 JACK1)).
- **AND, OR, IF:** These combine propositions. For example, "Jack loves Sue or Jack loves Mary" would be (OR1 (LOVES1 JACK1 SUE1) (LOVES1 JACK1 MARY1)).

5. Complex Sentences:

- To handle more complex sentences, we use **quantifiers** like "all," "some," "most."

- **Variables:** These are used to represent objects that can be referred to later in the text. For example, in the sentences "**A man entered the room. He walked over to the table,**" the word "**He**" refers back to the man introduced earlier.

6. Semantic Networks (Referencing the Image):

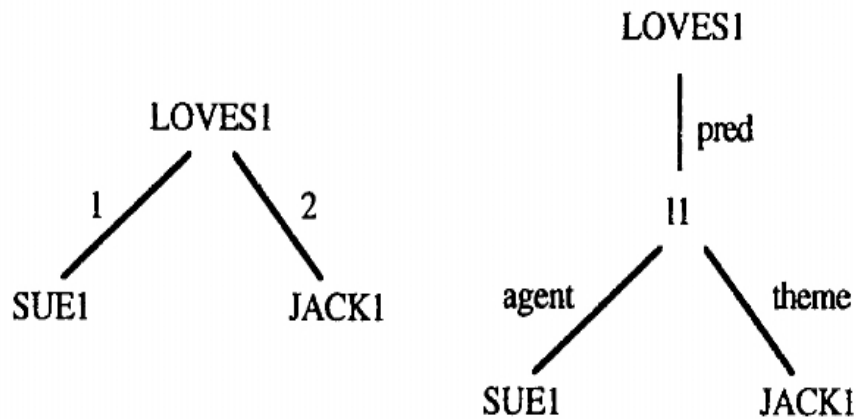


Figure 8.3 Two possible network representations of *Sue loves Jack*

- The image (Figure 8.3) shows **two possible network representations** of the sentence "**Sue loves Jack.**"
- In a **semantic network**, the words (like "Sue," "loves," and "Jack") are **represented as nodes**, and the relationships between them **are shown as arcs** (lines connecting the nodes).
- For example:
 - **Sue** and **Jack** would be nodes.
 - The relationship "**loves**" would be an arc connecting **Sue** to **Jack**.
- The image likely illustrates how the **same sentence can be represented in slightly different ways using nodes and arcs**, but both representations convey the same meaning.

7. Equivalence of Representations:

- Whether you use **logical forms** (like (LOVES1 SUE1 JACK1)) or **semantic networks** (like in the image), the meaning of the sentence remains the same.

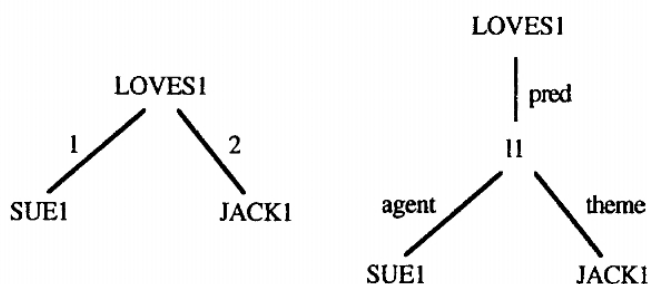


Figure 8.3 Two possible network representations of *Sue loves Jack*

- The image (Figure 8.3) likely shows that different visual or structural representations can still capture the same underlying meaning.

8. Logical Operators in Complex Propositions:

- More complex sentences use **logical operators** like **NOT**, **AND**, **OR**, and **IF**.
- For example, "**Sue does not love Jack**" would use the **NOT** operator, and "**Jack loves Sue or Jack loves Mary**" would use the **OR** operator.

9. Quantifiers and Variables:

- Quantifiers like "**all**," "**some**," **and** "**most**" are used to generalize statements.
- Variables (like **x**) are used to refer to objects introduced earlier in the text. For example, in "**A man entered the room. He walked over to the table,**" the word "**He**" refers back to the man introduced in the first sentence.

Where and How This is Used:

- **Natural Language Processing (NLP):** These diagrams help computers understand the structure and meaning of sentences.
- **Semantic Analysis:** Used to analyze relationships between words in a sentence.
- **Machine Translation:** Diagrams help convert sentences from one language to another by mapping logical forms.
- **Question Answering Systems:** Diagrams help identify the key components of a question to find the correct answer.
- **Text Summarization:** Diagrams help extract the main propositions from a text to generate summaries.

ENCODING AMBIGUITY IN LOGICAL FORM:

Let us know what is Ambiguity in sentence?

Encoding ambiguity happens when a sentence can have **more than one meaning**, and we need to represent these different meanings in logical form.

Identify the Ambiguity:

Use Quantifiers to Represent Ambiguity:

Handle Plural Forms:

Use Predicate Operators for Plurals:

Representing Definite and Indefinite References:

Steps to Understand Encoding Ambiguity:

1. Identify the Ambiguity:

- Some sentences can be interpreted in multiple ways. For example:
 - "**Two men bought a stereo.**"

- **Meaning 1:** Each man bought a stereo (distributive reading).
- **Meaning 2:** The two men bought one stereo together (collective reading).

2. Use Quantifiers to Represent Ambiguity:

- Quantifiers like "**the**," "**a**," "**some**," and "**two**" help us represent different meanings.
- For example:
 - "**Two men bought a stereo.**"
 - **Distributive Reading:** Each man bought a stereo.
 - Logical Form: **(FORALL x (MAN1 x) (BOUGHT1 x STEREO1))**
 - **Collective Reading:** The two men bought one stereo together.
 - Logical Form: **(BOUGHT1 (GROUP1 MAN1) STEREO1)**

Or let us take this sentence:

- "**Three students solved the problem.**"

Step 1: Identify the Ambiguity:

- This sentence can have **two meanings**:
 1. **Distributive Reading:** Each student solved the problem individually.
 2. **Collective Reading:** The three students worked together to solve the problem.

Step 2: Represent the Ambiguity Using Quantifiers:

1. Distributive Reading:

- **Meaning:** Each student solved the problem on their own.
- **Logical Form:**

(FORALL x (STUDENT1 x) (SOLVED1 x PROBLEM1))

Explanation:

- **FORALL x:** For every student (x).
- **(STUDENT1 x):** If x is a student.
- **(SOLVED1 x PROBLEM1):** Then x solved the problem.

2. Collective Reading:

- **Meaning:** The three students worked together to solve the problem.
- **Logical Form:**

(SOLVED1 (GROUP1 STUDENT1) PROBLEM1)

- **Explanation:**

- **(GROUP1 STUDENT1):** A group of three students.
- **(SOLVED1 ... PROBLEM1):** The group solved the problem.

Step 3: Compare the Two Readings:

- **Distributive Reading:** Focuses on **individual actions** (each student solved the problem separately).
- **Collective Reading:** Focuses on **group action** (the students solved the problem together).

3. Handle Plural Forms:

- Plural nouns (like "**dogs**") can also create ambiguity.
- For example:
 - "**The dogs bark.**"
 - **Distributive Reading:** Each dog barks individually.
 - Logical Form: **(FORALL x (DOG1 x) (BARKS1 x))**
 - **Collective Reading:** The group of dogs barks together.
 - Logical Form: **(BARKS1 (GROUP1 DOG1))**

4. Use Predicate Operators for Plurals:

- To handle plurals, we use **predicate operators** like **PLUR**.
- For example:
 - "**The dogs bark.**"
 - Logical Form: **(THE x: ((PLUR DOG1) x) (BARKS1 x))**
 - This means: There is a specific set of dogs, and they bark.

5. Representing Definite and Indefinite References:

- **Definite Reference:** Refers to a specific object (e.g., "**the dog**").
 - Logical Form: **(THE x: (DOG1 x) (BARKS1 x))**
- **Indefinite Reference:** Refers to any object (e.g., "**a dog**").
 - Logical Form: **(EXISTS x: (DOG1 x) (BARKS1 x))**

Example from the Image:

- The image (Figure 8.4) shows **common quantifiers** like "**the**," "**a**," and **bare noun phrases** (e.g., "**water**," "**dogs**").
- These quantifiers help us encode ambiguity in logical forms.

Working on the basic terms:

1. **Ambiguity** arises when a sentence has multiple meanings.
2. **Quantifiers** (like "**the**," "**a**," "**two**") help represent these meanings.
3. **Plural forms** (like "**dogs**") can create distributive or collective readings.
4. **Predicate operators** (like **PLUR**) handle plurals in logical forms.
5. **Definite** and **indefinite references** are represented using quantifiers.

Verbs and States in Logical Form

Here we need to understand the concept of how the verbs are used in the sentence.

Here when we learn this topic in the NLP we need to know **verbs represent actions** and **states represent conditions**. Using logical form like **EAT(John, Apple)** or **BLUE(Sky)**, computers can reason about meaning, which is useful in chatbots, translation, and question answering.

Why This Topic is Important

In **Natural Language Processing (NLP)**, computers need to **understand the meaning of sentences**, not just words.

- Sentences often talk about **actions (verbs)** and **situations (states)**.
- Logical form helps represent these in a way that a machine can **reason about them**.

Verbs in Logical Form

- **Verbs** usually represent **actions** or **events**.
- In logic, we write them like functions that take participants (subjects, objects, etc.).

Example:

Sentence: “**John eats an apple.**”

Logical Form: **EATS(John, Apple)**

- **EATS** = verb (action)
- **John** = agent (the doer)
- **Apple** = theme (the thing affected)

Real-Life NLP Use:

- In **chatbots** or **voice assistants**, verbs tell the system **what action is happening**.
- E.g., “Book a ticket” → **BOOK(User, Ticket)**.

States in Logical Form

- **States** describe **a condition or situation, not an action**.
- They often use verbs like *is, seem, has, belong, like*.

Example:

Sentence: “**The sky is blue.**”

Logical Form: **BLUE(Sky)**

- This is not an action → it’s a **state**.

Another Example:

“**Alice knows Bob.**”

Logical Form: **KNOW(Alice, Bob)** → a mental state, not a physical action.

Real-Life NLP Use:

- In **question answering**, states help understand facts.
- If user asks: “*Who is the pm of India?*”, system checks stored **states** like **PM_OF(X, Country)**.

Aspect	Verbs (Actions)	States
Nature	Something happens	Something is true / exists
Time	Dynamic (can start/stop)	Static (holds until it changes)
Example	RUN(John), EAT(John, Apple)	BLUE(Sky), KNOW(Alice, Bob)

Aspect	Verbs (Actions)	States
NLP Application	Command understanding, planning	Knowledge base, fact retrieval

Real-Time NLP Examples

1. Voice Assistants (Alexa, Siri)

- Action: "Play music" → PLAY(User, Music)
- State: "The weather is rainy" → RAINY(Weather)

2. Customer Support Chatbots

- Action: "Cancel my order" → CANCEL(User, Order)
- State: "My account is locked" → LOCKED(Account)

3. Machine Translation (Google Translate)

- Needs to know if the verb is action vs. state to translate correctly.
- Example: "I am running" (action) vs. "I am tired" (state).

1. Understanding Verbs and Their Roles

Verbs like "break" can describe actions or events, but the way they are used in sentences can vary. For example:

- "John broke the window with the hammer."
- "The hammer broke the window."
- "The window broke."

Even though all three sentences describe the same type of event (breaking a window), the verb "break" is used differently in each case. This creates a problem for NLP because the verb seems to have different "senses" or meanings depending on the number of arguments (like John, the hammer, or the window) it takes.

2. The Problem of Verb Senses

- In the first sentence, "break" involves three things: John, the window, and the hammer. This is a **ternary relation**.
- In the second sentence, "break" involves two things: the hammer and the window. This is a **binary relation**.
- In the third sentence, "break" involves only one thing: the window. This is a **unary relation**.

This means we might need three different versions of the verb "break" (BREAK1, BREAK2, BREAK3) to handle these cases, which is not ideal.

3. Davidson's Solution: Introducing Events

To solve this problem, philosopher Donald Davidson suggested treating verbs as events. Instead of creating multiple senses of the verb, we introduce an event variable (like "e1") to represent the action. For example:

- "John broke the window" becomes:
There is an event e1 where John is the agent, and the window is the theme (the thing being broken).
- "John broke the window with the hammer" adds:
The hammer is the instrument used in the event.

This way, we only need one sense of the verb "break," (once we add this we can then make the sentence based on the AGENT and THEME) and we can add more details (like the hammer or the time) as needed.

4. Thematic Roles

Thematic roles are labels that describe the relationship between the verb and its arguments. For example:

- **Agent:** The person or thing performing the action (e.g., John).
- **Theme:** The person or thing affected by the action (e.g., the window).
- **Instrument:** The tool or means used to perform the action (e.g., the hammer).

Using these roles, we can represent the meaning of sentences in a consistent way. For example:

- "John broke the window" is represented as:
There is an event e1 (we make this as the VERB) where John is the agent, and the window is the theme.

5. Extending to States

The same idea can be applied to states (like being unhappy) instead of events. For example:

- "Mary was unhappy" can be represented as:
There is a state s1 where Mary is the theme, and the state is unhappiness.
- "Mary was unhappy in the meeting" adds:
The meeting is the location where the state occurred.

6. Simplified Representations

To make things easier, we can use abbreviations for these representations. For example:

- "John broke the window" can be written as:
(`<PAST BREAK> e1 [AGENT John] [THEME window]`)
- "Mary was unhappy in the meeting" can be written as:
(`<PAST UNHAPPY> s1 [THEME Mary] [IN-LOC meeting]`)

7. Why This Matters for NLP

- **Consistency:** Using events and thematic roles helps us represent sentences in a consistent way, even when the verb is used differently.

- **Flexibility:** We can easily **add more details** (like time, location, or instruments) without changing the core meaning of the verb.
- **Simplicity:** We don't need to create **multiple senses of the same verb**, which makes NLP systems easier to build and maintain.

Multiple senses mean:

1. *John broke the window* (causing something to break),
2. *The window broke* (something breaking on its own), or
3. *Take a break* (resting). Each use represents a different sense

8. Key Takeaways

1. **Verbs** can describe **actions (events) or states**.
2. **Thematic roles (like agent, theme, and instrument)** help us understand the relationships between the verb and its arguments.
3. Introducing event variables allows us to represent sentences in a flexible and consistent way.
4. This approach simplifies NLP tasks like semantic analysis and meaning representation.

9. Example in Steps

Let's take the sentence:

"John broke the window with the hammer."

1. Identify the verb: **break**.
2. Identify the arguments:
 - Agent: John
 - Theme: the window
 - Instrument: the hammer
3. Introduce an event variable: **e1**.

4. Represent the sentence as:
`(<PAST BREAK> e1 [AGENT John] [THEME window] [INSTRUMENT hammer])`

Where do we use this?

We use this concept in the Natural Language Understanding (NLU) systems like chatbots, virtual assistants (e.g., Siri, Alexa), and machine translation tools to accurately interpret the meaning of sentences. It also helps in information extraction tasks, such as identifying relationships in text for knowledge graphs or summarizing documents.

Thematic roles:

Thematic roles help us understand the relationships between verbs and the nouns (or phrases) in a sentence. They tell us *who* is doing something, *what* is being acted upon, *how* it's being done, and more. In this we shall look into deep about it below:

1. What Are Thematic Roles?

Thematic roles are labels that describe the function of each noun in a sentence. For example:

- Who is doing the action? (Agent)
- What is being acted upon? (Theme)
- What tool is used? (Instrument)

These roles help us understand the meaning of sentences more clearly.

2. Common Thematic Roles

Here are the most common thematic roles with examples:

Agent

The doer of the action, often with intention or control.

Example: *John broke the window.*

John is the agent because he performed the action.

Theme

The thing being acted upon or experiencing a change.

Example: *John broke the window.*

The window is the theme because it is the object being broken.

Instrument

The tool or means used to perform the action.

Example: *John broke the window with a hammer.*

The hammer is the instrument because it was used to break the window.

Experiencer

The person who experiences a feeling, thought, or perception.

Example: *John saw the unicorn.*

John is the experiencer because he is the one seeing.

Beneficiary

The person who benefits from the action.

Example: *I baked a cake for Lucy.*

Lucy is the beneficiary because she benefits from the cake.

Location (AT-LOC)

The place where something happens.

Example: *The book is on the table.*

The table is the location.

Source (FROM-LOC)

The starting point of an action or movement.

Example: *I walked from the park.*

The park is the source.

Destination (TO-LOC)

The end point of an action or movement.

Example: *I walked to the store.*

The store is the destination.

Possession (AT-POSS, TO-POSS, FROM-POSS)

Describes ownership or transfer of ownership.

Example: *I gave the book to John.*

John is the recipient (TO-POSS) of the book.

Time (AT-TIME, FROM-TIME, TO-TIME)

Describes when something happens.

Example: *I worked from 9 AM to 5 PM.*

9 AM is the FROM-TIME, and 5 PM is the TO-TIME.

3. How to Identify Thematic Roles

Here's a simple way to identify roles in a sentence:

1. **Find the verb** (action or state).

Example: *John broke the window.*

- Verb: *broke*

2. **Ask questions** about the verb:

- *Who did it?* → **Agent** (John).
- *What was affected?* → **Theme** (the window).
- *What was used?* → **Instrument** (hammer, if mentioned).

3. **Look for modifiers** like "with," "for," "to," "from," etc., to identify roles like **Instrument**, **Beneficiary**, **Source**, or **Destination**.

Some more examples of Thematic Roles in Sentences

1. *John broke the window with a hammer.*

- Agent: John
- Theme: the window
- Instrument: the hammer

2. *Mary gave a book to John for Susan.*

- Agent: Mary

- Theme: the book
 - Recipient (TO-POSS): John
 - Beneficiary: Susan
3. *The sun dried the apples.*
- Instrument: the sun
 - Theme: the apples
4. *John saw the unicorn.*
- Experiencer: John
 - Theme: the unicorn

4. Why Are Thematic Roles Important?

- They help computers and humans understand **who is doing what to whom** in a sentence.
- They make it easier to handle **modifiers** like "with," "for," or "to."
- They provide a consistent way to analyze sentences, even when the verb is used differently.

5. Quick Summary of Key Roles

Role	Description	Example
Agent	<i>The doer of the action</i>	<i>John broke the window.</i>
Theme	<i>The thing being acted upon</i>	<i>The window broke.</i>
Instrument	<i>The tool used in the action</i>	<i>John used a hammer.</i>
Experiencer	<i>The one who experiences something</i>	<i>John saw the unicorn.</i>
Beneficiary	<i>The one who benefits from the action</i>	<i>I baked a cake for Lucy.</i>
Location	<i>The place where something happens</i>	<i>The book is on the table.</i>
Source	<i>The starting point of an action</i>	<i>I walked from the park.</i>
Destination	<i>The end point of an action</i>	<i>I walked to the store.</i>
Possession	<i>Ownership or transfer of ownership</i>	<i>I gave the book to John.</i>
Time	<i>When something happens</i>	<i>I worked from 9 AM to 5 PM.</i>

The concept of **thematic roles** came to **solve ambiguity** in language and help computers understand meaning, not just grammar. Sentences can have the **same structure but different meanings**, so roles like **agent (doer)** and **patient (receiver)** help clarify relationships. This makes NLP better at **understanding, translating, and responding** like humans.

Speech Acts and Embedded Sentences:

What Are Speech Acts?

Speech acts are the different ways we use sentences to communicate. For example:

- **Asserting a fact:** "The man ate a peach."
- **Asking a question:** "Did the man eat a peach?"
- **Giving a command:** "Eat the peach!"
- **Asking for information:** "What did the man eat?"

Each type of sentence has a different purpose, and we use **operators** in logical form to represent these purposes.

Why Do We Use Speech Acts?

We use speech acts to:

- Clearly distinguish between **statements**, **questions**, and **commands**.
- Help computers understand the **intent** behind a sentence (e.g., is the speaker asking for information or giving an order?).
- Build systems like **chatbots**, **virtual assistants**, and **search engines** that can respond appropriately to different types of sentences.

Types of Speech Acts and Their Logical Forms

Here are the main types of speech acts and how they are represented:

1. Assertion (ASSERT)

- **Used for declarative sentences** (statements).
- Example: *"The man ate a peach."*
- Logical Form:

```
(ASSERT (<PAST EAT>e1 [AGENT <THE m1 MAN1>] [THEME <A p1 PEACH1>]))
```

 - This means: *"It is asserted that the man ate a peach."*

2. Yes/No Question (Y/N-QUERY)

- Used for **yes/no questions**.
- Example: *"Did the man eat a peach?"*
- **Logical Form:**

```
(Y/N-QUERY (<PAST EAT> e1 [AGENT <THE m1 MAN1>] [THEME <A p1 PEACH1>]))
```

 - This means: *"Is it true that the man ate a peach?"*

3. Command (COMMAND)

- **Used for imperative sentences** (orders or requests).
- Example: *"Eat the peach!"*
- Logical Form:

```
(COMMAND (EAT e1 [THEME <THE p1 PEACH1>]))
```

 - This means: *"You must eat the peach."*

4. Wh-Question (WH-QUERY)

- **Used for questions asking for information** (who, what, where, etc.).
- Example: *"What did the man eat?"*
- Logical Form:

```
(WH-QUERY (<PAST EAT> e1 [AGENT <THE m1 MAN1>] [THEME <WH w1 PHYSOBJ>]))
```

 - This means: *"What object did the man eat?"*

4. Embedded Sentences

Embedded sentences are **sentences inside other sentences**, like **relative clauses**. For example:

- *"The man who ate a peach left."*
- Here, *"who ate a peach"* is an embedded sentence that describes *"the man."*

(ASSERT

(<PAST LEAVE> l1
[AGENT <THE m1 (& (MAN1 m1)
<PAST EAT1> e2
[AGENT m1]
[THEME <A p1 PEACH>]])>))

Where Do We Use This?

- **Natural Language Processing (NLP):**
Speech acts help chatbots and virtual assistants understand whether a user is asking a question, giving a command, or making a statement.
- **Information Retrieval:**
Search engines use this to distinguish between queries (e.g., "What is AI?") and commands (e.g., "Find AI articles").
- **Machine Translation:**
Translating sentences accurately requires understanding their purpose (e.g., translating a question as a question, not a statement).

Speech acts help us understand the **purpose** of a sentence (e.g., to inform, ask, or command).

Logical forms with operators like **ASSERT**, **Y/N-QUERY**, **COMMAND**, and **WH-QUERY** make it easier for computers to process and respond to sentences correctly.

Speech Acts and Embedded Sentences: short texting

What Are Speech Acts?

Sentences are used for different purposes in conversation. Depending on the type of sentence, the speaker has a different **goal** (such as making a statement, asking a question, or giving a command), here are some types:

- Propositional
- Locution: what is said and meant (is there any book on the table?)
- Illocution: the request, to find something from others.
- Per locution: what is done, actual effect (what causes us to get the book from the table).

Four Types of Sentences in Logical Form (LF)

Logical Form (LF) is a way to represent the meaning of sentences in a structured way. Each type of sentence has its own **operator** in LF:

Assertions (Statements)

- Used to state facts or information.
- Example: "The man ate a peach."
- LF

(ASSERT (<PAST EAT> e1 [AGENT <THE m1 MAN1>] [THEME <A p1 PEACH1>]))

- **ASSERT** means the sentence is a statement.
- **PAST EAT** means the action happened in the past.
- **AGENT** is the doer of the action (*the man*).
- **THEME** is the object (*a peach*).

Yes/No Questions

- Used to ask questions that can be answered with yes or no.
- Example: "Did the man eat a peach?"
- **LF Representation:**

(Y/N-QUERY (<PAST EAT> e1 [AGENT <THE m1 MAN1>] [THEME <A p1 PEACH1>]))

Y/N-QUERY means this is a yes/no question.

Commands

- Used to tell someone to do something.
- Example: "Eat the peach."
- **LOGICAL FORM Representation:**

(COMMAND (EAT e1 [THEME <THE p1 PEACH1>])) # (EAT THE PEACH)

COMMAND means this is a command.

WH-Questions

- Used to ask **who, what, where, when, why, how** questions.
- Example: "What did the man eat?"
- **LF Representation:**

(WH-QUERY (<PAST EAT> e1 (AGENT <THE m1 MAN1>] [THEME <WH w1 PHYSOBJ>]))

- WH-QUERY means it is a WH-question.
- WH w1 PHYSOBJ means we are asking about an object (what)

What Are Embedded Sentences?

- Sometimes, a sentence is part of a bigger sentence. These are called **embedded sentences**.
- Example: "The man who ate a peach left."
 - This contains the sentence "The man ate a peach." inside it.
- **LF Representation:**

(ASSERT(this is the sentenct)

(<PAST LEAVE> l1

[AGENT <THE m1 (& (MAN1 m1)

(<PAST EAT1> e2 [AGENT m1] [THEME <A p1 PEACH>]))>))

The sentence describes the man, but also includes the action he did (ate a peach).

Why Is This Important?

- Helps computers understand **natural language** (useful in AI, chatbots, and virtual assistants).
- Helps in **linguistics and logic** to analyze how sentences work.

Defining Semantic Structure: Model Theory

Here the topic all talks about how meaning (semantics) is defined using model theory in logical form language. Instead of just explaining what a sentence means, model theory helps define **how different parts of a sentence map to real-world concepts using set theory.**

1. What Is Model Theory?

- **Model theory** is a way to give meaning to logical sentences by connecting them to mathematical structures.
- It treats words, phrases, and sentences as **semantic objects** that can be analyzed like elements in a set.
- Instead of just looking at the structure of sentences, it focuses on how they relate to **real-world meanings**.

2. Key Elements in Model Theory for Logical Form

- A. Propositions & Logical Operators
- A **proposition** represents a complete idea (e.g., "*The man eats a peach.*").
- **Operators** modify these ideas (e.g., PAST means the action happened before).

(ASSERT (<PAST EAT> e1 [AGENT <THE m1 MAN1>] [THEME <A p1 PEACH1>]))

- **ASSERT** = Declaring a fact.
- **PAST** = Indicates past tense.
- **AGENT** = The subject (who is doing the action).
- **THEME** = The object (what the action is performed on).

B. Quantifiers & Variables (Mapping to Sets)

- **Quantifiers** help express general or specific meanings, such as:
 - **THE** → A specific object ("*the man*" = a particular person).
 - **SOME** → An unspecified object ("*some man*" = any man).
 - **WH** → A question variable ("*who*" = an unknown person).
- **Variables** represent different elements in a set. Example:

VARIABLE → b1 | man3

Here, **b1** or **man3** represents an individual in a set of all men.

C. Predicates & Relations (Defining Properties)

- **Predicates** describe characteristics or actions of objects.
- Example:

TYPE-PREDICATE → HAPPY1 | RED1

- **HAPPY1** means the object is happy.
- **RED1** means the object is red.

Events & States define **what happens** in a sentence:

EVENT-STATE-PRED → RUN1 | LOVE3 | GIVE1

- **RUN1** means "someone is running."
- **LOVE3** means "someone loves someone."
- **GIVE1** means "someone gives something to someone else."

Language modeling is the way of determining the probability of any sequence of words. Language modeling is used in various applications such as Speech Recognition, Spam filtering, etc. Language

modeling is the key aim behind implementing many state-of-the-art Natural Language Processing models.

Methods of Language Modelling

Two methods of Language Modeling:

1. **Statistical Language Modelling:** Statistical Language Modeling, or Language Modeling, is the development of probabilistic models that can predict the next word in the sequence given the words that precede. Examples such as N-gram language modeling.
2. **Neural Language Modeling:** Neural network methods are achieving better results than classical methods both on standalone language models and when models are incorporated into larger models on challenging tasks like speech recognition and machine translation. A way of performing a neural language model is through word embeddings.

Part B: Chapter 4

What is Language Modelling?

Language Modelling (LM) is a way of teaching computers to understand and generate human-like text. It predicts the next word in a sentence based on previous words.

Example:

When you type on your phone, it suggests the next word. That's language modelling!

Why is Language Modelling Important?

Language models help in:

- ✓ Chatbots (like Alexa, Siri)
- ✓ Google Search predictions
- ✓ Auto-correct and auto-suggest
- ✓ Text translation (Google Translate)

How Does It Work?

A language model learns from a large amount of text and finds patterns in word usage. There are two main types:

1. Statistical Language Models (Old Method)

- Uses probability to guess the next word.
- Example: If "I am feeling very..." is given, it checks how often words like "happy" or "sad" follow in past texts.

2. Neural Language Models (Modern AI-based)

- Uses deep learning (AI) to understand context better.
- Example: ChatGPT understands full sentences and generates meaningful replies.

Real-Time Example: Autocomplete in Google Search

When you type "Best place to visit in...", Google suggests:

- India
- USA
- Europe

N- GRAM MODEL:

What is an N-gram?

An **N-gram** is a group of **N words** that appear together in a sentence. This helps computers learn how words are used together.

Example Sentence:

"This article is on NLP."

Here are some **N-grams** from this sentence:

- **Unigrams (1 word):**
 - "This", "article", "is", "on", "NLP"
- **Bigrams (2 words):**
 - "This article", "article is", "is on", "on NLP"
- **Trigrams (3 words):**
 - "This article is", "article is on", "is on NLP"

So, an N-gram is just a group of words taken together from a sentence!

N-gram Language Model

A **N-gram language model** helps predict the **next word** in a sentence based on the **previous words**.

Example of Predicting the Next Word:

Let's say we have the sentence:

"This article is on..."

We want to predict what the next word will be.

- **In a Bigram Model:**
 - It looks at the last word "on" and checks which word is most likely to follow "on". It might predict "NLP" because in past examples, "on" is often followed by "NLP".
- **In a Trigram Model:** topic is on. Is on NLP – previous record.
 - It looks at the last two words, "is on", and checks which word often follows "is on". In many cases, "NLP" follows "is on", so it predicts "NLP" with higher confidence.

How does the N-gram Language Model Calculate Predictions?

The N-gram language model uses **probability** to predict the next word. Here's how:

- If we want to predict the next word after "This article is on", we calculate the probability of different words coming next.
 - $p(\text{"NLP"} \mid \text{"This article is on"})$ means the **probability** of the word "NLP" coming after the words "This article is on".
 - It checks past data and sees that "on NLP" is a common combination, so it predicts "NLP".

Formula for Prediction:

To simplify, here's a basic formula for how the model calculates the probability of a word:

- $p(w_5 | w_1, w_2, w_3, w_4)$

This means:

- $p(w_5)$ is the **probability** of the 5th word based on the first 4 words (history).
- w_1, w_2, w_3, w_4 are the **previous words**.

Generalizing the Idea:

The language model uses a **chain rule** to predict the sequence of words:

- The model looks at one word at a time and checks the probability of each word based on the words before it.
 - $P(w_1, w_2, w_3, w_4, w_5) = P(w_1) * P(w_2 | w_1) * P(w_3 | w_1, w_2) * P(w_4 | w_1, w_2, w_3) * P(w_5 | w_1, w_2, w_3, w_4)$

So, the model is predicting each word in the sentence one by one by looking at the previous words.

Markov Assumption for Simplicity:

The **Markov assumption** says that the **future state depends only on the present state**. In language models, it means we only need the **last few words** (like in bigrams or trigrams) to predict the next word, and not the entire sentence.

For example:

- For **bigram models**: The **next word depends** only on the **previous word**.
- For **trigram models**: The next word depends only on the **last two words**.

Why is this Important?

- **Autocomplete**: When you type "Best place to visit in", the model can predict the next word based on common phrases, like "India", "USA", or "Europe".
- **Speech Recognition**: If you say "I am going to", the system can predict that you might say "the store", "the park", or something similar.

Short understanding:

To summarize, an **N-gram model** helps computers predict the next word based on the previous ones.

- It can use **unigrams**, **bigrams**, or **trigrams** to make predictions.
- It looks at **patterns in text** and uses those patterns to guess what word is most likely to come next.

The **N-gram language model** is simple but effective in many applications like search engines, chatbots, and text prediction.

LANGUAGE MODEL ADAPTATION:

Language model adaptation means taking a pre-trained language model and fine-tuning it so it performs better for a specific **domain, language, or task**. For example:

- Adapting a general English model to **understand legal documents**.

- Adjusting a model trained on formal text to **understand social media slang.**
- Tuning a model trained on English to work better for **Hindi-English mixed text.**

It's basically **making a general model smarter for a specific use case.**

Different domains:

- A model trained on Wikipedia may not understand medical notes or legal documents.

Different languages or dialects:

- English-trained LM may fail on Hindi-English mixed text or regional expressions.

Task-specific needs:

- General models may not be good at summarizing, sentiment analysis, or chatbots without adaptation.

Methods of Language Model Adaptation

There are mainly **three ways**:

a) Fine-tuning

- Take a pre-trained model and train it further on **specific data.**
- Example: Train a general model on **medical articles** → model learns medical terms.

Real time Example:

- Suppose you have a **general English model.** It knows normal English sentences but doesn't know medical terms.
- You feed it a dataset of **medical articles and doctor notes.**
- Now the model learns words like *hypertension, myocarditis, cholesterol*, etc., and can answer questions, summarize, or classify medical text.

b) Domain-adaptive pretraining

- Continue **pretraining the LM on domain-specific text** before fine-tuning.
- Example: Pretrain on **financial news articles** → better for finance tasks.

Real-time example:

- You have a **general model trained on Wikipedia and news articles.**
- You want it to understand **financial news.**
- First, you pretrain the model on thousands of financial articles, stock reports, and blogs.
- After that, you fine-tune it for tasks like **predicting stock sentiment** or **extracting financial events.**

c) Feature-based adaptation

- Extract embeddings from a pre-trained LM and **use them as features** in another model.

- Example: Use BERT embeddings of tweets to classify **positive/negative sentiment**.

Real-time example:

- Take a **BERT model**. Feed tweets into it, and extract its embeddings (vector representations of each tweet).
- Then use these embeddings as input to a **simple classifier** (like logistic regression) to detect **positive, negative, or neutral sentiment**.

Real-life Examples

1. **Healthcare:** Adapting a model to read **clinical notes** or lab reports.
2. **Finance:** Adapting a model to **analyze stock news or reports**.
3. **Social Media:** Adapting a model to understand **slang, emojis, or abbreviations**.
4. **Multilingual NLP:** Adapting English-trained LM to **Hindi-English mixed text**.

LANGUAGE MODEL EVALUATION:

What is a Language Model?

A **language model is a type of computer** program that understands and generates human language. It learns from lots of text data and tries to predict the next word in a sentence or generate new sentences.

Why Do We Need to Evaluate Language Models?

We evaluate language models to check how well they perform. If a model is good, it can understand and generate text that makes sense, sounds natural, and is useful for tasks like translation, answering questions, or writing stories.

How Do We Evaluate Language Models?

1. Perplexity:

- **What is it?** Perplexity measures **how confused** the model is **when it tries to predict the next word**. **A lower perplexity means the model is better at predicting.**
- **Example:** If the model is good, it will have low perplexity because it can guess the next word easily.

2. Accuracy:

- **What is it?** Accuracy checks how often the model predicts the correct word or sentence.
- **Example:** If the model **predicts the right word 9 out of 10 times**, it has high accuracy.

3. BLEU Score:

- **What is it?** BLEU (Bilingual Evaluation Understudy) score is **used to check how close the model's output is to a human-written sentence**. It is often used in translation tasks.
- **Example:** If the **model's translation is very similar to a human translation**, it gets a high BLEU score.

4. ROUGE Score:

- **What is it?** ROUGE (Recall-Oriented Understudy for Gisting (summarize) Evaluation) is used to evaluate summaries. **It checks how much of the important information from the original text is included in the summary.**
- **Example:** If the summary includes most of the key points, it gets a high ROUGE score.

5. Human Evaluation:

- **What is it?** Sometimes, people evaluate the model's output by reading it and judging if it makes sense, sounds natural, and is useful.
- **Example:** A person might rate the model's story on a scale of 1 to 10 based on how interesting it is.

6. Task-Specific Evaluation:

- **What is it?** Some models are built for specific tasks like answering questions or translating text. We evaluate them based on how well they perform that task.
- **Example:** For a question-answering model, we check if it gives the correct answer to a question.

Challenges in Evaluating Language Models:

- **Subjectivity:** Sometimes, what sounds good to one person might not sound good to another?
- **Context Understanding:** Models might struggle to understand the context of a conversation or text.
- **Bias:** Models can sometimes show bias based on the data they were trained on, which can affect their performance.

Conclusion of the Concept:

Evaluating language models is important to make sure they work well and are useful for real-world tasks. We use different methods like perplexity, accuracy, BLEU, ROUGE, and human evaluation to check their performance. Each method has its strengths and weaknesses, so we often use a combination of them to get a complete picture of how good the model is.

Parameter estimation:

Parameter estimation is the process of figuring out the best settings (parameters) for a language model so it can predict words or sentences accurately. Think of it as teaching a computer how language works by showing it lots of examples and letting it adjust its "rules" to make better predictions.

Real-Time Example:

Imagine you're teaching a robot to complete sentences. You give it examples like:

- "I love to eat pizza."
- "I love to eat ice cream."
- "I love to play soccer."

The robot learns that after "I love to eat," the next word is likely "pizza" or "ice cream," but not "soccer." It adjusts its internal settings (parameters) to reflect this. Later, when you say, "I love to eat...," the robot can predict "pizza" or "ice cream" because it has learned the patterns from the examples you provided.

In this case, **parameter estimation** is the process of the robot figuring out the probabilities (e.g., "pizza" has a 50% chance, "ice cream" has 40%, and "soccer" has 10%) based on the data it has seen.

We have 3 types of parameter estimation in the concept of NLP, listed below:

Maximum-Likelihood Estimation (MLE)

How does MLE work?

MLE tries to find the best parameters (like probabilities) that make the training data most likely. Imagine you have a sentence: "The cat says meow." If in your training data, the word "cat" is followed by "meow" 90% of the time, MLE will set the probability of "meow" following "cat" to 90%. This way, the model learns from what it sees most often.

Example: If you see the sentence "I love ice cream" 100 times and "I love pizza" 50 times, MLE will say the probability of "ice cream" after "I love" is higher ($100/150 = 66.67\%$) than "pizza" ($50/150 = 33.33\%$).

Smoothing Techniques

Why smoothing is needed: for the unseen words in the training, the robot will do it on own.

Sometimes, words or combinations of words are rare or never seen in the training data. Without smoothing, the model might think these rare or unseen words have zero probability, which is not realistic. Smoothing helps by giving small probabilities to these rare or unseen events.

Common smoothing methods:

- **Laplace Smoothing:** Add 1 to every word count, so no word has zero probability. For example, if "dog" never follows "the" in your data, Laplace smoothing will still give it a small chance.

Example: If "the cat" appears 10 times and "the dog" never appears, without smoothing, "the dog" has 0% chance. With Laplace smoothing, "the dog" might have a small chance, like $1/11$ (around 9%).

Bayesian Parameter Estimation

This concept is all dependent on below two points:

1. What we see in the training data.
2. What we already know about the problem (prior knowledge).

For example, in language modeling, we might know some general rules about how words work in a sentence. Bayesian estimation uses these rules to help make better guesses, especially when the training data is small or doesn't cover everything.

Using prior knowledge to estimate parameters:

Bayesian estimation uses prior knowledge (like general language rules) to help estimate probabilities. Unlike MLE, which only looks at the training data, Bayesian methods combine the data with what we already know about language.

How it differs from MLE:

MLE only uses the training data, but Bayesian estimation uses both the data and prior knowledge. This can make predictions better, especially when the data is limited.

Example: If you know that "the" is usually followed by a noun (like "cat" or "dog"), even if your training data doesn't have many examples, Bayesian estimation can use this rule to improve predictions.

Bayesian Approach:

Prior Belief: Let's say you know that "the" is almost always followed by a noun (based on your understanding of English grammar). You can encode this into your system as a "prior." Let's make up arbitrary values here.

Prior probability that "the" is followed by a noun: 0.9

Prior probability that "the" is followed by something *other* than a noun: 0.1

Data (Likelihood): You observe in your training data:

"the cat": 50 times

"the dog": 40 times

"the quickly": 5 times (adverb)

"the is": 5 times (verb)

Bayesian Calculation (Concept

ual): The Bayesian approach *combines* the prior belief (that it's usually a noun) with the data you've observed. It would *adjust* the probabilities based on both. The system is more likely to predict "cat" or "dog" because of the prior, even if the raw frequency in the data is slightly different. This could be beneficial, in limited data, where something like "the aardvark" occurs zero times, Bayesian learning would be biased towards a noun still.

Large-Scale Language Models

How parameter estimation works in big models (e.g., GPT, BERT):

In large models like GPT or BERT, there are millions or even billions of parameters. **These models use advanced versions of MLE (maximum likelihood estimation)and Bayesian methods to learn from huge amounts of text data.**

They look at patterns in sentences, paragraphs, and even entire documents to predict words.

Challenges with large datasets and complex models:

- **Data Size:** Large models need a lot of data, which can be hard to collect and process.
- **Computational Power:** Training these models requires powerful computers and can take a long time.
- **Overfitting:** Sometimes, the model learns too much from the training data and doesn't work well on new, unseen data.

Example: GPT-3 is trained on billions of sentences from books, websites, and more. It uses this massive data to predict the next word in a sentence. For example, if you type "The sky is", it might predict "blue" because it has seen that combination many times in its training data.

Language Model Evaluation:

What is a Language Model?

A language model is **a tool used in Natural Language Processing (NLP) to understand and predict how words are used in language.** It analyzes patterns in language to guess what word comes next in a sentence. This is useful for tasks like writing suggestions in email or translating languages.

What is Language Model Adaptation?

Language model adaptation is a way to make these models work better for specific tasks or areas of interest. Instead of training a new model from scratch, you can take an existing model and fine-tune it with a smaller amount of data related to your task. This makes the model more accurate and efficient for that particular job

Let us take the concept **of MBBS and related medical field here we need to put our effort to train the language**

model about the concept of how to deal with these words and deal it with all these words after the training.

Techniques for Language Model Adaptation

1. Domain-Specific Fine-Tuning

Imagine you have a model that is good at understanding general language, but you want it to be better at understanding medical terms. You can fine-tune this model using medical texts so it becomes more accurate for medical tasks.

Example: A company wants to develop a chatbot for a hospital. They take a general language model and fine-tune it with medical texts to make it understand medical questions and answers better.

2. Adapter Modules

These are small additions to the model that can be trained for specific tasks without changing the whole model. It's like adding a new tool to a toolbox without replacing the whole toolbox.

Example: You have a language model that is good at translating English to Spanish. You can add an adapter module to make it also translate English to French without retraining the whole model.

3. Low-Rank Adaptation (LoRA)

This method changes the model slightly to make it work better for a task without using too much computer power. It's useful when you don't have a lot of data or computing resources.

Example: A researcher wants to adapt a large language model for a small project but doesn't have enough computing power. They use LoRA to fine-tune the model efficiently.

Real-Time Examples of Language Model Adaptation

Google's Smart Compose: This feature uses language models to suggest the next words in your emails. It is adapted to understand your writing style and the context of your email.

Speech Recognition Systems: These systems use language models to recognize spoken words. They are often adapted to understand different accents and dialects.

Chatbots: Many customer service chatbots use adapted language models to understand and respond to customer queries more accurately.

Text to read for easy understanding:

Language model adaptation is a powerful tool in NLP that allows you to improve the performance of existing models for specific tasks without starting from scratch. By using techniques like domain-specific fine-tuning, adapter modules, and LoRA, you can make language models more efficient and accurate for a wide range of applications.

Types of Language Models:

1. Class-Based Language Models

- **Core Idea:** These models categorize words into classes (like nouns, verbs, adjectives) and use those classes to predict probabilities. Think of it like having labeled boxes for words, and the model guesses based on which box a word belongs to.
- **Strengths:**
 - Simple to understand and implement.
 - Computationally efficient, meaning they don't need a lot of processing power.
 - Can be effective for basic language tasks.
- **Weaknesses:**
 - Limited in capturing complex relationships between words.
 - May not be accurate for tasks requiring nuanced understanding of language.
- **Example:** A class-based model might predict that "cat" is more likely to follow "the" than "run" because "cat" is a noun and typically follows articles.

2. Variable-Length Language Models

- **Core Idea:** These models consider a variable number of preceding words (the context) when predicting the next word. They can capture longer-range dependencies in language.
- **Strengths:**
 - More accurate than class-based models for capturing linguistic patterns.
 - Can handle different sentence structures and word orders.
- **Weaknesses:**
 - Can be more complex to implement and train.
 - May require more data to learn effectively.
- **Example:** A variable-length model might predict that "sun" is more likely to follow "The cat sat in the" than "The cat ran quickly" because it can capture the relationship between "cat" and "sun" over a longer distance.

3. Discriminative Language Models

- **Core Idea:** These models focus on directly predicting the next word based on the entire preceding context. They are often used in tasks where the goal is to generate a sequence of words, like in speech recognition or machine translation.
- **Strengths:**
 - Effective for generating fluent and coherent text.
 - Can be trained on large amounts of data to achieve high accuracy.
- **Weaknesses:**
 - May not capture the underlying structure of language as well as syntax-based models.
 - Can be prone to generating biased or offensive text if not trained carefully.
- **Example:** A discriminative model used in speech recognition might predict that "the" is more likely to follow "I went to" than "store" because it has learned the common phrase "I went to the store."

4. Syntax-Based Language Models

- **Core Idea:** These models incorporate syntactic information (grammar rules) when predicting the next word. They can better understand the relationships between words in a sentence.
- **Strengths:**
 - Improved accuracy for tasks requiring understanding of sentence structure.
 - Can handle complex grammatical constructions.
- **Weaknesses:**
 - Can be more challenging to implement, as they need to parse sentences.
 - May struggle with ungrammatical or informal language.

- **Example:** A syntax-based model might predict that "is" is more likely to follow "The dog" than "running" because it understands that "The dog" is a noun phrase that requires a verb.

5. MaxEnt Language Models

- **Core Idea:** These models use the principle of maximum entropy (to measure any thing) to estimate probabilities. They try to find the most uniform probability distribution that is consistent with the observed data.
- **Strengths:**
 - Flexible and can incorporate various features of the language.
 - Can handle sparse data effectively.
- **Weaknesses:**
 - Can be computationally expensive to train.
 - May require careful feature engineering.
- **Example:** A MaxEnt model might use features like the previous word, the part-of-speech tag of the previous word, and the presence of certain keywords to predict the next word.

6. Factored Language Models

- **Core Idea:** These models break down the task of language modeling into smaller, more manageable subproblems. They can handle long-range dependencies and complex interactions between words.
- **Strengths:**
 - Can capture more complex patterns in language.
 - Scalable to large datasets and vocabularies.
- **Weaknesses:**
 - Can be more difficult to design and train.
 - May require careful selection of factors.
- **Example:** A factored language model might use separate factors for word order, grammatical relations, and semantic roles to predict the next word.

7. Other Tree-Based Language Models

- **Core Idea:** These models represent sentences as tree-like structures, where nodes represent words and edges represent relationships between words. They are often used for parsing and understanding the grammatical structure of sentences.
- **Strengths:**
 - Can capture hierarchical relationships between words.
 - Useful for tasks requiring understanding of sentence structure.
- **Weaknesses:**
 - Can be complex to implement and train.
 - May struggle with ambiguous sentences.
- **Example:** A tree-based model might represent the sentence "The cat sat on the mat" as a tree with "sat" as the root node, "cat" and "mat" as child nodes, and "on" as a link between them.

8. Bayesian Topic-Based Language Models

- **Core Idea:** These models incorporate topic information into language modeling. They assume that words are generated from a mixture of topics, and they use Bayesian inference to estimate the probability of a word given its topic.
- **Strengths:**
 - Can capture the semantic meaning of text.

- Useful for tasks like text classification and information retrieval.
- **Weaknesses:**
 - Can be computationally expensive.
 - May require careful selection of topics.
- **Example:** A Bayesian topic-based model might predict that "doctor" is more likely to follow "The patient went to the" than "restaurant" because it has learned that the sentence is about healthcare.

9. Neural Network Language Models

- **Core Idea:** These models use artificial **neural networks to learn complex patterns in language from large amounts of data.** They are the most popular type of language model today due to their high accuracy and flexibility.
- **Strengths:**
 - Can capture intricate relationships between words.
 - Achieve state-of-the-art performance on various NLP tasks.
 - Can be adapted to different languages and domains.
- **Weaknesses:**
 - Can be computationally expensive to train.
 - Can be difficult to interpret.
 - May generate biased or offensive text if not trained carefully.
- **Example:** Neural language models are used in a wide range of applications, including **machine translation, question answering,** text summarization, and chatbots.

Language Model Type	Core Idea	Strengths	Weaknesses	Example
Class-Based	Assigns probabilities based on word classes (e.g., noun, verb).	Simple, efficient.	Limited accuracy, misses complex patterns.	Predicts "cat" after "the" based on word classes.
Variable-Length	Considers a variable context of preceding words.	Captures longer-range dependencies, more accurate.	More complex, requires more data.	Predicts "sun" after "The cat sat in the."
Discriminative	Directly predicts the next word based on the entire context.	Generates fluent text, high accuracy with large data.	May not capture underlying structure, potential for bias.	Predicts "the" after "I went to" in speech recognition.
Syntax-Based	Incorporates grammar rules (syntax) in prediction.	Improved accuracy for meaning and structure.	Challenging to implement, struggles with ungrammatical language.	Predicts "is" after "The dog."
MaxEnt	Uses maximum entropy estimation for probabilities.	Flexible, handles sparse data.	Computationally expensive.	Uses features like previous word and keywords for prediction.
Factored	Decomposes the problem into smaller subproblems.	Captures complex patterns, scalable.	Difficult to design and train.	Uses factors for word order, grammar, and semantics.
Tree-Based	Represents sentences as tree structures.	Captures hierarchical relationships.	Complex, struggles with ambiguity.	Represents "The cat sat on the mat" as a tree.

Bayesian Topic-Based	Incorporates topic information using Bayesian inference.	Captures semantic meaning.	Computationally expensive.	Predicts "doctor" after "The patient went to the."
Neural Network	Uses artificial neural networks to learn from data.	High accuracy, flexible, adaptable.	Computationally expensive, difficult to interpret, potential for bias.	Used in machine translation, question answering, etc.

Language specific modelling problems:

Language-specific modeling problems in NLP happen when a model struggles because each language has unique rules, words, or structures. For example, word order, grammar, or idioms in Hindi differ from English, making direct translation or processing hard. These problems require models to adapt specifically to the quirks of each language.

Where We Use NLP (Language-Specific Challenges Apply)

1. **Chatbots & Virtual Assistants** (Customer support, AI agents)
2. **Machine Translation** (Google Translate, DeepL)
3. **Speech Recognition** (Voice commands, subtitles, transcriptions)
4. **Sentiment Analysis** (Social media, reviews, stock market trends)
5. **Legal & Medical Document Processing** (Law firms, hospitals, government offices)
6. **Fake News Detection & Spam Filtering** (Social media, email filtering)
7. **Low-Resource Language Processing** (Regional language apps, government services)

1. Word Segmentation (Spaces in Words)

- **Problem:** Some languages, like Chinese and Japanese, do not use spaces between words, making it hard for NLP models to separate words.
- **Example:**
 - English: "I love NLP." (Easy to separate words)
 - Chinese: "我喜欢自然语言处理。" (The model must figure out where each word starts and ends)

2. Morphological Complexity (Word Forms)

- **Problem:** Some languages, like Finnish and Turkish, add many suffixes to words, making it hard for models to understand meaning.
- **Example:**
 - English: "house" → "houses" (simple plural form)

- Turkish: "ev" (house) → "evlerimden" (from my houses)
(A single word has multiple meanings inside it!)

3. Word Order (Sentence Structure)

- **Problem:** Different languages follow **different word orders**, making translation and sentence generation hard.
- **Example:**
 - English: "I eat an apple."
 - Japanese: "私はりんごを食べる。" (I apple eat.)
 - German: "Ich esse einen Apfel." (I eat an apple.)
(Some languages place the verb at the end!)

4. Ambiguity (Same Words, Different Meanings)

- **Problem:** Some words have multiple meanings depending on the context.
- **Example:**
 - English: "bank" (Could mean a financial institution or a riverbank)
 - Hindi: "कल" (Could mean "yesterday" or "tomorrow" based on context)

5. Pronoun Usage (Gender and Formality)

- **Problem:** Some languages have gender-specific words or different levels of politeness that affect translation.
- **Example:**
 - English: "You" (same for everyone)
 - French: "Tu" (informal "you") vs. "Vous" (formal "you")
 - Hindi: "तू" (very informal), "तुम" (casual), "आप" (respectful)

6. Low-Resource Languages (Not Enough Data)

- **Problem:** Some languages, like many **African and Indigenous languages**, have very little digital data for NLP training.
- **Example:**
 - English has millions of Wikipedia articles, but languages like Khasi (spoken in India) have very few online resources.

7. Code-Switching (Mixing Languages)

- **Problem:** People mix languages while speaking or typing, confusing NLP models.
- **Example:**
 - "Mujhe NLP bahut interesting lagta hai." (Hindi + English)
("I find NLP very interesting.")

Multilingual and Cross-Lingual Modelling in NLP

In NLP, we often deal with multiple languages. There are two main approaches:

1. **Multilingual Modelling** (One Model, Many Languages)
2. **Cross-Lingual Modelling** (Understanding Across Languages)

1. Multilingual Modelling

What is it?

- A single AI model is trained on multiple languages.
- It can process and understand different languages but treats them separately.

Real-Life Example:

- **Google Translate:** Can translate between English, Spanish, Hindi, and more using one model.
- **Multilingual Chatbots:** Customer support bots that understand English, French, and German in the same system.

Challenge:

- Some languages have more training data (like English), so less common languages may not be as accurate.

2. Cross-Lingual Modelling

What is it?

- The model understands and transfers knowledge between languages.
- It allows a system trained in one language to work in another without much extra training.

Real-Life Example:

- **Cross-Lingual Search:** You type in English, but the system finds relevant documents in Hindi.
- **Zero-Shot Translation:** A model trained to translate English-French can also handle English-German without explicit training.

Challenge:

- Different languages have unique grammar and sentence structures, making accurate transfer difficult.

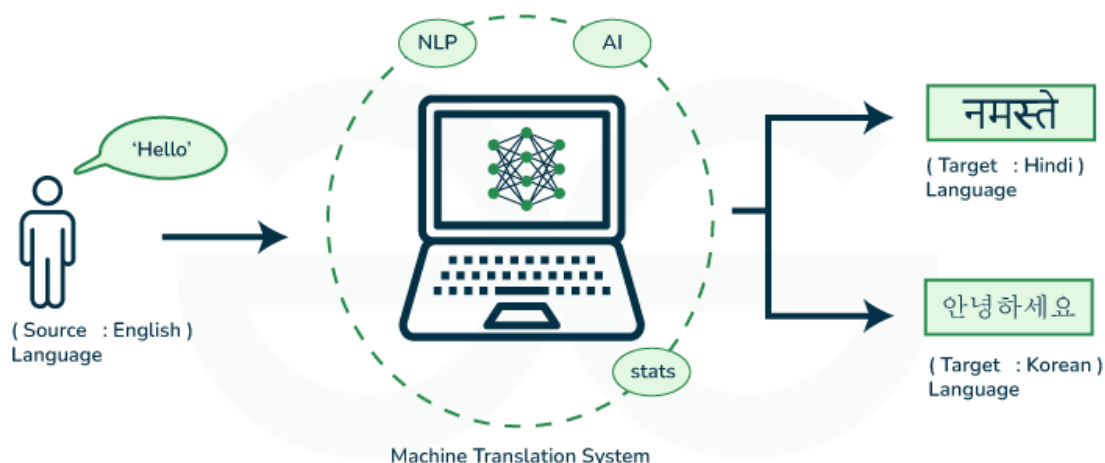
Feature	Multilingual Modelling	Cross-Lingual Modelling
Works with multiple languages?	Yes	Yes
Treats languages separately?	Yes	No
Knowledge transfer between languages?	No	Yes
Example	Google Translate (Standard)	Google Search finding answers in multiple languages

CHAPTER 5

Unit 5

What is Machine Translation?

Machine translation is a sub-field of computational linguistics that focuses on developing systems capable of automatically translating text or speech from one language to another. In Natural Language Processing (NLP), the goal of machine translation is to produce translations that are not only grammatically correct but also convey the meaning of the original content accurately.



History of Machine Translation

The automatic translation of text from one natural language (the source) to another is known as machine translation (the target). It was one of the first applications for computers that were imagined (Weaver, 1949).

There have been three primary uses of machine translation in the past:

1. Rough translation, such as that given by free internet services, conveys the “gist” of a foreign statement or document but is riddled with inaccuracies. Companies utilize pre-edited translation to publish documentation and sales materials in several languages.
2. The original source content is written in a limited language that makes machine translation easier, and the outputs are often edited by a person to rectify any flaws.

3. Restricted-source translation is totally automated, but only for highly stereotyped language like a weather report.

Machine Translation Approaches:

What are the key approaches in Machine Translation?

In machine translation, the original text is decoded and then encoded into the target language through two step process that involves various approaches employed by language translation technology to facilitate the translation mechanism.

1. Rule-Based Machine Translation

Rule-based machine translation relies on these resources to ensure precise translation of specific content. The process involves the software parsing input text, generating a transitional representation, and then **converting it into the target language with reference to grammar rules and dictionaries.**

A government office in Andhra Pradesh wants to translate official documents from Telugu to English using a rule-based machine translation system.

How RBMT Works in This Case:

Parsing the Input Text:

- The system takes a Telugu sentence:
"ప్రభుత్వవిధానాలుప్రజలఅభివృద్ధికిదోహదపడతాయి."
- It breaks it down into words and grammar components.

Generating a Transitional Representation:

- The system identifies:
 - "ప్రభుత్వ" → "Government" (Noun)
 - "విధానాలు" → "Policies" (Noun)
 - "ప్రజల" → "People's" (Possessive noun)
 - "అభివృద్ధి" → "Development" (Noun)
 - "దోహదపడతాయి" → "Help" (Verb)

Applying Grammar Rules & Dictionaries:

- Telugu follows Subject-Object-Verb (SOV) order.
- English follows Subject-Verb-Object (SVO) order.
- The system **rearranges** the words according to English grammar rules.

Generating the Final Translation:

- Telugu: "ప్రభుత్వవిధానాలుప్రజలఅభివృద్ధికిదోహదపడతాయి."
- Direct Rule-Based Translation (Wrong Order): "Government policies people's development help."
- Corrected Output: "Government policies help in the development of people."

2. Statistical Machine Translation

Rather than depending on linguistic rules, [statistical machine translation](#) utilizes machine learning for text translation. Machine learning algorithms examine extensive human translations, identifying statistical patterns. When tasked with translating a new source text, the software intelligently guesses

based on the statistical likelihood of specific words or phrases being associated with others in the target language.

Scenario:

A Telugu news website wants to automatically translate its articles into English using **Statistical Machine Translation (SMT)**.

How SMT Works in This Case:

Learning from Large Data Sets:

- The system is trained on thousands of Telugu-to-English translated news articles.
- It learns which Telugu words are most commonly translated into specific English words or phrases.

Identifying Statistical Patterns:

- Suppose the system finds that in most translations:
 - "ఆర్థికవ్యవస్థ" is translated as "economy"
 - "అభివృద్ధి" is translated as "development"
 - "ప్రభుత్వనిధులు" is translated as "government funds"

Translating a New Sentence Using Probability:

- New Telugu text:
"ప్రభుత్వనిధులు ఆర్థికవ్యవస్థ అభివృద్ధికి ఉపయోగపడతాయి."
- Based on statistical probability, the system generates:
"Government funds help in the development of the economy."

Adjusting for Accuracy:

- If multiple translations exist for a word, the system picks the most statistically likely one.
- For example, "ఆర్థికవ్యవస్థ" could be translated as "financial system", but if "economy" appears more frequently in past translations, it chooses "economy" instead.

3. Neural Machine Translation (NMT)

A [neural network](#), inspired by the human brain, is a network of interconnected nodes functioning as an information system. Input data passes through these nodes to produce an output. Neural machine translation software utilizes neural networks to process vast datasets, with each node contributing a specific change from source text to target text until the final result is obtained at the output node.

Scenario:

A tech company is building an AI-powered chatbot that can translate customer queries from Telugu to English using Neural Machine Translation (NMT).

How NMT Works in This Case:

Training the Neural Network with Large Datasets

- The system is trained using **millions of Telugu-English sentence pairs** from conversations, books, and websites.
- Unlike **rule-based** or **statistical** methods, NMT **understands full sentences** instead of translating word by word.

Using Context Instead of Just Words

- If a customer types "మీరునాకుసహాయంచేయగలరా?"
- NMT **doesn't just translate each word** separately; it understands that this is a polite request.
- Possible translations:
 - "Can you help me?" ✓ (Natural)
 - "You able to help me?" ✗ (Awkward)
- Since "Can you help me?" is a more natural phrase, the model picks it.

Handling Complex Sentences More Accurately

- Telugu: "ఈసాఫ్ట్‌వేర్‌నియోగదారులకుఉపయోగపడేలాడిజైన్ చేయబడింది."
- Word-by-word translation (Incorrect): "This software users useful designed."
- NMT translation (Correct): "This software is designed to be useful for users."
- The neural network **rearranges the words** to match the natural English structure.

Continuous Learning & Improvement

- Over time, NMT **keeps improving** by learning from new translations and user feedback.
- If many users correct a translation, the system **adjusts itself** to produce better results in the future.

4. Hybrid Machine Translation

Hybrid machine translation tools integrate multiple machine translation models within a single software application, leveraging a combination of approaches to enhance the overall effectiveness of a singular translation model. This process typically involves the incorporation **of rule-based and statistical machine** translation subsystems, with the ultimate translation output being a synthesis of the results generated by each **subsystem**.

Is machine translation possible? Or Why we need Machine Translation in NLP?

Machine translation in Natural Language Processing (NLP) has several benefits, including:

1. **Improved communication:** Machine translation makes it easier for people who speak different languages to communicate with each other, breaking down language barriers and facilitating international cooperation.

2. **Cost savings:** Machine translation is typically faster and less expensive than human translation, making it a cost-effective solution for businesses and organizations that need to translate large amounts of text.
3. **Increased accessibility:** Machine translation can make digital content more accessible to users who speak different languages, improving the user experience and expanding the reach of digital products and services.
4. **Improved efficiency:** Machine translation can streamline the translation process, allowing businesses and organizations to quickly translate large amounts of text and improving overall efficiency.
5. **Language learning:** Machine translation can be a valuable tool for language learners, helping them to understand the meaning of unfamiliar words and phrases and improving their language skills.

Problems in Machine Translation

Machine translation is useful, but it has some challenges. Here are the main problems and their explanations:

Incorrect Word Meanings

- ◆ Sometimes, the machine picks the **wrong meaning** of a word because many words have multiple meanings.
- ◆ This happens because machines **don't always understand the context** like humans do.

✓**Example:**

Telugu: "ఆమెగదినువదిలివెళ్లిపోయింది."

Wrong translation: "She left the stomach." ✗

Correct translation: "She left the room." ✓ (Here, "గది" means "room," not "stomach")

Poor Sentence Structure

- ◆ Machines often **translate word by word** instead of rearranging sentences correctly.
- ◆ This makes the translation sound **unnatural** or **confusing**.

✓**Example:**

Telugu: "నేనుసినిమాచూశాను."

Wrong translation: "I movie watched." ✗

Correct translation: "I watched a movie." ✓

Struggles with Cultural Differences

- Some words or phrases are **specific to a culture** and **don't have direct translations**.
- Machines **cannot understand emotions, jokes, or idioms** properly.

✓Example:

Telugu: "మీరు నన్ను మోసంచేశారు."

Direct translation: "You cheated me." ✗

Better translation: "You betrayed me." ✓ (More natural in English)

Difficulty with Complex Sentences

- Long or complicated sentences **confuse** the machine.
- It may **split the meaning incorrectly** or **skip important details**.

Example:

Telugu: "ఈ పుస్తకం చదివితే మీకు మంచి విజ్ఞానం వస్తుంది."

Wrong translation: "This book reads you good knowledge comes." ✗

Correct translation: "Reading this book will give you good knowledge." ✓

Translation Errors in Technical or Legal Language

- Machines **struggle with technical, legal, or medical terms** because they require **specialized knowledge**.
- A small mistake in these areas **can change the entire meaning** of a sentence.

Example:

Telugu (Legal term): "కోర్టు తీర్పు దోషిని ఉరితీయాలని ఆదేశించింది."

Wrong translation: "The court ordered to hang the guilty." ✗

Correct translation: "The court sentenced the guilty to death." ✓ (More legally accurate)

Lack of Human Emotion & Tone

- Machines **cannot understand** the **tone, sarcasm, or emotional depth** of a sentence.
- Translations might sound **too robotic** or **lose the real feeling**.

✓Example:

Telugu: "బహుశా నువ్వు చాలా తెలివైనవాడివి!" (Sarcasm)

Wrong translation: "Maybe you are very intelligent!" ✗

Correct translation: "Oh, so you're really smart!" ✓ (Captures sarcasm better)

Errors in Speech-to-Text Translations

- ◆ When **spoken language** is translated, errors happen due to **accent, pronunciation, or unclear speech**.
- ◆ This affects **voice assistants** and **live translations**.

✓Example:

Spoken Telugu: "నేనుమంచిఆరోగ్యాన్నికోరుకుంటున్నాను."

Wrong translation: "I am asking for a good hospital." ✗

Correct translation: "I wish for good health." ✓

What is Anusaaraka?

Anusaaraka is a special computer program designed to help people understand texts in different Indian languages. It translates English into various Indian languages like Hindi, using unique methods based on ancient Indian grammar rules, specifically Pāṇini's Ashtadhyayi.

Background

Anusaaraka was developed by the Chinmaya International Foundation (CIF) at the International Institute of Information Technology, Hyderabad (IIIT-H), and the University of Hyderabad's Department of Sanskrit Studies. The name "Anusaaraka" comes from the Sanskrit word "anusaran," meaning "to follow." This reflects how the program shows translations step by step, allowing users to see how the translation process works.

How It Works

Layered Translation: Anusaaraka translates texts in layers, showing each step of the translation process. This helps users understand how the final translation was made and ensures that the original meaning is preserved as much as possible.

Reversibility: Unlike other translation systems, Anusaaraka allows users to go back to previous steps in the translation process without losing any information. This feature is very useful for bilingual users who want to check the original text.

Transparency: The program displays each step of the translation clearly, giving users confidence in the accuracy of the translation. Users can trace back to the source text and understand the context better.

Purpose

Anusaaraka aims to bridge the language gap in India by making English texts accessible in various Indian languages. It is particularly useful for people whose primary language is not English but need to access information available mainly in English.

Key Features

Faithfulness to the Source: Anusaaraka prioritizes conveying the original meaning over producing perfect sentences in the target language.

Unique Translation Approach: It uses Pāṇini's grammar rules, which are well-suited for Indian languages that have a free word order.

Layered Output: This allows users to see how the translation is done step by step.

Overall, Anusaaraka is a unique tool that combines traditional Indian knowledge with modern technology to help people access information across different languages.

Cutting the Gordian Knot is a phrase that comes from an ancient story. It means finding a simple solution to a very difficult problem. Here's how it relates to Anusaaraka:

The Story of the Gordian Knot

In ancient times, there was a king named Gordius who tied a very complicated knot around a chariot. A prophecy said that whoever could untie this knot would become the ruler of all of Asia. Many people tried but failed. Then, Alexander the Great came and solved the problem by cutting the knot with his sword instead of trying to untie it.

Applying This to Anusaaraka

In the context of Anusaaraka, "cutting the Gordian Knot" means finding a new and effective way to solve the complex problem of translating English into Indian languages. Anusaaraka does this by using a unique approach based on ancient Indian grammar rules, which simplifies the translation process and makes it more accurate.

How Anusaaraka "Cuts the Knot"

Simplifying Translation: Anusaaraka uses a layered approach to translation, breaking down **complex sentences into simpler parts**. This makes it easier to translate accurately.

Using Ancient Wisdom: By applying Pāṇini's grammar rules, **Anusaaraka finds a straightforward way to handle the complexities of Indian languages, which often have free word order**.

Transparency: The program shows each step of the translation process, allowing users to understand how the translation was made. This transparency helps build trust in the accuracy of the translation.

In summary, Anusaaraka "cuts the Gordian Knot" of language translation by providing a clear, step-by-step approach that simplifies the process and ensures accuracy, much like Alexander the Great solved the puzzle of the Gordian Knot with a bold and innovative solution.

WHY IS THE GORDIAN KNOT IMPORTANT?

The concept of "cutting the Gordian Knot" originates from an ancient Greek legend associated with Alexander the Great. It symbolizes finding a bold and innovative solution to a complex problem. Here's how this metaphor is significant across different cultures:

Cultural Significance

Problem-Solving Metaphor: In many cultures, cutting the Gordian Knot **represents the idea of solving seemingly intractable problems with creative or forceful solutions**. It emphasizes the importance of thinking outside the box or using unconventional methods to achieve success.

Leadership and Power: The legend is often linked to leadership and conquest, as Alexander's action was seen as a demonstration of his genius and destiny to rule Asia. This aspect resonates with cultures that value strong leadership and strategic decision-making.

Symbolism of Change: The act of cutting the knot can symbolize breaking free from traditional constraints or outdated systems. It represents a willingness to challenge existing norms and embrace

change, which is a theme found in various cultural narratives.

Interpretations Across Cultures

Western Cultures: In Western cultures, the Gordian Knot is often used to illustrate the power of innovative thinking and bold action. It is seen as a symbol of overcoming obstacles through determination and creativity.

Eastern Cultures: While the specific legend may not be as deeply rooted in Eastern cultures, the idea of finding novel solutions to complex problems resonates universally. For example, in Indian culture, similar themes of overcoming challenges through wisdom and strategic thinking are found in ancient texts like the Mahabharata.

Modern Usage: Today, the phrase "cutting the Gordian Knot" is used across cultures to describe situations where a complex problem is solved through a bold or unconventional approach. It is often applied in business, politics, and technology to highlight innovative solutions.

Conclusion

The significance of cutting the Gordian Knot lies in its universal appeal as a metaphor for creative problem-solving and strategic decision-making. It transcends cultural boundaries, symbolizing the power of bold action and innovative thinking in overcoming seemingly insurmountable challenges.

Gordian knot in the NLP with ANUSARAKA:

The term **Gordian Knot** represents a **complex problem** that is difficult to solve in a straightforward way. In language translation, understanding and maintaining **meaning, context, and grammar** is a very complex task—like an unsolvable knot.

Anusaraka tries to "**cut**" **this Gordian Knot** by:

1. **Using Machine Learning & NLP:** It simplifies translation instead of making it perfect.
2. **Focusing on Meaning over Exact Words:** Even if the sentence is not fully accurate, the reader can understand the essence.
3. **Handling Complex Indian Languages:** Since Indian languages share **common roots**, Anusaraka takes advantage of these similarities to provide a smart approximation.

Anusaraka in NLP (with Gordian Knot Concept)

Scenario: Translating Hindi to English

Suppose you have a Hindi sentence:

☞ "राम बाजार गया और आम खरीदा।"

A normal translation tool might give:

✗ "Ram market went and mango bought." (Incorrect grammar)

But **Anusaraka**, instead of trying to create a perfect sentence, would give a structured translation like:

✓ "Ram went to the market and bought a mango."

Even though it does not perfectly restructure the sentence, it **preserves the meaning** while keeping the Hindi sentence structure visible.

Gordian Knot Connection in This Example

The Challenge (The Knot):

- Indian languages **have** different word orders than English.
- Some words **don't have exact translations** in other languages.

The Solution (Cutting the Knot):

- Instead of trying to make the sentence sound perfect, Anusaraka gives a "**meaning-focused translation**" that still keeps the original structure.
- This allows people to **understand the sentence even if they don't know the language well**.

Thus, Anusaraka simplifies complex language translation challenges, much like cutting the Gordian Knot instead of trying to untangle it!

1. The Problem with Machine Translation

- **Goal:** Building a fully automatic, high-quality machine translation system (FGH-MT) is very hard.
- **Why?**
 - Translation is a creative process. It requires understanding the text, interpreting it, and adapting it for the audience.
 - Machines lack **world knowledge** (common sense) and cannot interpret text like humans.
 - Example: In Hindi, "chAvala rAma khAtA hE" could mean "Ram eats rice" or "Rice eats Ram." Humans know rice can't eat, but machines don't.

2. Challenges in Machine Translation

- **World Knowledge:** Machines don't know basic facts (e.g., people eat rice, not the other way around).
- **Metaphors and Exceptions:** Language often uses metaphors or irony, which override common sense. For example, "Alcohol drinks you" makes sense in a poetic way, but machines can't handle such exceptions easily.
- **Ellipses (Missing Information):** Sentences often leave out parts that humans can infer, but machines struggle with this.
- **Complex Reasoning:** Understanding long texts or chains of reasoning is hard for machines.

3. The Solution: Anusaaraka Approach

Instead of trying to build a perfect machine translation system, the **Anusaaraka approach** divides the task into two parts:

Step 1: Core Anusaaraka (Language-Based Analysis)

- This module focuses on **language knowledge**.
- It takes the source text and converts it into an **intermediate language** close to the target language.
- Example: Translating Telugu to Hindi by mapping words and grammar structures.

Step 2: Domain-Specific Module (World Knowledge)

- This module uses **world knowledge, statistics, and context** to refine the translation.
- Example: Deciding whether "Ram eats rice" or "Rice eats Ram" is correct based on common sense.

4. Advantages of the Anusaaraka Approach

1. **Early Availability:** The first module is easier to build, so the system becomes usable sooner.
2. **Early Feedback:** Users can start using the system early, which helps improve the second module.
3. **Robustness:** The first module is more reliable, even if the second module fails.
4. **Clear Task Division:** Separating language knowledge and world knowledge makes it easier to build the system.
5. **Evolving Intermediate Language:** Over time, the intermediate language improves naturally as more languages are added.
6. **Easier for Developers:** Developers only need to know the intermediate language, not both source and target languages.
7. **Beyond Translation:** The Anusaaraka approach can also be used for studying texts (e.g., law books or classics) without full translation.

5. How Anusaaraka Works

- **Mapping Constructions:** It maps words and grammar from the source language to the target language.
 - Example: Telugu "mIru pustakaM caduvutunnArA?" becomes Hindi "Apa pustaka paDha rahe hE kyA?" (Are you reading a book?).
- **Handling Differences:** If a construction doesn't exist in the target language, it finds a similar way to express it.
 - Example: Telugu participial verbs are mapped to Hindi relative clauses.
- **Adding Notation:** Sometimes, extra notation is added to preserve meaning that would otherwise be lost.

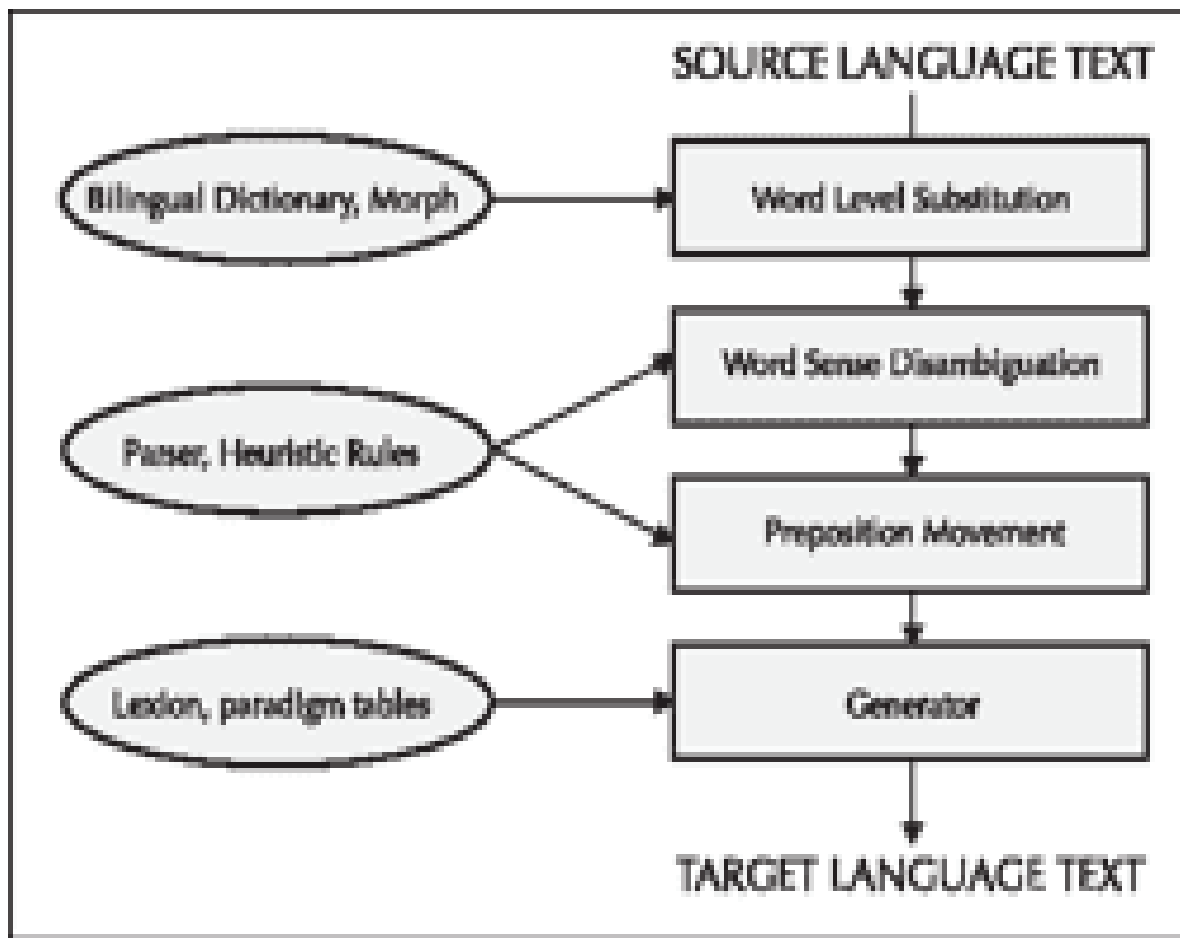
6. Challenges in Anusaaraka

- **Fragile Second Module:** The second module (world knowledge) can fail, especially with metaphors or exceptions.
- **Unexpected Errors:** Proper nouns might get mistranslated (e.g., "Nalini" becoming "illness").
- **User Training:** Users need some training to understand the intermediate language output.

7. Conclusion

- The **Anusaaraka approach** splits machine translation into two parts: language-based analysis and world knowledge processing.
- It's easier to build and more robust than trying to create a fully automatic system.
- For Indian languages, this approach works well because they share vocabulary, grammar, and culture.
- A similar approach can be used for English to Hindi translation, though it might require more user training.

Anusaraka architecture with working concept:



Anusaaraka Architecture Overview

Anusaaraka is a machine translation system designed to translate text from a **source language** to a **target language** by breaking the process into two main modules:

1. Core Anusaaraka (Language-Based Analysis):

- Focuses on linguistic knowledge (grammar, morphology, syntax).
- Converts the source language text into an **intermediate representation** close to the target language.

2. Domain-Specific Module (World Knowledge-Based Processing):

- Uses world knowledge, context, and statistical methods to refine the translation.
- Produces the final output in the target language.

Anusaaraka Architecture

1. Bilingual Dictionary, Morph

• Word Level Substitution:

- The core Anusaaraka uses a bilingual dictionary to replace source language words with target language equivalents.
- Example: Translating "book" (English) to "kitab" (Hindi).

• Word Sense Disambiguation:

- Resolves ambiguity in word meanings using context.
- Example: Deciding whether "bank" refers to a riverbank or a financial institution.

• Preposition Movement:

- Adjusts prepositions to match the grammatical rules of the target language.
- Example: In Hindi, prepositions often come after the noun (postpositions), unlike English.

- **Generator:**

- Generates the intermediate or final text in the target language after processing.

2. Passer, Heuristic Rules

- **Heuristic Rules:**

- These are rules of thumb applied during translation to handle exceptions, idiomatic expressions, or complex constructions.
- Example: Handling verb conjugations or noun declensions based on paradigm tables.

- **Lesson, Paradigm Tables:**

- Predefined tables for verb conjugations, noun declensions, and other grammatical rules.
- Example: A table for conjugating the verb "to be" in the target language.

3. Source Language Text and Target Language Text

- **Source Language Text:**

- The input text in the original language (e.g., English, Hindi, Telugu).

- **Target Language Text:**

- The final output text in the desired language after translation.

Connection to Anusaaraka

The components described in the image are part of the **Core Anusaaraka module**, which focuses on **language-based analysis**. Here's how they fit:

1. **Bilingual Dictionary and Morphological Analysis:**

- Used to map words and grammatical structures from the source language to the target language.

2. **Heuristic Rules and Paradigm Tables:**

- Applied to handle exceptions, idiomatic expressions, and complex grammatical constructions.

3. **Intermediate and Final Output:**

- The Core Anusaaraka produces an intermediate representation, which is further refined by the domain-specific module to generate the final target language text.

Example of Anusaaraka in Action

Source Language Text (English):

"The boy eats an apple."

Core Anusaaraka Processing:

1. **Word Level Substitution:**

- "boy" → "ladka" (Hindi)
- "eats" → "khata hai" (Hindi)
- "apple" → "seb" (Hindi)

2. **Preposition Movement:**

- Adjusts word order to match Hindi syntax: "ladka seb khata hai."

3. **Generator:**

- Produces the intermediate Hindi text: "लड़कासेबखाताहै।"

Domain-Specific Module (Optional):

- Refines the translation further if needed, based on context or world knowledge.

Target Language Text (Hindi):

"लड़कासेबखाताहै।"

Anusaraka Interface

What is the Anusaraka Interface?

The **Anusaraka Interface** is a tool designed to help people understand and translate content from one language to another, especially from English to Indian languages. It does not give a perfect translation but provides a "language bridge"—helping users understand the general meaning of a sentence even if they are not fluent in the source language.

How Does It Work?

It follows a **rule-based approach**, meaning it breaks down a sentence, understands its structure, and then translates it while keeping the meaning intact. Unlike Google Translate, which uses AI and large datasets, **Anusaraka relies on grammar rules and linguistic patterns.**

Real-Life Examples to Understand Anusaraka Interface

Example 1: Translating a News Article

Suppose you are reading an English news article:

"The government has launched a new scheme for farmers to improve crop production."

You are a Hindi speaker and use Anusaraka. It may provide:

"सरकारनेकिसानोंकेलिएएकनईयोजनाशुरूकीहैताकिफसलउत्पादनमेंसुधारहो।"

How it helps?

- Even if the sentence structure is not perfect, you can still **understand the key idea** of the news.

Example 2: Understanding a Science Concept

You are a Telugu speaker and reading:

"Photosynthesis is the process in which plants convert sunlight into energy."

Anusaraka translates it into:

"ఫోటోసింథసిస్ అనేది మొక్కలు సూర్యకాంతిని శక్తిగా మారుస్తాయి."

How it helps?

- The **word-for-word translation** may not be grammatically perfect, but it helps you grasp the concept.

Where is the Anusaraka Interface Used?

1. **Education** – Students learning English can understand textbooks or articles in their native language.
2. **Government Services** – Official documents can be translated for people who don't know English.
3. **News & Media** – Helps regional language speakers understand English news.
4. **Communication** – Useful for people who want to understand basic English without deep knowledge of grammar.

Main technical understanding:

Not a perfect translation tool, but a meaning-based assistance tool.

Helps understand foreign language text in a simplified way.

Best for Indian languages, especially for English-to-Indian language translation.

Linguistics and the Anusaraka Interface

Introduction to Linguistics

Linguistics is the scientific study of language—how it is structured, how it evolves, and how humans use it to communicate. It includes:

- **Phonetics & Phonology** – Sounds and pronunciation
- **Morphology** – Word formation
- **Syntax** – Sentence structure
- **Semantics** – Meaning of words
- **Pragmatics** – Context-based meaning

Understanding linguistics is essential for **language processing tools like Anusaraka**, which aim to help people understand languages they are not fluent in.

How Does Anusaraka Use Linguistics?

Anusaraka is **not just a simple translation tool**. It uses linguistic rules to convert English text into Indian languages while keeping the original meaning intact.

Key Linguistic Concepts in Anusaraka

1. **Syntax Mapping (Sentence Structure Matching)**
 - English: "She is eating an apple."
 - Hindi: "वह एक सेब खा रही है।"
 - Telugu: "ఆమె ఒక యాపిల్ ని ఉండోంది."
 - Tamil: "அவள் ஒரு ஆப்பிள் சாப்பிடுகிறாள்."
 - Anusaraka **rearranges the sentence structure** to match the target language.

2. Morphological Analysis (Word Forms & Inflections)

- English: "He runs fast."
- Hindi: "वह तेज़ दौड़ता है।" (दौड़+ता → verb conjugation)
- Anusaraka understands how verbs change based on gender, tense, and plurality.

3. Semantics (Meaning Retention)

- If a word has multiple meanings, Anusaraka chooses the most suitable one based on context.
- English: "Bank" (riverbank vs. financial institution)
- Hindi: "किनारा" (if river context) or "बैंक" (if financial context).

Why Is Linguistics Important for Anusaraka?

- ✓Helps provide more accurate translations
- ✓Handles complex sentence structures
- ✓Understands language rules and variations
- ✓Makes Indian language learning easier

Real-Life Applications of Anusaraka in Linguistics

- ◆**Education** – Helps students understand English-based study material.
- ◆**Translation** – Assists in converting official documents into native languages.
- ◆**News and Media** – Enables local-language speakers to understand global news.
- ◆**Government & Administration** – Helps policymakers communicate effectively with non-English speakers.

Conclusion

Anusaraka **bridges the gap between English and Indian languages** by using **linguistic principles** to improve translation accuracy. It is not just about word-to-word translation but ensuring **meaning-based understanding**, making language accessible to more people.

What is "Giving Up Agreement" in Anusaraka?

Anusaraka is a **language translation tool** that helps in translating Indian languages into English while maintaining the **original meaning and grammar structure**. However, sometimes certain language structures **cannot be directly translated**, and some words or grammatical rules need to be **"given up"** (ignored) to make the translation **clear and meaningful** in English. This process is called **"Giving Up Agreement."**

Why is "Giving Up Agreement" Needed?

Indian languages (like Hindi, Telugu, or Kannada) **follow different grammar rules** than English. If we translate word-for-word, the meaning might become **incorrect or confusing**. In such cases, Anusaraka **modifies or removes** certain agreements (like gender, number, or tense) to produce **correct English sentences**.

Real-Life Example of Giving Up Agreement

Example 1: Gender Agreement in Hindi → English

Hindi Sentence:

रामस्कूलजाताहै। (Rām skūl jātā hai.)

Word-for-word translation:

Ram school goes is. ✗ (Incorrect in English)

Correct translation after giving up agreement:

Ram goes to school. ✓

What happened?

- In Hindi, "जाताहै" (jātā hai) has a **gender-based verb agreement** (जाता for male, जाती for female).
- In English, **verbs do not change based on gender**.
- So, **Anusaraka removes gender agreement** to make the correct translation.

Example 2: Plural Agreement in Telugu → English

Telugu Sentence:

వారుపుస్తకాలుచదువుతున్నారు. (Vāru pustakālu caduvutunnāru.)

Word-for-word translation:

They books reading are. ✗ (Incorrect)

Correct translation after giving up agreement:

They are reading books. ✓

What changed?

- Telugu verbs change based on **plural subjects**, but English does not require this.
- The verb structure was **simplified** by Anusaraka.

Final Thoughts

"Giving Up Agreement" is an important part of **Anusaraka's translation process**. It helps to:

Remove unnecessary grammar rules that don't exist in English.

Ensure the **translated sentence sounds natural** in English.

Keep the **original meaning intact** without extra complexity.

Language Bridges in Anusaraka and with Real-Life Example

What is a Language Bridge in Anusaraka?

Anusaraka is a **Natural Language Processing (NLP) tool** designed to translate Indian languages into English while maintaining their original structure and meaning. **Language bridging** is a technique used in Anusaraka where an **intermediate representation (bridge language)** helps to translate between languages more effectively.

In simple terms, instead of **directly translating** from one language to another, **Anusaraka uses a bridge** (like Sanskrit-based grammar structures or an intermediate language) to ensure accurate and meaningful translation.

Why are Language Bridges Needed?

Different languages have **different grammar rules, sentence structures, and vocabulary**. If we translate word-for-word, the meaning might become **wrong or unclear**. A language bridge helps by:

✓ Breaking the sentence into **smaller understandable units**

✓ Mapping the meaning in a **neutral format**

✓ Converting it into the **target language smoothly**

Real-Life Example of Language Bridge in Anusaraka

Example 1: Hindi → Sanskrit (Bridge) → English

Hindi Sentence:

मैंने किताब पढ़ी। (Maine kitāb paḍhī.)

Direct translation (without bridge):

I book read. (Incorrect in English)

Using a Language Bridge:

1. Convert to Sanskrit-like structure:
"मया पुस्तकं पठितम्।"
2. Now translate to English:
"I read the book." (Correct translation)

What changed?

- The **bridge language (Sanskrit structure)** helped to clarify the meaning.
- The verb agreement was adjusted properly before final translation.

Example 2: Telugu → Sanskrit (Bridge) → English

Telugu Sentence:

నాకు ఇంగ్లీష్ తెలుసు. (Nāku īṅglīṣṭeḷusu.)

Direct translation (without bridge):

To me English knows. ✗ (Incorrect)

Using a Language Bridge:

1. Convert to Sanskrit-like structure:
"मम इंग्लिश ज्ञातम् अस्ति।"
2. Now translate to English:
"I know English." ✓ (Correct)

What changed?

- Telugu uses **dative case (to me)**, which does not exist in English.
- The bridge language helped map the correct **subject-verb structure**.

Simple understanding with 3 sentences below:

Language bridging in Anusaraka helps to:

- ✓ Make translations more accurate and meaningful
- ✓ Solve grammar mismatches between different languages
- ✓ Improve machine translation efficiency using NLP

MULTI LINGUAL INFORMATION RETRIEVAL:

What is Multilingual Information Retrieval (MLIR)?

- MLIR is about finding information in documents written in different languages.
- Example: Searching for "climate change" in English, French, and Spanish documents.

11.2 Document Preprocessing

- **Why Preprocess Documents?**

- To make documents easier to search and analyze by cleaning and organizing the text.

11.2.1 Document Syntax and Encoding

- **What is Syntax and Encoding?**

- Syntax: The structure of sentences and words in a document.
- Encoding: How text is stored in a computer (e.g., UTF-8 for multilingual text).
- Example: A document in French might use accents (é, è), which need proper encoding.

11.2.2 Tokenization

- **What is Tokenization?**

- Breaking text into smaller parts like words or sentences.
- Example: "I love ice cream." → ["I", "love", "ice", "cream"].

11.2.3 Normalization

- **What is Normalization?**

- Making text consistent by converting it to lowercase, removing punctuation, etc.
- Example: "Hello, World!" → "hello world".

11.2.4 Best Practices for Preprocessing

- **Tips for Preprocessing:**

- Use standard encoding (e.g., UTF-8).
- Handle special characters (e.g., accents in French).
- Normalize text to make searching easier.

11.3 Monolingual Information Retrieval

- **What is Monolingual Information Retrieval?**

- Searching for information in documents written in one language.

11.3.1 Document Representation

- **How to Represent Documents?**

- Use vectors or keywords to represent documents.
- Example: A document about "cats" might be represented as ["cat", "pet", "animal"].

11.3.2 Index Structures

- **What is an Index?**
 - A data structure that helps quickly find documents containing specific words.
 - Example: Like an index in a book, but for words in a database.

11.3.3 Retrieval Models

- **What are Retrieval Models?**
 - Algorithms that rank documents based on how well they match a query.
 - Example: Searching for "apple" might rank documents about the fruit higher than those about the company.

11.3.4 Query Expansion

- **What is Query Expansion?**
 - Adding related words to a search query to find more relevant documents.
 - Example: Searching for "car" might also include "automobile" and "vehicle".

11.3.5 Document A Priori Models

- **What are A Priori Models?**
 - Models that use prior knowledge about documents to improve search results.
 - Example: Knowing that "bank" often refers to a financial institution in business documents.

11.3.6 Best Practices for Model Selection

- **Tips for Choosing Models:**
 - Use models that match the type of documents (e.g., scientific papers vs. social media posts).
 - Test models with real queries to see which works best.

Real-Time Examples

1. Multilingual Search:

- A user searches for "global warming" and gets results in English, Spanish, and French.

2. Tokenization:

- Breaking "The quick brown fox" into ["The", "quick", "brown", "fox"].

3. Normalization:

- Converting "Café" to "cafe" for easier searching.

4. Query Expansion:

- Searching for "smartphone" also includes "mobile phone" and "cell phone".

5. Index Structures:

- A search engine uses an index to quickly find all documents containing the word "dog".

Monolingual information retrieval:

11.3 Monolingual Information Retrieval

What is Monolingual Information Retrieval?

- It is the process of finding information in documents written in one language.
- Example: Searching for "machine learning" in English documents only.

11.3.1 Document Representation

- **What is Document Representation?**
 - It is how a document is stored and organized so that it can be searched easily.
 - Documents are represented as **vectors** or **sets of keywords**.
- **Example:**
 - A document about "dogs" might be represented as ["dog", "pet", "loyal", "friendly"].
- **Common Methods:**
 1. **Bag of Words (BoW):**
 - Represents a document as a collection of words, ignoring grammar and word order.
 - Example: "The dog is loyal" → ["the", "dog", "is", "loyal"].
 2. **TF-IDF (Term Frequency-Inverse Document Frequency):**
 - Weights words based on how important they are in a document.
 - Example: The word "dog" might have a higher weight in a document about pets than in a document about cars.

11.3.2 Index Structures

- **What is an Index?**
 - An index is like a **map** that helps quickly find documents containing specific words.
 - Example: Like an index in a book, but for words in a database.
- **Types of Indexes:**
 1. **Inverted Index:**
 - Lists all documents that contain a specific word.
 - Example: The word "apple" might point to documents 1, 3, and 5.
 2. **Forward Index:**
 - Lists all words in a specific document.
 - Example: Document 1 contains ["apple", "fruit", "healthy"].

11.3.3 Retrieval Models

- **What are Retrieval Models?**
 - These are algorithms that rank documents based on how well they match a search query.
 - Example: Searching for "apple" might rank documents about the fruit higher than those about the company.
- **Common Models:**
 1. **Boolean Model:**
 - Uses **AND**, **OR**, **NOT** operators to find documents.
 - Example: Searching for "apple AND fruit" will return documents that contain both words.
 2. **Vector Space Model:**
 - Represents documents and queries as **vectors** and calculates their similarity.
 - Example: A document about "healthy fruits" might be similar to a query about "nutritious apples".
 3. **Probabilistic Model:**
 - Estimates the probability of a document being relevant to a query.
 - Example: A document with the word "apple" multiple times is more likely to be relevant to a query about "apple recipes".

11.3.4 Query Expansion

- **What is Query Expansion?**
 - It is the process of adding **related words** to a search query to improve results.
 - Example: Searching for "car" might also include "automobile" and "vehicle".
- **Methods:**
 1. **Thesaurus-Based Expansion:**
 - Uses a thesaurus to find synonyms.
 - Example: Expanding "car" to include "automobile" and "vehicle".
 2. **Relevance Feedback:**
 - Expands the query based on user feedback.
 - Example: If a user clicks on documents about "electric cars", the system might add "electric" to the query.

11.3.5 Document A Priori Models

- **What are A Priori Models?**
 - These models use **prior knowledge** about documents to improve search results.
 - Example: Knowing that "bank" often refers to a financial institution in business documents.
- **How It Works:**
 - The system uses information like document type, author, or topic to improve relevance.
 - Example: A document about "river banks" might be ranked lower for a query about "bank loans".

11.3.6 Best Practices for Model Selection

- **Tips for Choosing Models:**
 1. **Match the Model to the Document Type:**
 - Use simpler models (e.g., Boolean) for structured data and advanced models (e.g., Vector Space) for unstructured data.
 - Example: Use Boolean models for searching legal documents and Vector Space models for searching social media posts.
 2. **Test Models with Real Queries:**
 - Test how well the model performs with actual user queries.
 - Example: Test a model with queries like "best smartphones" and see if it returns relevant results.
 3. **Consider Scalability:**
 - Choose models that can handle large datasets efficiently.
 - Example: Use inverted indexes for large document collections.

Real-Time Examples

1. **Document Representation:**
 - A news article about "climate change" is represented as ["climate", "change", "global", "warming"].
2. **Index Structures:**
 - A search engine uses an inverted index to quickly find all documents containing the word "dog".
3. **Retrieval Models:**

- A user searches for "healthy recipes" and gets results ranked by relevance using the Vector Space Model.
- 4. **Query Expansion:**
 - A search for "car" is expanded to include "automobile" and "vehicle" to improve results.
- 5. **A Priori Models:**
 - A query about "bank loans" prioritizes documents from financial websites over those about rivers.

Summary for MONO LINGUAL INFORMATION:

- **Document Representation:** How documents are stored as keywords or vectors.
- **Index Structures:** Maps that help find documents quickly.
- **Retrieval Models:** Algorithms that rank documents based on relevance.
- **Query Expansion:** Adding related words to improve search results.
- **A Priori Models:** Using prior knowledge to improve relevance.
- **Best Practices:** Choose models based on document type, test with real queries, and ensure scalability.

CROSS LINGUAL INFORMATION RETRIEVAL:

11.4 Cross-Language Information Retrieval (CLIR)

What is CLIR?

- CLIR is the process of searching for information in one language and retrieving documents written in another language.
- Example: A user searches for "climate change" in English but gets results in French, Spanish, and German.

11.4.1 Challenges in CLIR

- **Language Barriers:**
 - Differences in grammar, vocabulary, and writing systems make CLIR challenging.
 - Example: The word "bank" in English can mean a financial institution or the side of a river, but in French, it's "banque" (financial) or "rive" (river).
- **Translation Accuracy:**
 - Machine translation may not always be accurate, leading to irrelevant results.
 - Example: Translating "hot dog" from English to French might result in "chien chaud" (hot dog) instead of the correct term "hot-dog".

11.4.2 Document Translation

- **What is Document Translation?**
 - Translating the entire document from the source language to the target language before indexing.
 - Example: Translating a French document about "réchauffement climatique" (climate change) into English for indexing.
- **Pros:**
 - Allows monolingual retrieval models to be used.
- **Cons:**
 - Expensive and time-consuming for large datasets.

11.4.3 Query Translation

- **What is Query Translation?**
 - Translating the user's query into the target language before searching.
 - Example: Translating "climate change" from English to French ("changement climatique") and searching French documents.
- **Pros:**
 - Faster and cheaper than document translation.
- **Cons:**
 - Translation errors can lead to irrelevant results.

11.4.4 Intermediate Language Representation

- **What is Intermediate Language Representation?**
 - Using an intermediate language (like Esperanto or a pivot language) to bridge the source and target languages.
 - Example: Translating an English query into Esperanto and then into French.
- **Pros:**
 - Reduces the number of translation pairs needed.
- **Cons:**
 - Adds complexity and potential translation errors.

11.4.5 Bilingual Dictionaries and Thesauri

- **What are Bilingual Dictionaries and Thesauri?**
 - Tools that map words and phrases from one language to another.
 - Example: A bilingual dictionary maps "dog" (English) to "perro" (Spanish).
- **Pros:**
 - Simple and effective for word-level translation.
- **Cons:**
 - May not handle context or idiomatic expressions well.

11.4.6 Machine Translation in CLIR

- **What is Machine Translation?**
 - Using algorithms to automatically translate text from one language to another.
 - Example: Google Translate can translate "climate change" from English to French as "changement climatique".
- **Pros:**
 - Fast and scalable for large datasets.
- **Cons:**
 - May produce inaccurate translations for complex sentences.

11.4.7 Best Practices for CLIR

- **1. Use High-Quality Translation Tools:**
 - Use reliable machine translation systems like Google Translate or DeepL.
 - Example: Translate "sustainable development" from English to Spanish as "desarrollo sostenible".
- **2. Combine Multiple Approaches:**

- Use both query translation and document translation for better results.
- Example: Translate the query and also translate key documents.
- **3. Handle Ambiguity:**
 - Use context to resolve ambiguous terms.
 - Example: Determine if "bank" refers to a financial institution or a river based on the query context.
- **4. Test with Real Queries:**
 - Test the CLIR system with real user queries to ensure accuracy.
 - Example: Test with queries like "global warming" and check if relevant results are retrieved in multiple languages.

Real-Time Examples

1. Query Translation:

- A user searches for "global warming" in English, and the system translates it to "calentamiento global" in Spanish to retrieve relevant documents.

2. Document Translation:

- A French document about "réchauffement climatique" is translated into English as "climate change" and indexed for retrieval.

3. Bilingual Dictionaries:

- A bilingual dictionary maps "apple" (English) to "manzana" (Spanish) for query translation.

4. Machine Translation:

- A user searches for "sustainable energy" in English, and the system translates it to "energía sostenible" in Spanish using Google Translate.

5. Intermediate Language:

- An English query is translated into Esperanto and then into French to retrieve relevant documents.

Summary for CLIR:

- **CLIR:** Searching in one language and retrieving documents in another.
- **Challenges:** Language barriers and translation accuracy.
- **Document Translation:** Translating entire documents before indexing.
- **Query Translation:** Translating the user's query before searching.
- **Intermediate Language:** Using a pivot language to bridge source and target languages.
- **Bilingual Dictionaries:** Tools for word-level translation.
- **Machine Translation:** Algorithms for automatic translation.
- **Best Practices:** Use high-quality tools, combine approaches, handle ambiguity, and test with real queries.

Aspect	Monolingual Information Retrieval	Multilingual Information Retrieval	Cross-Lingual Information Retrieval
Definition	Retrieves information in one language.	Retrieves information in multiple languages.	Searches in one language and retrieves in another language.
Example	Searching for "machine learning" in	Searching for "climate change" in	Searching for "climate change" in

	English documents.	English, French, and Spanish documents.	English but retrieving French and Spanish documents.
Language Handling	Works with a single language.	Works with multiple languages simultaneously.	Bridges two or more languages (source and target).
Challenges	Limited to one language; no translation required.	Handling differences in grammar, vocabulary, and encoding across languages.	Accurate translation of queries or documents between languages.
Document Preprocessing	Tokenization, normalization, and indexing in one language.	Preprocessing for each language (e.g., tokenization in French, Spanish, etc.).	Translation of documents or queries before preprocessing.
Query Processing	Query is in the same language as the documents.	Query can be in any of the supported languages.	Query is translated from the source language to the target language.
Translation Required	No translation needed.	No translation needed; documents are in multiple languages.	Translation of queries, documents, or both is required.
Indexing	Single-language index (e.g., English).	Multiple indexes for each language (e.g., English, French, Spanish).	Indexes are created after translating documents or queries.
Retrieval Models	Standard models like Boolean, Vector Space, or Probabilistic.	Models adapted for multilingual datasets.	Models that handle translated queries or documents.
Real-Time Example	Searching for "apple" in English documents.	Searching for "apple" and retrieving results in English, French, and Spanish.	Searching for "apple" in English but retrieving French ("pomme") and Spanish ("manzana") documents.
Use Cases	Searching within a single-language database (e.g., English Wikipedia).	Searching across multilingual databases (e.g., EU documents).	Searching for information in a foreign language (e.g., English user searching Chinese documents).
Tools/Technologies	Monolingual search engines (e.g., Google for English).	Multilingual search engines (e.g., Google Translate).	Translation tools (e.g., Google Translate, bilingual dictionaries).

11.6 Evaluation in Information Retrieval

Evaluation in Information Retrieval (IR) is the process of checking how well a search system works.

It helps us understand if the system is finding the right information for users. Let's break it down into simple steps with real-time examples.

11.6.1 Experimental Setup

- **What is Experimental Setup?**

- It is the process of designing tests to evaluate how well a search system performs.
- Example: Testing a search engine by giving it 100 queries and checking if it retrieves the correct documents.

11.6.2 Relevance Assessments

- **What is Relevance Assessment?**

- It is the process of deciding if a document is relevant to a query.
- Example: For the query "healthy recipes," a document about "10 healthy smoothies" is relevant, but a document about "car maintenance" is not.

11.6.3 Evaluation Measures

- **What are Evaluation Measures?**

- These are tools to measure how good a search system is. Common measures include:

1. **Precision:**

- The percentage of retrieved documents that are relevant.
- Example: If 8 out of 10 retrieved documents are relevant, precision is 80%.

2. **Recall:**

- The percentage of relevant documents that are retrieved.
- Example: If there are 20 relevant documents and the system retrieves 15, recall is 75%.

$$F1\text{-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

In your example, with precision = 80% and recall = 75%, the F1-Score is **77.5%**.

3. **F1-Score:**

- A balance between precision and recall.
- Example: If precision is 80% and recall is 75%, the F1-score is 77.5%.

11.6.4 Established Data Sets

- **What are Established Data Sets?**

- These are standard collections of documents and queries used to test search systems.
- Example: The **TREC (Text REtrieval Conference)** dataset is widely used for testing IR systems.

11.6.5 Best Practices

- **Tips for Evaluation:**

1. **Use Standard Datasets:**

- Use **datasets like TREC or CLEF for fair comparisons.**
- 2. **Test with Real Queries:**
 - Use **real user queries** to see how the system performs in real-world scenarios.
- 3. **Measure Both Precision and Recall**
 - A good system should have high precision (few irrelevant results) and high recall (most relevant results retrieved).

Real-Time Examples

1. **Experimental Setup:**
 - A **company tests its search engine by giving it 50 queries and checking if it retrieves the correct documents.**
2. **Relevance Assessment:**
 - For the query "best smartphones," a document about "Top 10 smartphones in 2023" is relevant, but a document about "laptop reviews" is not.
3. **Precision and Recall:**
 - A search engine retrieves 10 documents for the query "climate change." If 8 are relevant, precision is 80%. If there are 20 relevant documents in total, recall is 40%.
4. **Established Data Sets:**
 - Researchers use the TREC dataset to compare their search system with others.
5. **Best Practices:**
 - A search engine is tested with real user queries like "how to bake a cake" to ensure it works well in real-world scenarios.

Summary with technical words:

- **Experimental Setup:** Designing tests to evaluate a search system.
- **Relevance Assessments:** Deciding if a document is relevant to a query.
- **Evaluation Measures:** Tools like precision, recall, and F1-score to measure performance.
- **Established Data Sets:** Standard datasets like TREC for testing.
- **Best Practices:** Use standard datasets, test with real queries, and measure both precision and recall.

Tools, Software, and Resources for Information Retrieval

1. Search Engines

- **What are Search Engines?**
 - Tools that help users find information on the web or in a database.
 - Example: **Google, Bing, and Elasticsearch.**
- **Real-Time Example:**
 - A user searches for "best laptops 2023" on Google and gets a list of relevant websites.

2. Libraries and Frameworks

- **What are Libraries and Frameworks?**
 - **Pre-built code** and tools that help developers build IR systems.

- Example: Apache Lucene, NLTK, and spaCy.
- **Real-Time Example:**
 - A developer uses Apache Lucene to build a search engine for a company's internal document database.

3. Machine Learning Tools

- **What are Machine Learning Tools?**
 - Tools that use algorithms to improve search results.
 - Example: TensorFlow, Scikit-learn, and Gensim.
- **Real-Time Example:**
 - A company uses Scikit-learn to train a machine learning model that ranks search results based on user clicks.

4. Datasets

- **What are Datasets?**
 - Collections of documents and queries used to test and train IR systems.
 - Example: TREC, CLEF, and Reuters-21578.
- **Real-Time Example:**
 - Researchers use the TREC dataset to test the performance of their search algorithms.

5. Evaluation Tools

- **What are Evaluation Tools?**
 - Tools that measure how well an IR system performs.
 - Example: trec_eval, RankLib, and Elasticsearch Ranking Evaluation.
- **Real-Time Example:**
 - A developer uses trec_eval to measure the precision and recall of their search engine.

6. Natural Language Processing (NLP) Tools

- **What are NLP Tools?**
 - Tools that help process and understand human language.
 - Example: Stanford NLP, spaCy, and NLTK.
- **Real-Time Example:**
 - A company uses spaCy to tokenize and normalize text before indexing it in their search engine.

7. Cloud-Based IR Services

- **What are Cloud-Based IR Services?**
 - Services that provide IR capabilities over the cloud.
 - Example: Google Cloud Search, Amazon Kendra, and Azure Cognitive Search.
- **Real-Time Example:**
 - A business uses Amazon Kendra to build a smart search system for their customer support portal.

8. Open-Source IR Systems

- **What are Open-Source IR Systems?**

- Free and customizable IR systems that can be modified as needed.
- Example: Apache Solr, Elasticsearch, and Terrier.

- **Real-Time Example:**

- A university uses **Apache Solr** to build a searchable database of research papers.

9. Visualization Tools

- **What are Visualization Tools?**

- Tools that help visualize search results and system performance.
- Example: Kibana, Tableau, and Matplotlib.

- **Real-Time Example:**

- A data scientist uses **Kibana** to visualize search trends and user behavior on a website.

10. APIs for IR

- **What are APIs for IR?**

- **Application Programming Interfaces** that allow developers to integrate IR capabilities into their applications.
- Example: Google Custom Search API, Bing Search API, and Elasticsearch API.

- **Real-Time Example:**

- A developer uses the **Google Custom Search API** to add a search feature to their website.

Real-Time Examples of Tools and Resources in Action

1. Search Engine:

- A user searches for "healthy recipes" on **Google** and gets a list of relevant websites.

2. Library/Framework:

- A developer uses **Apache Lucene** to build a search engine for a company's internal document database.

3. Machine Learning Tool:

- A company uses **Scikit-learn** to train a model that ranks search results based on user clicks.

4. Dataset:

- Researchers use the **TREC dataset** to test the performance of their search algorithms.

5. Evaluation Tool:

- A developer uses **trec_eval** to measure the precision and recall of their search engine.

6. NLP Tool:

- A company uses **spaCy** to tokenize and normalize text before indexing it in their search engine.

7. Cloud-Based IR Service:

- A business uses **Amazon Kendra** to build a smart search system for their customer support portal.

8. Open-Source IR System:

- A university uses **Apache Solr** to build a searchable database of research papers.

9. Visualization Tool:

- A data scientist uses **Kibana** to visualize search trends and user behavior on a website.

10. API for IR:

- A developer uses the **Google Custom Search API** to add a search feature to their website.

Detailed Summary with technical words:

- **Search Engines:** Tools like Google and Elasticsearch for finding information.
- **Libraries/Frameworks:** Pre-built tools like Apache Lucene and spaCy for building IR systems.
- **Machine Learning Tools:** Tools like TensorFlow and Scikit-learn for improving search results.
- **Datasets:** Collections like TREC and CLEF for testing and training.
- **Evaluation Tools:** Tools like trec_eval for measuring system performance.
- **NLP Tools:** Tools like spaCy and NLTK for processing human language.
- **Cloud-Based IR Services:** Services like Amazon Kendra for cloud-based search.
- **Open-Source IR Systems:** Free systems like Apache Solr and Elasticsearch.
- **Visualization Tools:** Tools like Kibana for visualizing search data.
- **APIs for IR:** APIs like Google Custom Search API for integrating search features.

Multilingual Automatic Summarization in action. Imagine you are a journalist working for an international news agency, and you need to quickly summarize a long article written in **Spanish** about a global event, like a climate change conference. This is the simple example how multilingual automatic summarization would help:

Real-Time Example: Summarizing a Spanish News Article

Step 1: Input

- **Original Text:** You have a long article in Spanish about the **climate change** conference. The article discusses key points like global warming, carbon emissions, and agreements made by countries.
- **Language:** Spanish.

Step 2: Processing

1. Understanding the Content:

- The system reads the **Spanish text and understands the main topics**, such as:
 - Global warming.
 - Carbon emissions.
 - Agreements between countries.
- It identifies the most important sentences or phrases that talk about these topics.

2. Picking Key Points:

- The system extracts sentences like:
 - "Los países acordaron reducir las emisiones de carbono en un 50% para 2030." (Countries agreed to reduce carbon emissions by 50% by 2030.)
 - "El calentamiento global está aumentando a un ritmo alarmante." (**Global warming** is increasing at an **alarming rate**.)
- These sentences are considered **important because they contain key facts and figures**.

Step 3: Output

• Summary in Spanish:

- The system generates a short summary in Spanish:
 - "Los países acordaron reducir las emisiones de carbono en un 50% para 2030 debido al aumento alarmante del calentamiento global." (Countries agreed to reduce carbon emissions by 50% by 2030 due to the alarming increase in global warming.)

• Summary in English (if needed):

- If you need the summary in English, the system can translate it:
 - "Countries agreed to reduce carbon emissions by 50% by 2030 due to the alarming increase in global warming."

Why is This Useful?

1. Saves Time:

- Instead of reading the entire article in Spanish, you get a quick summary in just a few seconds.
- If you need the summary in English, you don't have to manually translate the whole article.

2. Cross-Language Understanding:

- If your team includes people who don't speak Spanish, they can still understand the key points of the article through the English summary.
- This helps in collaborative work across different language speakers.

3. **Wide Applications:**

- **News Agencies:** Quickly summarize international news articles.
- **Business:** Summarize reports or documents written in different languages.
- **Research:** Summarize academic papers or studies from around the world.

Challenges in This Example

1. **Language Differences:**

- Spanish and English have different grammar rules. For example, adjectives come after nouns in Spanish ("calentamiento global" = "global warming"), but the system needs to adjust this when translating to English.
- The system must be smart enough to handle these differences without losing the meaning.

2. **Accuracy:**

- The summary must capture the true meaning of the original text. For example, if the Spanish article mentions "un aumento alarmante" (an alarming increase), the system must ensure that "alarming" is correctly translated and not replaced with a less impactful word like "significant."

How This Helps in Real Life

Imagine you're on a tight deadline, and you need to report on the climate change conference. With multilingual automatic summarization:

- You can quickly get the key points from the Spanish article.
- You can share the summary with your team in English, even if they don't speak Spanish.
- You save hours of reading and translating, allowing you to focus on writing your news report.

12.2.1 The Classics

This is about the **old-school methods** of summarizing text. Think of it like picking out the most important sentences from a book or article. Here's how it works:

- **Extraction:** The system picks sentences that seem important, often based on keywords or how often certain words appear.
- **Rules-based:** It follows simple rules, like "if a sentence has the word 'important,' include it in the summary."
- **Limitations:** While easy to use, these methods might miss the deeper meaning or context of the text. (here we miss the important meaning exactly leading to the concept)

12.2.2 Graph-Based Approaches

This method is a bit fancier. It treats the text like a **network or web of ideas**. Here's the idea:

- **Sentences as nodes:** Each sentence is like a dot in a network.
- **Connections as links:** If two sentences talk about the same idea, they're connected by a line.

- **Finding the center:** The system looks for sentences that are connected to many others (like the center of a spiderweb). These are considered the most important and are used in the summary.
- **Better than classics:** This method can handle more complex texts and relationships between sentences.

12.2.3 Learning How to Summarize

This is where **machine learning** comes in. The system learns how to summarize by looking at lots of examples. Here's how:

- **Training:** The system is given many examples of texts and their summaries. It learns patterns, like which sentences are usually included in summaries.
- **Prediction:** After training, the system can read new texts and predict what the summary should look like.
- **More human-like:** This method can create summaries that feel more natural, like something a person would write.

12.2.4 Multilingual Summarization

This is about summarizing text in **many languages**. Here's the breakdown:

- **Language flexibility:** The system can work with text in different languages, like English, Spanish, or Chinese.
- **Translation + summarization:** Sometimes, the system translates the text into one language, summarizes it, and then translates the summary back into the original language.
- **Challenges:** Each language has its own rules and quirks, so summarizing across languages can be tricky.

Approaches to summarization:

Multilingual summarization means **creating summaries for text written in many languages** — for example, Telugu, Hindi, or English. Sometimes the summary is in the **same language** (Telugu → Telugu), or in a **different language** (Telugu → English).

There are **three main approaches** used for this.

1. Extractive Summarization

Pick the **most important sentences** directly from the text.

It does **not change or rewrite** the text — it just extracts key parts.

How it works:

1. Break the text into sentences.

2. Give each sentence a score (based on importance).
3. Select the top few sentences for the summary.

Example (Telugu → English):

Telugu Text:

ఆంధ్రప్రదేశ్ వ్యవసాయంలో పెద్ద మార్పులు చేసుకుంటున్నాయి. ప్రభుత్వం రైతులకు కొత్త పథకాలు అందిస్తోంది. పంట ఉత్పత్తి పెరుగుతోంది.

Extractive Summary (English):

The government is introducing new schemes for farmers. Crop production is increasing.

Pros: Simple and fast.

Cons: Doesn't sound natural or fluent sometimes.

Models:

- **mBERT** (Multilingual BERT)
- **XLNet**

2. Abstractive Summarization

The system **understands the text meaning** and then **writes the summary in its own words**, like a human does.

How it works:

1. Use a multilingual sequence-to-sequence model (like mBART or mT5).
2. The model learns to rewrite the important content.
3. Can summarize in the **same** or **different** language.

Example (Telugu → English):

Telugu Text:

చంద్రయాన్-3 విజయంతో భారత అంతరిక్ష సంస్థకు గర్వకారణం లభించింది. ప్రపంచం భారత్ ను ప్రశంసించింది.

Abstractive Summary (English):

India's space agency gained global praise for the Chandrayaan-3 success.

Pros: Natural, human-like summaries.

Cons: Needs more data and GPU power to train.

Models:

- **mBART (Multilingual BART)**
- **mT5 (Multilingual T5)**
- **IndicBERT** (for Indian languages)

3. Cross-Lingual Summarization

Idea:

Input and output languages are **different**.

For example, you give **Telugu text**, and get an **English summary**.

How it works:

1. **Method 1 – Translate + Summarize**
 - Step 1: Translate Telugu → English
 - Step 2: Summarize English text
2. **Method 2 – Direct Summarization**
 - A single model does both together.

Example:

Telugu Input:

ఆర్థిక సంక్షోభం కారణంగా పలు పరిశ్రమలు మూతపడ్డాయి.
ప్రభుత్వం ఉపశమన చర్యలు తీసుకుంటోంది.

English Summary:

Several industries closed due to the economic crisis, but the government is taking relief measures.

Pros: Useful for multilingual news or research.

Cons: Translation errors may affect summary quality.

Models:

- **mBART-50**
- **mT5**
- **NLLB (No Language Left Behind)**

4. (Optional) Hybrid Approach

Idea:

Combine **Extractive** + **Abstractive** methods.

It first extracts important lines, then rewrites them nicely.

Example:

Telugu Text:

విద్యుత్కేరతసమస్యను పరిష్కరించేందుకు ప్రభుత్వం కొత్త ప్రణాళికలు రూపొందిస్తోంది. సారశక్తి ప్రాజెక్టులు ప్రారంభించబడ్డాయి.

English Summary:

The government is addressing power shortages by launching new solar projects.

Pros: Balanced and accurate.

Cons: More complex system.

EVALUATION:

12.3.1 Manual Evaluation Methodologies

This is about **humans checking summaries** to see if they are good. Here's how it works:

Real-Time Example:

Imagine **you are a teacher**, and your students have written summaries of a long article about climate change. **You need to grade their summaries**. Here's what you do:

1. **Read the original article:** You read the full article to understand all the key points.
2. **Compare the summaries:** You check each student's summary to see if it includes the main ideas, like:
 - The causes of climate change.
 - The effects of global warming.
 - The solutions proposed by scientists.
3. **Give feedback:** You tell the students what they did well and what they missed.

Why is this useful?

- **Accuracy:** Humans can judge if the summary captures the true meaning of the original text.
- **Flexibility:** Humans can understand context and nuances that machines might miss.

Challenges:

- **Time-consuming:** It takes a lot of time to read and evaluate summaries manually.
- **Subjectivity:** Different people might have different opinions on what makes a good summary.

12.3.2 Automated Evaluation Methods

This is about using **computers to check summaries** automatically. Here's how it works:

Real-Time Example:

Imagine you are a software developer building a tool to summarize news articles. You want to test if your tool is working well. Here's what you do:

1. **Use pre-written summaries:** You **have a set of articles** and their ideal summaries (written by humans).
2. **Compare machine summaries:** Your **tool generates summaries** for the same articles.

3. **Check for matches:** The system compares the machine-generated summaries with the ideal ones to see how similar they are.

Why is this useful?

- **Speed:** Computers can evaluate summaries in seconds.
- **Consistency:** The system **uses the same rules to evaluate every summary**, so there's no subjectivity.

Challenges:

- **Limited understanding:** Computers might not fully understand the meaning of the text, so they might miss subtle errors.
- **Dependence on data:** The system needs high-quality examples to compare against.

12.3.3 Recent Developments in Evaluating Summarization Systems

This section talks about **new and improved ways** to evaluate summaries. Here's what's happening:

Real-Time Example:

Imagine you are a **researcher working on a new summarization tool. You want to test it using the latest methods.** Here's what you do:

1. **Use advanced metrics:** Instead of **just checking if the summary matches the original text**, you use metrics that measure:
 - **Coherence:** Does the summary make sense?
 - **Relevance:** Does it include the most important points?
 - **Diversity:** Does it cover different aspects of the topic?
2. **Test on multiple languages:** You evaluate how well your tool works in different languages, like English, Spanish, and Chinese.

Why is this useful?

- **Better evaluation:** These new methods give a more complete picture of how good a summary is.
- **Multilingual support:** You can test your tool on texts in many languages.

Challenges:

- **Complexity:** These methods are more advanced and require more computing power.
- **Data needs:** You need a lot of data to train and test these systems.

12.3.4 Automatic Metrics for Multilingual Summarization

This is about using **automatic tools** to evaluate summaries in **multiple languages**. Here's how it works:

Real-Time Example:

Imagine you are working for a global news agency, and your summarization tool needs to work in English, French, and Arabic. Here's how you evaluate it:

1. **Use multilingual metrics:** **The system checks** summaries in all three languages using metrics like:

- **BLEU**: Measures how similar the summary is to a reference summary.
 - **ROUGE**: Checks if the summary includes the same key phrases as the original text.
2. **Compare across languages**: The system evaluates how well the tool performs in each language and identifies areas for improvement.

Why is this useful?

- **Efficiency**: You can evaluate summaries in multiple languages quickly.
- **Consistency**: The same metrics are used for all languages, so the evaluation is fair.

Challenges:

- **Language differences**: Each language has its own grammar and style, so the metrics need to be adjusted for each one.
- **Accuracy**: The system might struggle with languages that have very different structures, like English and Arabic.

Summary in simple terms:

- **Manual Evaluation**: Humans check summaries for accuracy and relevance (e.g., a teacher grading student summaries).
- **Automated Evaluation**: Computers compare machine-generated summaries with ideal ones (e.g., a developer testing a summarization tool).
- **Recent Developments**: New methods evaluate summaries for coherence, relevance, and diversity (e.g., a researcher testing a multilingual summarization tool).
- **Automatic Metrics for Multilingual Summarization**: Tools like BLEU and ROUGE are used to evaluate summaries in multiple languages (e.g., a global news agency testing summaries in English, French, and Arabic).

How to build a summarizer, which is like creating a tool that can automatically summarize text. Here's the explanation:

12.4.1 Ingredients

This is about the **basic components** you need to build a summarizer. Think of it like the ingredients you need to bake a cake. Here's what you need:

Real-Time Example:

Imagine you are building a summarizer for news articles. Here are the "ingredients" you need:

1. **Text Data**: You need lots of news articles to train your summarizer. For example:
 - Articles about sports, politics, and technology.
 - Each article should have a summary written by humans (this is called "labeled data").
2. **Algorithms**: These are the "recipes" that tell the summarizer how to pick out important sentences. For example:
 - **Extraction algorithms**: Pick sentences directly from the article.
 - **Abstraction algorithms**: Rewrite sentences in a shorter way.
3. **Computing Power**: You need a strong computer or cloud service to process all the data and run the algorithms.

Why is this useful?

- **Foundation:** Without these ingredients, you can't build a summarizer.
- **Flexibility:** You can use different types of data and algorithms depending on what you want to summarize (e.g., news, research papers, or social media posts).

Challenges:

- **Data quality:** If the articles or summaries are poorly written, the summarizer won't work well.
- **Computing cost:** Running complex algorithms can be expensive.

12.4.2 Devices

This is about the **tools and platforms** you use to build and run your summarizer. Think of it like the kitchen appliances you need to bake a cake.

Real-Time Example:

Imagine you are building a summarizer for a mobile app that summarizes long emails. Here's what you need:

1. **Programming Languages:** You write the code for the summarizer using languages like:
 - **Python:** A popular language for building AI tools.
 - **Java:** Used for mobile app development.
2. **Libraries and Frameworks:** These are pre-built tools that make coding easier. For example:
 - **NLTK or spaCy:** Libraries for processing text.
 - **TensorFlow or PyTorch:** Frameworks for building machine learning models.
3. **Cloud Services:** You use platforms like **Google Cloud or AWS** to store data and run your summarizer.

Why is this useful?

- **Efficiency:** These tools help you build the summarizer faster.
- **Scalability:** You can handle large amounts of data and users.

Challenges:

- **Learning curve:** You need to learn how to use these tools.
- **Cost:** Cloud services can be expensive if you have a lot of users.

HOW TO BUILD THE SUMMARIZER?

What is a Summarizer?

A **summarizer** is an NLP (Natural Language Processing) system that can **read a long text** and produce a **short, meaningful summary** — just like how humans read and write short notes.

Step 1: Collect Data

You need a dataset that has:

- Full text
- Corresponding short summary

Example datasets:

- **CNN/DailyMail** (English news)
- **WikiLingua** (Multilingual, includes Telugu)
- **MLSUM** (Multilingual)

Step 2: Preprocess the Data

Clean the text so the model can understand it.

Remove:

- HTML tags
- Special characters
- URLs
- Extra spaces

Tokenize:

Split text into **words or subwords** (depends on the model).

Step 3: Choose Your Model

Pick model type based on language and task.

Model Type	Example Models	Use Case
Extractive	TextRank, BERTSum	Fast and simple
Abstractive	T5, BART, mT5, mBART	Human-like summary
Multilingual	mBART-50, mT5	Telugu, Hindi, English etc.

```
from transformers import MBartForConditionalGeneration,  
MBart50TokenizerFast  
  
tokenizer =  
MBart50TokenizerFast.from_pretrained("facebook/mbart-large-  
50-many-to-many-mmt")  
model
```

```
MBartForConditionalGeneration.from_pretrained("facebook/mbar  
t-large-50-many-to-many-mmt")
```

```
text =  
"ఆంధ్రప్రదేశ్ శాసనసభలో ఆంధ్రప్రదేశ్ ప్రభుత్వం రైతులకు పలు సహాయక పథకాలు ప్రారం  
భించింది."
```

```
tokenizer.src_lang = "te_IN"  
encoded = tokenizer(text, return_tensors="pt")
```

```
summary_ids = model.generate(**encoded,  
forced_bos_token_id=tokenizer.lang_code_to_id["en_XX"],  
max_length=40)
```

```
summary = tokenizer.decode(summary_ids[0],  
skip_special_tokens=True)  
print(summary)
```

Output → "The Andhra Pradesh government has launched new schemes for farmers."

Step 6: Evaluate

Use metrics to check how good the summaries are.

Common metrics:

- **ROUGE** (compares overlap between model and human summary)
- **BLEU** (checks similarity)
- **METEOR** (word-level meaning)

Field	Use Case
News Websites	Summarize long articles automatically
Chatbots	Summarize long conversations

Field	Use Case
Business Reports	Create executive summaries
Education	Summarize research papers or notes
Multilingual Systems	Telugu, Hindi, English summarization and translation
Legal / Medical Docs	Extract key points from long documents

12.4.3 Instructions

This is about the **steps you follow to build the summarizer**. Think of it like following a recipe to bake a cake.

Real-Time Example:

Imagine you are building a summarizer for a website that summarizes long blog posts. Here's how you do it:

- Step 1: Collect Data:**
 - Gather lots of blog posts and their summaries.
 - Example: Collect 10,000 blog posts about travel, food, and fashion, each with a short summary.
- Step 2: Preprocess the Data:**
 - Clean the text by removing unnecessary **parts like ads or images**.
 - Example: Remove hashtags and emojis from social media posts.
- Step 3: Train the Model:**
 - Use the data to teach the summarizer how to pick important sentences.
 - Example: Train the model to recognize that sentences with words like "best," "top," or "must-see" are important in travel blogs.
- Step 4: Test the Summarizer:**
 - Give the summarizer new blog posts it hasn't seen before and check if the summaries are good.
 - Example: Test it on a blog post about "Top 10 Travel Destinations in 2024" and see if the summary includes the key points.
- Step 5: Deploy the Summarizer:**
 - Put the summarizer on your website so users can start using it.
 - Example: Add a "Summarize" button to your blog platform.

Why is this useful?

- Step-by-step guidance:** These instructions help you build the summarizer in a structured way.
- Iterative improvement:** You can keep testing and improving the summarizer over time.

Challenges:

- Time-consuming:** Building a good summarizer takes time and effort.

- **Debugging:** If the summarizer doesn't work well, you need to figure out what went wrong and fix it.

Summary in simple terms:

- **Ingredients** You need text data, algorithms, and computing power to build a summarizer (e.g., news articles and extraction algorithms).
- **Devices** You use programming languages, libraries, and cloud services to create and run the summarizer (e.g., Python and Google Cloud).
- **Instructions** You follow steps like collecting data, training the model, testing, and deploying the summarizer (e.g., building a blog summarizer for a website).

12.5 competitions and datasets related to summarization, which are like challenges and resources that help improve summarization tools. Here's the explanation:

12.5.1 Competitions

This is about **challenges or contests** where people or teams compete to build the best summarization tools. Think of it like a cooking competition where chefs try to make the best dish.

Real-Time Example:

Imagine there's a competition called "**Summify Challenge**" where participants build summarization tools. Here's how it works:

1. **Goal** Create a tool that can summarize news articles in multiple languages.
2. **Data Provided** The organizers give participants a large dataset of news articles and their summaries in English, Spanish, and French.
3. **Evaluation** The tools are tested on new articles, and the summaries are judged based on:
 - **Accuracy:** Does the summary capture the main points?
 - **Readability:** Is the summary easy to understand?
 - **Speed:** How quickly can the tool generate summaries?
4. **Prize** The team with the best tool wins a prize, like money or recognition in the AI community.

Why is this useful?

- **Innovation** Competitions encourage people to come up with new and creative ways to summarize text.
- **Benchmarking** They help set standards for what a good summarization tool should be able to do.
- **Collaboration** Participants often share ideas and learn from each other.

Challenges:

- **Pressure:** Competitions can be stressful because everyone wants to win.
- **Resource-intensive:** Building a competitive tool can require a lot of time and computing power.

12.5.2 Data Sets

This is about **collections of data used to train and test summarization tools**. Think of it like the ingredients you need to practice cooking.

Real-Time Example: it is all about TRAIN AND TEST

Imagine you are building a summarizer for social media posts. Here's how you use datasets:

1. **Dataset Example:** You use a dataset called "**SocialMediaPosts-2023**" that contains:
 - 100,000 social media posts from platforms like Twitter and Instagram.
 - Each post has a short summary written by humans.
2. **Training:** You use 80% of the dataset to train your summarizer. For example:
 - The tool learns that posts with hashtags like #BreakingNews or #Trending are often important.
3. **Testing:** You use the remaining 20% of the dataset to test your summarizer. For example:
 - You give the tool a new post like, "Just saw the most amazing sunset at the beach! 🌅 #NatureLovers," and check if the summary captures the main idea.

Why is this useful?

- **Training:** Datasets help teach the summarizer what good summaries look like.
- **Testing:** They allow you to check if the summarizer works well on new, unseen data.
- **Standardization:** Using the same datasets allows researchers to compare different summarization tools.

Challenges:

- **Data Quality:** If the dataset has errors or biases, the summarizer might not work well.
- **Size:** Large datasets are better for training, but they can be expensive to create and store.

Summary in simple terms:

- **Competitions:** These are challenges where people build and test summarization tools (e.g., the "Summify Challenge" for summarizing news articles).
- **Datasets:** These are collections of text and summaries used to train and test summarizers (e.g., the "SocialMediaPosts-2023" dataset for summarizing social media posts).

Defining semantics structure model theory:

Introduction to Semantics

In computer science and logic, **semantics** refers to the *meaning* of symbols, statements, or programs — not just their syntax (structure). It explains **what a statement does or represents**, rather than how it looks.

Aspect	Syntax	Semantics
Meaning	Structure / Grammar	Meaning / Interpretation
Example (C code)	<code>x = x + 1;</code> is syntactically correct	Semantically, it increases x by 1
Checked by	Parser / Grammar	Semantic analyzer / Logic

Syntax: defines what strings are valid.

Semantics: defines what those strings mean

Types of Semantics in Computer Science

There are three main kinds of semantics used to describe the meaning of programs:

1. Operational Semantics:

- Describes how the program *executes step by step*.
- Defines meaning in terms of *state transitions*.
- Example: “If $x = 3$, and the statement is $x = x + 2$, the new state becomes $x = 5$.”

2. Denotational Semantics:

- Describes each program construct by a *mathematical function*.
- Maps syntactic elements to abstract mathematical objects.
- Example: A statement $x = x + 1$ maps to a function that adds 1 to the value of x .

3. Axiomatic Semantics:

- Uses *logical assertions* (preconditions and postconditions).
- Based on **Hoare logic**, where $\{P\} S \{Q\}$ means:
“If precondition P is true before statement S , then postcondition Q will be true after.”

Structural Model of Semantics

The **structural semantics** (or **structural operational semantics**) describes meaning by **breaking programs into smaller structures** and assigning meaning to each.

Each construct in a programming language (like *if*, *while*, *assignment*, *expression*) has rules that define **how its meaning is built from its parts**.

Example Rule (for a `while` loop):

while (condition) do statement

Semantics:

1. Evaluate the condition.
2. If true $\rightarrow$ execute the statement and repeat.
3. If false $\rightarrow$ terminate the loop.

Model Theory (in Logic and Semantics)

Model Theory is a branch of mathematical logic that connects **formal languages** with their **interpretations (models)**.

It helps in defining **what is true** under a given interpretation.

Concept	Explanation
Formal Language (L)	A set of well-formed formulas defined by syntax rules.
Structure / Model (M)	Gives meaning to symbols — assigns objects, values, or truth.
Interpretation	A mapping from symbols $\rightarrow$ actual entities in the model.
Satisfaction	A formula is <i>satisfied</i> in a model if it is true under that interpretation.

Example:

Let's define a simple logic:

- **Language L has symbols:** $\{x, >, 0\}$
- **Sentence:** $x > 0$

Now define a model M :

- $x = 5$
- **Interpretation of “ $>$ ” = greater than relation.**

✓ $x > 0$ is true **in this model.**

✗ **But if $x = -2$, then the sentence is false in that model.**

So, semantics (meaning) **depends on the model** (interpretation).

Applications in Engineering and Computer Science

1. Compiler Design:

- Semantic analysis ensures variables, types, and scopes are valid.
- Denotational semantics helps define intermediate representations.

2. Formal Verification:

- Used to *prove correctness* of software and hardware systems.

3. Programming Language Design:

- Ensures new languages have *clear meaning* for every construct.

What is the application of Machine Translation?

Machine translation has many applications, including:

1. **Cross-border communication:** Machine translation allows people from different countries to communicate with each other more easily, breaking down language barriers and facilitating international cooperation.
2. **Localization:** Machine translation can be used to quickly and efficiently translate websites, software, and other digital content into different languages, making them more accessible to users around the world.
3. **Business:** Machine translation can be used by businesses to translate documents, contracts, and other important materials, enabling them to work with partners and customers from around the world.
4. **Education:** Machine translation can be used in education to help students learn new languages and improve their language skills.

5. Government: Machine translation can be used by governments to translate official documents and communications, improving accessibility and transparency.

Can Human Translators be replaced by AI?

Challenges arise as different languages categorize and express concepts in diverse ways, demanding careful consideration in translation. The use of an inter lingua, a representation language capturing distinctions among languages, becomes essential. Whether undertaken by humans or machines, effective translation involves grasping the entirety of the scenario presented in the source, not just individual words. Complexities further intensify when dealing with cultural and grammatical nuances, underscoring the intricacies of the translation endeavor.

It can be challenging for translators (both machine and human) to make this decision. “The baseball struck the window,” for example, maybe translated as “The baseball hit the window.” In order to translate “it broke” into French, we must pick between the feminine “elle” and the masculine “il,” indicating whether “it” refers to the baseball or the window. To get the translation properly, you’ll need to know both physics and language.

While AI and machine translation systems continue to improve, the goal should be seen as augmentation rather than replacement. Many applications benefit from a collaborative approach where AI tools assist human translators in their work, improving efficiency and consistency. The human touch remains crucial for nuanced, accurate, and culturally sensitive translations.

1. Designing a New Natural Language
2. Implementing a Top-Down Parser
3. Handling Questions in Context-Free Grammars
4. Semantic Interpretation in AI Systems
5. Language Model Evaluation
6. Machine Translation System Development
7. Multilingual Information Retrieval
8. Automatic Summarization for News Articles.